

Laporan Tugas Besar
IF2224 Teori Bahasa Formal dan Automata
Milestone 3 - Semantic Analysis



Disusun oleh:

13524071 Kalyca Nathania B Manullang

13524073 Keisha Rizka Syofyani

13524087 Muhammad Fakhry Zaki

13524109 Helena Kristela Sarhawa

Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Landasan Teori

1.1 Semantic Analysis

Semantic analysis merupakan salah satu tahapan pada proses kompilasi setelah Lexical Analysis dan Syntax Analysis. Semantic analysis adalah proses yang melakukan pengecekan apakah program sumber sudah memiliki makna atau aturan yang logis sesuai dengan kaidah bahasa pemrograman. Fase ini bekerja pada Abstract Syntax Tree (AST) untuk memastikan bahwa penggunaan variabel, tipe data, dan operasi sudah sesuai dengan definisi bahasa yang digunakan.

Semantic analysis ini bertujuan untuk menangkan kesalahan semantik yang tidak dapat dideteksi oleh lexer dan parser pada prosesnya, serta dapat menyediakan informasi tambahan berupa tipe data, scope, dan informasi deklarasi yang nantinya digunakan oleh tahap berikutnya pada proses kompilasi ini.

Semantic analysis memiliki beberapa perbedaan dari syntax analisis pada milestone sebelumnya. Semantic analysis lebih berfokus pada makna dan konsistensi program dibandingkan struktur kalimat berdasarkan input berupa parse tree yang dihasilkan pada tahap parsing sebelumnya. Pada semantic analysis ini dapat berpotensi beberapa kesalahan, diantaranya kesalahan tipe, variabel yang tidak dideklarasikan, serta scope yang tidak sesuai. Semantic analysis ini juga menghasilkan informasi tambahan berupa anotasi tipe dan entri symbol table ke AST.

Berikut beberapa pengecekan yang dilakukan pada tahap semantic analysis ini

1. Type Checking (Pengecekan Tipe)

Memastikan operator dan operand memiliki tipe yang kompatibel.

contoh: tidak menjumlahkan string dengan integer, perbandingan char terhadap string.

2. Symbol Table Management (Pengecekan Deklarasi)

Memastikan setiap identifier (variabel/fungsi) telah dideklarasikan sebelum digunakan dan tidak dideklarasikan ulang dalam cakupan yang sama.

3. Scope Resolution (Pengecekan Lingkup)

Menentukan validitas akses variabel berdasarkan hierarki blok kode.

Contoh: scope global vs scope local.

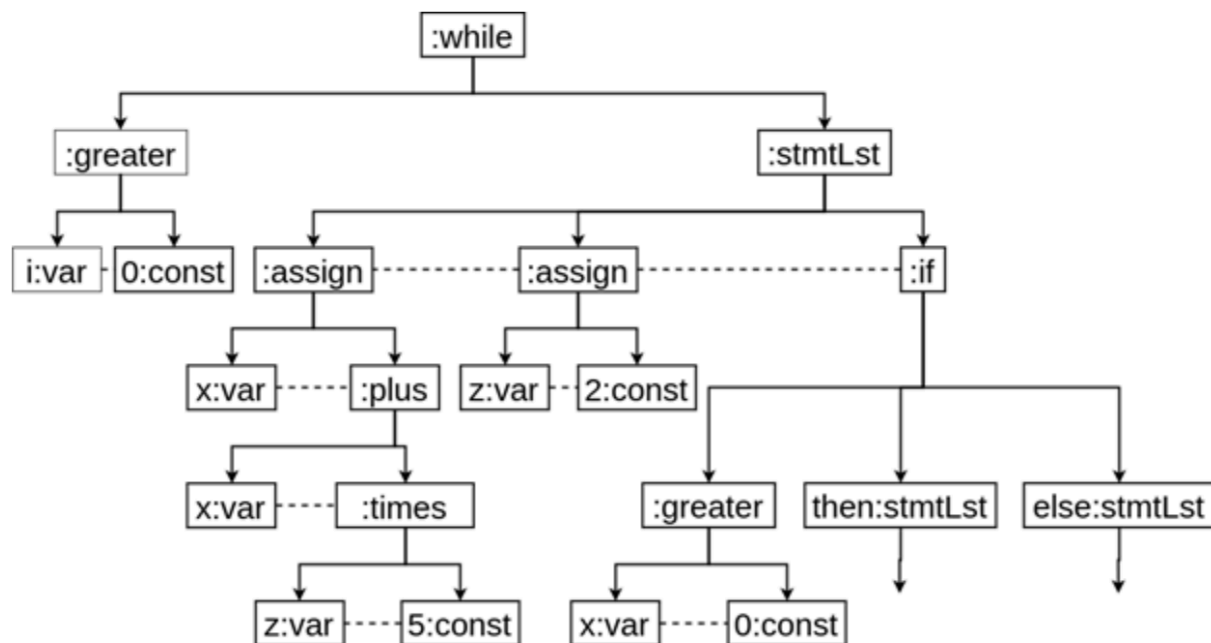
4. Control Flow Validation (Pengecekan Alur Kontrol)

Memastikan struktur alur kontrol yang logis.

Contoh: instruksi break/continue hanya dalam loop, fungsi dipastikan memiliki return pada semua cabang kontrol, warning untuk unreachable code.

Pada milestone sebelumnya, yakni semantic analysis, AST yang dihasilkan dianalisis untuk memastikan program benar secara semantik. Proses ini dilakukan dengan traversal AST menggunakan visitor pattern dan memanfaatkan symbol table untuk menyimpan informasi identifier seperti nama, tipe data, dan scope. Semantic analyzer melakukan pengecekan deklarasi identifier, validasi scope, serta type checking pada expression, assignment, dan pemanggilan prosedur atau fungsi. Hasil dari proses ini adalah Decorated AST, yaitu AST yang telah diberi informasi semantik seperti tipe data, lexical level, dan referensi symbol table untuk digunakan pada tahap compiler selanjutnya.

1.2 Abstract Syntax Tree (AST) dan Decorated AST



Abstract Syntax Tree adalah versi ringkas dan terstruktur dari parse tree. Parse Tree sering berisi node yang tidak penting seperti node untuk tanda kurung, operator, atau non-terminal yang tidak relevan, sedangkan AST hanya menyimpan informasi penting yang diperlukan untuk tahap berikutnya, terutama semantic analysis dan code generation. Oleh karena itu, proses traversal dan pengecekan semantik pada AST menjadi lebih efisien.

Pada semantic analysis ini, AST digunakan sebagai struktur utama untuk mengakses node-node penting, seperti deklarasi variabel, ekspresi, pemanggilan fungsi dan pernyataan kontrol. Selain itu, AST juga dipakai untuk menyimpan atribut yang diperlukan untuk pengecekan, seperti tipe data, scope, dan apakah suatu variabel sudah dideklarasikan. AST dimaksudkan dirancang untuk memudahkan penambahan anotasi semantik pada setiap node yang relevan.

Decorated AST (annotated AST) adalah AST yang telah diperkaya dengan atribut semantik pada setiap nodenya. Setiap node tidak hanya menyimpan struktur sintaks, tetapi juga informasi tambahan yang berguna untuk pengecekan tipe, scope dan deklarasi, serta pengambilan keputusan pada setiap tahap kompilasi selanjutnya. Beberapa contoh anotasi yang umum disimpan di Decorated AST antara lain sebagai berikut.

- type : menyimpan tipe data dari node, misalnya int, bool, string, atau function dengan tipe return dan parameter.
- lexical level (level scope) : menyimpan informasi tentang kedalaman scope atau level lingkup, misalnya global, function, loop, atau nested block. Ini berguna untuk scope checking.
- symbol table reference : menyimpan referensi ke entri di symbol table untuk node yang menyatakan identifier, misalnya node x merujuk ke entri x di symbol table.

Dengan adanya anotasi-anotasi in, Decorated AST menjadi struktur data yang siap untuk semantic analysis (type checking, scope checking, declaration checking) dan tahap kompilasi selanjutnya.

1.3 Symbol Table dan Scope Management

Symbol table adalah struktur data yang digunakan compiler untuk menyimpan dan mengelola informasi tentang identifier (variabel, fungsi, parameter, konstanta, dan lainnya) dalam sebuah program. Setiap entri symbol table biasanya berisi nama identifier, tipe data, jenis, scope / level lexical, dan informasi tambahan seperti alamat atau offset. Adapun fungsi utama symbol table dalam semantic analysis adalah untuk menyimpan hasil deklarasi identifier pada saat ditemukan di program dan juga menyediakan data untuk lookup saat identifier digunakan, sehingga compiler dapat memeriksa apakah identifier sudah dideklarasikan, mendapatkan tipe dan scopenya, serta mencegah re-deklarasi dalam scope yang sama.

Pada implementasinya, symbol table ini melibatkan proses insert dan lookup identifier. Proses insert (masukkan entri) dilakukan ketika compiler menemukan deklarasi seperti `var x : int` sehingga dibuatkan entri baru di symbol table untuk nama tersebut. Proses insert ini biasanya dilakukan pada scope terdalam yang sedang aktif, apabila terdapat nama yang sama pada scope itu maka compiler akan menandai hal tersebut sebagai kesalahan. Sedangkan proses lookup (pencarian entri) dilakukan ketika compiler melakukan pencarian untuk menemukan entri tertentu berdasarkan identifier yang digunakan. Pencarian ini dilakukan dari scope terdalam ke scope terluar sehingga identifier di scope lokal diprioritaskan jika ada nama yang sama di scope luar, sedangkan apabila tidak ditemukan sama sekali maka akan ditandai sebagai sebuah kesalahan.

Pada semantic analysis, setiap deklarasi identifier dicatat dengan level lexical-nya sehingga compiler tahu bahwa identifier masih hidup di scope saat ini dan nama mana yang akan diutamakan jika terdapat nama yang sama muncul bersamaan. Pencarian pada lexical scoping dilakukan dari yang terdalam ke yang terluar sampai entri ditemukan. Apabila sudah mencapai scope terluar dan tidak ditemukan, maka compiler menyatakan identifier belum dideklarasikan.

Berikut merupakan beberapa implementasi struktur symbol table yang saling berkombinasi pada compiler yang dapat mengelola scope, deklarasi, dan struktur kompleks seperti array.

- tab : identifier table, yang menyimpan daftar identifier beserta atributnya
- btab : block table, yang menyimpan struktur block/scope
- atab : array table, yang menyimpan informasi khusus untuk array, seperti dimensi, batas indeks, dan tipe elemen.

1.4 Type Checking dan Type Compatibility

Type checking merupakan proses pengecekan apakah penggunaan tipe data yang digunakan di dalam program sudah konsisten dan sesuai dengan aturan bahasa. Type checking ini bertujuan untuk menangkap kesalahan tipe sebelum runtime dan memastikan bahwa operasi hanya dilakukan pada tipe yang sesuai.

Type compatibility merupakan kondisi ketika suatu operasi cocok dan valid, yakni jika operand-operand kompatibel dengan tipe datanya. Misalnya, penjumlahan dua variabel dengan tipe data integer merupakan kompatibel dan akan menghasilkan nilai dengan tipe data integer juga. Di beberapa bahasa, terdapat aturan assignment compatibility, yakni ketika assignment nilai pada tipe data yang berbeda diperbolehkan. Contohnya seperti assignment nilai integer kepada variabel bertipe real, namun tidak diperbolehkan sebaliknya. Syarat dari assignment compatibility ini yaitu assignment dengan tipe yang sama, assignment tipe yang lebih kecil ke tipe yang lebih besar, dan aturan assign subtype. Selain itu, compiler juga dapat menginferensi tipe dari ekspresi berdasarkan tipe operand dan operator, ini disebut dengan type inference. Apabila kita menjumlahkan int dengan int maka akan menghasilkan tipe int juga.

1.5 Visitor Pattern / AST Traversal

Dalam implementasi milestone 3 ini, traversal AST dilakukan secara Depth Search First (DFS) menggunakan visitor pattern. Visitor pattern yaitu ketika setiap jenis node AST memiliki fungsi visitor sendiri, misalnya `visit_BinaryOp`, `visit_Assignment`, `visit_VarDecl`. Fungsi visitor ini digunakan untuk melakukan type checking, symbol lookup, serta melakukan semantic validation.

1.6 Semantic Error Handling

Semantic analyzer harus dapat mendeteksi error tanpa menghentikan program secara total pada setiap kesalahan yang terjadi. Semantic error handling merupakan proses pendeteksian kesalahan semantik pada program selama tahap semantic analysis berlangsung. Dengan pendekatan ini, compiler dapat mengumpulkan beberapa error sekaligus sehingga pengguna

memperoleh informasi kesalahan yang lebih lengkap. Beberapa contoh semantic error yang dapat diperiksa yakni:

- Undeclared variable, adanya penggunaan variabel yang belum dideklarasikan,
- Incompatible assignment, terdapat assignment dengan tipe data yang tidak kompatibel,
- Redclaration, yakni ketika adanya deklarasi ulang identifier dalam scope yang sama, serta
- Invalid parameter, yakni terjadi ketidaksesuaian jumlah atau tipe parameter pada pemanggilan prosedur atau fungsi.

Perancangan & Implementasi

2.1 Teknologi yang Digunakan

Program pada Milestone 3 ini dikembangkan menggunakan bahasa pemrograman C++11 sebagai kelanjutan langsung dari Milestone 2, sesuai dengan ketentuan spesifikasi tugas yang mewajibkan penggunaan bahasa C/C++. Pemilihan C++11 secara spesifik didasarkan pada ketersediaan fitur modern seperti *lambda expression*, *range-based for loop*, dan *initializer list* yang sangat membantu dalam penulisan kode yang ringkas dan ekspresif tanpa mengorbankan kompatibilitas kompiler lintas *platform*.

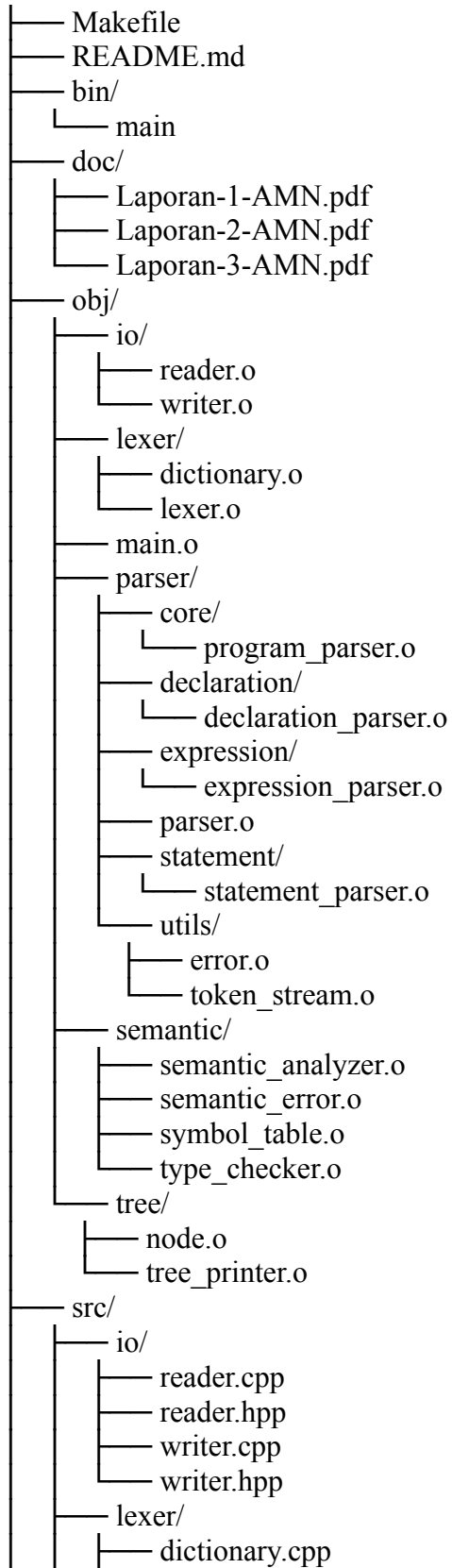
Tidak ada *library* atau *framework* eksternal tambahan yang digunakan untuk tahap semantic analysis ini. Seluruh implementasi memanfaatkan pustaka standar C++ yang meliputi string dan sstream untuk manipulasi teks dan parsing label node, vector untuk representasi tabel simbol dan daftar children node, algorithm untuk operasi case-insensitive comparison via `std::transform`, `iomanip` untuk formatting output symbol table dalam bentuk kolom yang rapi, serta `stdexcept` untuk mekanisme exception pada syntax error dari tahap parser. Keputusan untuk tidak menggunakan library eksternal ini menjamin portabilitas penuh program di berbagai lingkungan pengembangan, baik Linux/Unix maupun Windows dengan MinGW, tanpa ketergantungan tambahan yang perlu diinstall terlebih dahulu.

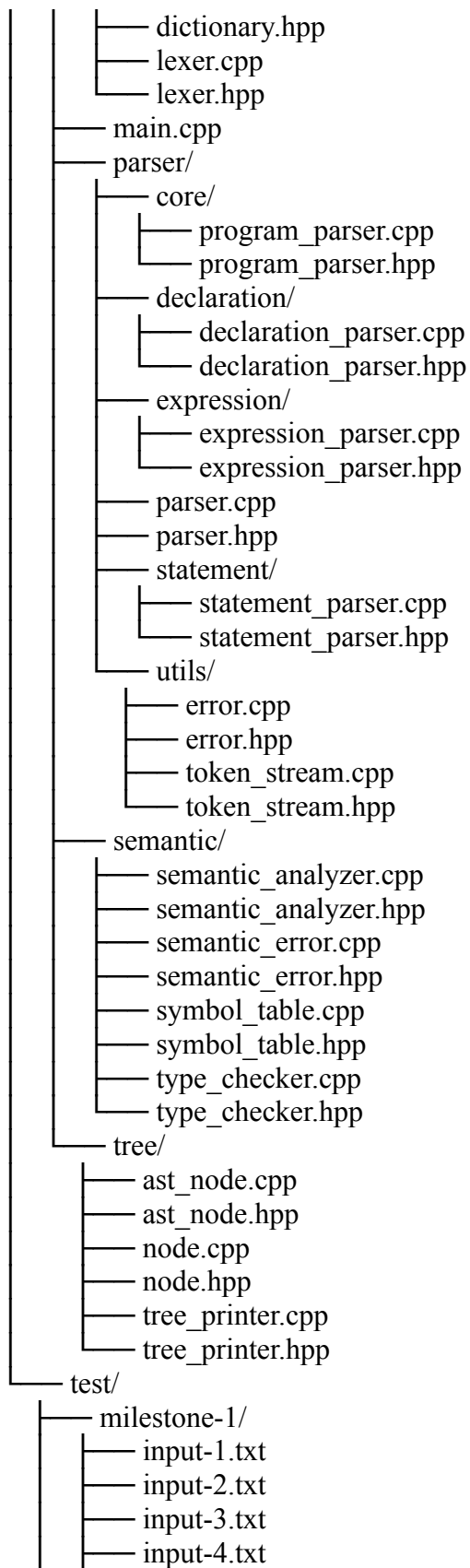
Proses kompilasi menggunakan g++ dengan flag `-std=c++11` dan `-Wall` melalui GNU Make. Makefile dikonfigurasi untuk secara otomatis menemukan semua file .cpp secara rekursif menggunakan `find`, mengkonversi path source ke path object, dan membuat direktori object secara otomatis saat dibutuhkan. Dengan konfigurasi ini, penambahan file baru di direktori semantic/ tidak memerlukan perubahan pada Makefile sama sekali.

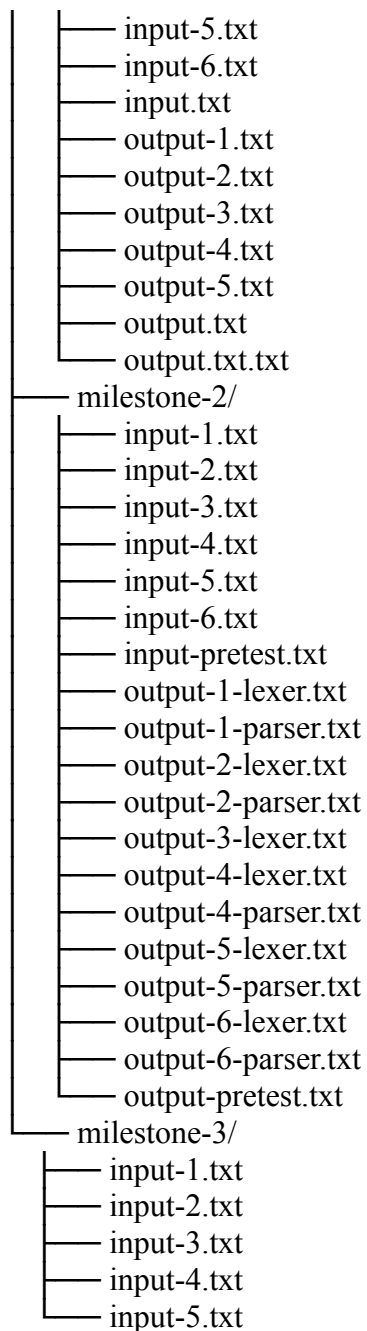
2.2 Struktur File Program

Berikut adalah struktur *folder* dan *file* program pada tugas semantic analysis ini.

amn-tubes-if2224-2026/







Implementasi Semantic Analysis pada Milestone 3 ini dikembangkan dengan menggunakan bahasa pemrograman C++ yang melanjutkan pendekatan OOP dan modular dari Milestone 2. Penambahan pada Milestone 3 berpusat pada dua area utama: direktori baru `src/semantic/` sebagai inti dari seluruh proses analisis semantik, dan penambahan file `src/tree/ast_node.cpp/.hpp` sebagai representasi AST yang terpisah dan lebih ringkas dari parse tree hasil parser. Selain penambahan file baru, beberapa file yang sudah ada dari Milestone 2

juga dimodifikasi untuk mengakomodasi kebutuhan baru: `src/tree/node.hpp/.cpp` diperluas dengan empat atribut anotasi semantik baru, `src/io/writer.hpp/.cpp` ditambahkan beberapa fungsi output baru untuk mencetak symbol table dan Decorated AST, serta `src/main.cpp` diperbarui untuk memanggil tahap semantic analysis setelah parsing selesai dan menangani outputnya.

Direktori `semantic/` sebagai inti Milestone 3 dibagi ke dalam empat submodul utama yang masing-masing memiliki tanggung jawab yang terdefinisi dengan jelas. Submodul `symbol_table` mendefinisikan tiga struct tabel simbol (`TabEntry`, `AtabEntry`, `BtabEntry`), enum `ObjClass` dan `TypeCode`, serta kelas `SymbolTable` yang mengelola seluruh operasi pada ketiga tabel tersebut termasuk manajemen scope via `enter_block()/exit_block()`, registrasi identifier via `insert()`, pencarian identifier via `lookup()` dan `lookupCurrentBlock()`, serta registrasi array via `insertArray()`. Submodul `type_checker` mendefinisikan struct `TypeInfo` sebagai representasi tipe data yang kaya informasi, beserta fungsi-fungsi pengecekan kompatibilitas tipe yaitu `isCompatible()`, `isAssignmentCompatible()`, `resultType()`, dan `resultTypeUnary()`. Submodul `semantic_analyzer` merupakan komponen terbesar yang berisi lebih dari 30 fungsi `visit_*` yang secara kolektif melakukan traversal DFS pada parse tree untuk memvalidasi semantik program, serta fungsi-fungsi `build_*_ast()` yang membangun representasi `ASTNode` secara paralel. Submodul `semantic_error` mendefinisikan `SemanticErrorCollector` yang mengakumulasi seluruh pesan error semantik selama traversal dan mencetaknya sekaligus di akhir analisis.

Untuk Decorated AST, kelompok kami mengimplementasikan dua mekanisme yang berjalan secara paralel dan saling melengkapi. Mekanisme pertama adalah dekorasi in-place pada kelas `Node` yang sudah ada: setiap fungsi `visit_*` memanggil metode `annotate()` pada node parse tree yang dikunjungi, sehingga parse tree yang sudah dihasilkan parser secara bertahap dihiasi dengan informasi semantik selama traversal berlangsung. Mekanisme kedua adalah pembangunan `ASTNode` baru secara paralel: fungsi-fungsi `build_*_ast()` yang dipanggil dari dalam fungsi `visit_*` membangun representasi AST yang lebih ringkas dan terstruktur secara semantik, membuang semua node sintaksis yang tidak relevan (seperti token `beginsy`, `endsy`, semicolon, tanda kurung berlebih) dan hanya mempertahankan informasi yang bermakna. Output Decorated AST yang dicetak ke terminal dan disimpan ke file menggunakan representasi `ASTNode` ini melalui fungsi `PRINT_AST_NEW()` dan `SAVE_AST_NEW()`, sehingga output

yang ditampilkan jauh lebih bersih, ringkas, dan mudah dibaca dibandingkan jika mencetak ulang parse tree lengkap dengan semua token sintaksis.

2.3 Predefined Identifier

Sebelum proses analisis semantik dimulai, konstruktor SymbolTable secara otomatis menginisialisasi dua kelompok entri ke dalam symbol table pada lexical level 0 (global scope). Kelompok pertama adalah reserved words yang diisi oleh fungsi initReservedWords() sebanyak 32 kata kunci bahasa Arion pada indeks 0 hingga 31. Kelompok kedua adalah predefined identifier yang diisi oleh fungsi initPredefined() mulai dari indeks 32 ke atas. Predefined identifier ini dapat digunakan langsung oleh program tanpa perlu dideklarasikan oleh pengguna karena sudah tersedia secara built-in dalam bahasa Arion. Berikut adalah daftar seluruh predefined identifier yang digunakan dalam implementasi ini:

Identifier	ObjClass	TypeCode	Nilai (adr)	Keterangan
Integer	TYPE	INTEGER	0	Tipe data bilangan bulat
Real	TYPE	REAL	0	Tipe data bilangan real atau desimal
Char	TYPE	CHAR	0	Tipe data karakter tunggal
Boolean	TYPE	BOOLEAN	0	Tipe data boolean
String	TYPE	STRING	0	Tipe data string
True	BOOLEAN	BOOLEAN	1	Konstanta boolean bernilai benar
False	BOOLEAN	BOOLEAN	0	Konstanta boolean bernilai salah
writeln	PROCEDURE	NOTYPE	0	Prosedur output dengan newline di akhir
readln	PROCEDURE	NOTYPE	0	Prosedur input dengan newline di akhir
write	PROCEDURE	NOTYPE	0	Prosedur output tanpa newline
read	PROCEDURE	NOTYPE	0	Prosedur input tanpa newline

Lima tipe bawaan (Integer, Real, Char, Boolean, String) diinisialisasi dengan `obj = TYPE` dan `type` yang bersesuaian sehingga dapat digunakan sebagai tipe data dalam deklarasi variabel, parameter, dan return type fungsi. Dua konstanta boolean (True dan False) diinisialisasi dengan `obj = CONSTANT` dan `type = BOOLEAN`; nilai boolean disimpan di field `adr` dengan True bernilai 1 dan False bernilai 0. Keempat prosedur I/O (`writeln`, `readln`, `write`, `read`) diinisialisasi dengan `obj = PROCEDURE` dan diperlakukan sebagai variadic procedure, artinya saat prosedur-prosedur ini dipanggil dalam program, validasi jumlah dan tipe argumen dilewati karena keempatnya dapat menerima jumlah dan kombinasi tipe argumen yang beragam. Meskipun validasi tipe dilewati, semua argumen yang diberikan tetap dikunjungi dan dianotasi oleh `visit_expression()` sehingga informasi semantik tetap lengkap. Semua predefined identifier dimasukkan ke dalam `chain btab[0].last` melalui field `link` sehingga membentuk linked list di global block dan dapat ditemukan oleh fungsi `lookup()` dari scope manapun, baik dari scope global maupun dari scope bersarang di dalam prosedur atau fungsi.

2.4 Semantic Action dan Production Rule

Berikut adalah semantic action yang digunakan selama proses konversi parse tree menjadi AST dan proses dekorasi untuk setiap production rule yang relevan dalam grammar bahasa Arion:

Production Rule	Semantic Action
<code><program> → programsy ident semicolon <declaration-part> <compound-statement> period</code>	<code>g_astRoot = ProgramNode(ident.value); insert(ident, PROGRAM, NOTYPE); visit_declaration_part(); visit_compound_statement();</code>
<code><const-declaration> → constsy ident eql <constant> semicolon</code>	<code>ti = visit_constant_value(constant); validate not redeclared; insert(ident, CONSTANT, ti.type, adr=ti.low); ConstDeclNode(ident)</code>
<code><type-declaration> → typesy ident eql <type> semicolon</code>	<code>ti = visit_type(type); validate not redeclared; insert(ident, TYPE, ti.baseType, ref=ti.ref); TypeDeclNode(ident)</code>
<code><var-declaration> → varsy <identifier-list> colon <type> semicolon</code>	<code>ti = visit_type(type); for each id in identifier-list: validate not redeclared; insert(id, VARIABLE, ti.baseType, ref=ti.ref); VarDeclNode(id)</code>
<code><procedure-declaration> → proceduresy ident</code>	<code>insert(ident, PROCEDURE, NOTYPE); btabIdx=enter_block(); ref=btabIdx;</code>

<formal-parameter-list> semicolon <block> semicolon	visit_formal_parameter_list(btabIdx); visit_block(); exit_block(); ProcedureDeclNode(ident)
<function-declaration> → functionsy ident <formal-parameter-list> colon ident semicolon <block> semicolon	rtIdx=lookup(retTypeId); retType=typeInfoFromTab(rtIdx); insert(ident, FUNCTION, retType.baseType, ref=0); btabIdx=enter_block(); tab[funcIdx].ref=btabIdx; visit_formal_parameter_list(btabIdx); visit_block(); exit_block(); FunctionDeclNode(ident)
<parameter-group> → <identifier-list> colon ident	idx=lookup(typeId); paramType=typeInfoFromTab(idx); for each id: validate not redeclared; insert(id, VARIABLE, paramType.baseType, paramType.ref, nrm=1, adr=offset)
<parameter-group> → <identifier-list> colon <array-type>	paramType=visit_array_type(); for each id: insert(id, VARIABLE, ARRAY, ref=atabIdx)
<type> → ident	idx=lookup(ident); validate obj==TYPE; ti=typeInfoFromTab(idx); node.annotate(ti.baseType, idx, lev)
<type> → <array-type>	ti=visit_array_type(); node.annotate(ARRAY, atabIdx, lev); return ti
<type> → <record-type>	btabIdx=enter_block(); visit_field_list(); exit_block(); node.annotate(RECORD, btabIdx, lev); return TypeInfo(RECORD, btabIdx)
<type> → <range>	ti=visit_range(); node.annotate(SUBRANGE, atabIdx, lev); return ti
<type> → <enumerated>	ti=visit_enumerated(); node.annotate(ENUMERATED, 0, lev); return ti
<array-type> → arraysy lbrack <range> rbrack ofsy <type>	indexType=visit_range(); validate isValidIndexType(indexType); elemType=visit_type(); atabIdx=insertArray(indexType.baseType, elemType.baseType, elemRef, low, high, 1); return TypeInfo(ARRAY, atabIdx)
<array-type> → arraysy lbrack ident rbrack ofsy <type>	idx=lookup(ident); indexType=typeInfoFromTab(idx); validate isValidIndexType(indexType); elemType=visit_type(); atabIdx=insertArray(...); return TypeInfo(ARRAY, atabIdx)
<record-type> → recordsy <field-list> endsy	btabIdx=enter_block(); for each field-part: fieldType=visit_type(); for each id: insert(id, FIELD,

	fieldType.baseType, fieldType.ref); exit_block(); return TypeInfo(RECORD, btabIdx, isNamed, namedId)
<range> → <constant> period period <constant>	loTi=visit_constant(lo); hiTi=visit_constant(hi); validate loTi.type==hiTi.type; validate not REAL; validate lo<=hi; atabIdx=insertArray(loTi.baseType, loTi.baseType, 0, loVal, hiVal, 1); return TypeInfo(SUBRANGE, atabIdx, loVal, hiVal)
<enumerated> → lparent ident (comma ident)* rparent	ordinal=0; for each ident: validate not redeclared; insert(ident, CONSTANT, ENUMERATED, adr=ordinal); ordinal++; return TypeInfo(ENUMERATED)
<assignment-statement> → <variable> becomes <expression>	lhsType=visit_variable(var); rhsType=visit_expression(expr); validate isAssignmentCompatible(lhsType, rhsType); AssignNode(build_variable_ast(var), build_expression_ast(expr))
<if-statement> → ifsy <expression> thensy <statement> (elsesy <statement>)?	condType=visit_expression(expr); validate condType==BOOLEAN; IfNode(ConditionNode(build_expr), ThenNode(visit_stmt), [ElseNode(visit_stmt)])
<while-statement> → whilesy <expression> dosy <compound-statement>	condType=visit_expression(expr); validate condType==BOOLEAN; WhileNode(ConditionNode(build_expr), visit_compound_statement())
<repeat-statement> → repeatsy <statement-list> untilsy <expression>	visit_statement_list(); condType=visit_expression(expr); validate condType==BOOLEAN; RepeatNode(BlockNode(stmts), ConditionNode(build_expr))
<for-statement> → forsy ident becomes <expr1> (tosy/downtosy) <expr2> dosy <compound-statement>	loopType=lookup(ident); validate isOrdinal(loopType); initType=visit_expression(expr1); finalType=visit_expression(expr2); validate isAssignmentCompatible(loopType,initType) and isAssignmentCompatible(loopType,finalType); ForNode(AssignNode, LimitNode(direction, build_expr2), visit_compound_statement())
<case-statement> → casesy <expression> ofsy <case-list> endsy	exprType=visit_expression(); validate isOrdinal(exprType); CaseNode(ExprNode, visit_case_list(exprType))
<case-branch> → <constant-list> colon <statement>	for each const in constList: constType=visit_constant_value(const); validate isCompatible(constType, exprType);

		CaseBlockNode(constNodes, visit_statement())
<expression> → <simple-expression> relop <simple-expression>		leftTi=visit_simple_expression(left); rightTi=visit_simple_expression(right); validate isCompatible(left,right) OR (left.isNumeric() AND right.isNumeric()); node.annotate(BOOLEAN, -1, lev); return TypeInfo(BOOLEAN)
<expression> → <simple-expression>		ti=visit_simple_expression(); node.annotate(ti.baseType, ti.ref, lev); return ti
<simple-expression> → unaryop <term>		operandTi=visit_term(); ti=resultTypeUnary(op, operandTi); validate ti!=NOTYPE; node.annotate(ti.baseType, ti.ref, lev); return ti
<simple-expression> → <term> (addop <term>)*		current=visit_term(first); for each (op, term): next=visit_term(term); validate op-specific rules; current=resultType(op, current, next); node.annotate(current.baseType, current.ref, lev); return current
<term> → <factor> (mulop <factor>)*		current=visit_factor(first); for each (op, factor): next=visit_factor(factor); validate op-specific rules; current=resultType(op, current, next); node.annotate(current.baseType, current.ref, lev); return current
<factor> → intcon		node.annotate(INTEGER, -1, lev); return TypeInfo(INTEGER, low=val, high=val)
<factor> → realcon		node.annotate(REAL, -1, lev); return TypeInfo(REAL)
<factor> → charcon		node.annotate(CHAR, -1, lev); return TypeInfo(CHAR)
<factor> → string		node.annotate(STRING, -1, lev); return TypeInfo(STRING)
<factor> → ident		idx=lookup(ident); validate idx>=0; ti=typeInfoFromTab(idx); node.annotate(ti.baseType, idx, lev); return ti
<factor> → notsy <factor>		operand=visit_factor(); validate operand.type==BOOLEAN; node.annotate(BOOLEAN, -1, lev); return TypeInfo(BOOLEAN)
<factor> → lparent <expression> rparent		ti=visit_expression(); node.annotate(ti.baseType, ti.ref, lev); return ti
<factor> → ident lparent		idx=lookup(ident); validate obj==FUNCTION; visit and

<parameter-list> rparent	validate args; ti=typeInfoFromTab(idx); node.annotate(ti.baseType, idx, lev); return ti
<variable> → ident	idx=lookup(ident); validate obj!=TYPE and obj!=PROCEDURE; ti=typeInfoFromTab(idx); node.annotate(ti.baseType, idx, lev); return ti
<variable> → <variable> lbrack <expression> rbrack	validate baseType==ARRAY; idxTi=visit_expression(); validate isCompatible(idxTi, atab[ref].xtyp); elemType=TypeInfo(atab[ref].etyp, atab[ref].eref); node.annotate(elemType.baseType, elemType.ref, lev); return elemType
<variable> → <variable> period ident	validate baseType==RECORD; fieldIdx=searchChain(btab[ref].last, ident); validate fieldIdx>=0; fieldType=typeInfoFromTab(fieldIdx); node.annotate(fieldType.baseType, fieldIdx, lev); return fieldType

2.5 Type Compability Rule

Berikut adalah aturan kompatibilitas tipe yang diimplementasikan dalam bahasa Arion pada milestone ini. Aturan-aturan ini menentukan operasi dan assignment mana yang diperbolehkan antara nilai dan variabel bertipe tertentu.

a. Aturan Assignment-Compatible

Suatu nilai bertipe T2 dikatakan assignment-compatible dengan variabel bertipe T1 jika memenuhi salah satu kondisi berikut:

T1 (target)	T2 (nilai)	Keterangan
REAL	INTEGER	Implicit widening: integer dapat di-assign ke real tanpa konversi eksplisit
REAL	SUBRANGE dengan base INTEGER	Implicit widening untuk subrange integer
T1 compatible dengan T2	T1 compatible dengan T2	Berlaku semua aturan compatible di atas
SUBRANGE [lo..hi]	INTEGER	Integer dapat di-assign ke subrange (validasi batas tidak dilakukan saat compile time)
SUBRANGE T1	SUBRANGE T2	Subrange ke subrange dengan base type

	dengan base type sama	yang sama
STRING	STRING	String ke string

b. Aturan Tipe per Konteks Node

Berikut adalah aturan tipe yang berlaku untuk masing-masing konteks penggunaan dalam program:

Konteks	Aturan Tipe yang Diberlakukan
Kondisi if	Ekspresi kondisi wajib bertipe BOOLEAN, tipe lain menghasilkan semantic error
Kondisi while	Ekspresi kondisi wajib bertipe BOOLEAN
Kondisi until pada repeat	Ekspresi kondisi wajib bertipe BOOLEAN
Variabel loop for	Wajib bertipe ordinal (INTEGER, CHAR, BOOLEAN, SUBRANGE, ENUMERATED)
Ekspresi batas for (initial dan final)	Wajib assignment-compatible dengan tipe variabel loop
Selektor case	Wajib bertipe ordinal
Label konstanta dalam case	Wajib compatible dengan tipe selektor
Index array saat subscript	Wajib bertipe ordinal dan compatible dengan tipe indeks array (xtyp di atab)
Tipe indeks dalam deklarasi array	Wajib ordinal, tidak boleh REAL atau STRING
Batas bawah dan atas subrange	Tidak boleh REAL, tipe batas kiri dan kanan wajib sama, lower bound tidak boleh lebih besar dari upper bound
Operand not	Wajib bertipe BOOLEAN
Kedua operand and, or	Wajib bertipe BOOLEAN, tipe lain menghasilkan error
Kedua operand div, mod	Wajib bertipe INTEGER atau SUBRANGE integer, tipe lain menghasilkan error
Kedua operand / (rdiv	Wajib numerik (INTEGER, REAL, atau SUBRANGE integer), hasil selalu REAL
Kedua operand +, -, *	Wajib numerik, hasil REAL jika salah satu REAL, selain

	itu INTEGER
Kedua operand operator relasional	Wajib compatible satu sama lain atau keduanya numerik, hasil selalu BOOLEAN
Akses field .	Variabel kiri wajib bertipe RECORD, field yang diakses wajib ada dalam record tersebut
Assignment LHS dan RHS	RHS wajib assignment-compatible dengan LHS

c. Aturan Tipe untuk Structured Type

Structured Type	Aturan
ARRAY	Tipe indeks wajib ordinal (tidak boleh REAL atau STRING), tipe elemen boleh berupa tipe apapun termasuk tipe komposit
RECORD named	Dua variabel bertipe named record yang sama (merujuk type declaration yang sama) adalah compatible dan dapat saling di-assign
RECORD anonymous	Dua anonymous record tidak pernah compatible meskipun deklarasi field-nya identik
SUBRANGE	Tipe dasar tidak boleh REAL, lower bound wajib lebih kecil atau sama dengan upper bound
ENUMERATED	Setiap identifier di dalam enumerated didaftarkan sebagai CONSTANT bertipe ENUMERATED dengan nilai ordinal dimulai dari 0

2.6 Implementasi Node dan ASTNode

Kelas Node: `src/tree/node.hpp` dan `src/tree/node.cpp`

```
#include "lexer/lexer.hpp"
#include "io/reader.hpp"
#include "io/writer.hpp"
#include "parser/parser.hpp"
#include "parser/utils/error.hpp"
#include "semantic/semantic_analyzer.hpp"
```

```

int main () {

    /* ===== LEXER ===== */
    INIT_DICTIONARY();
    INPUT_FILE();
    READ_ALL_FILE();
    PRINT_TOKEN_LIST();
    SAVE_TOKEN_LIST();

    /* ===== PARSER ===== */
    Node* root = nullptr;
    try{
        Parser parser(token_list);
        root = parser.parse();

        PRINT_PARSE_TREE (root, 0);

        SAVE_PARSE_TREE (root, 0);
    } catch (const SyntaxError& e) {
        std::cerr << e.what() << std::endl;
        return 0;
    }

    /* ===== SEMANTIC ANALYSIS ===== */
    if (root != nullptr) {
        SemanticContext ctx;
        analyze(root, ctx);

        // Kalo ada error ga bisa hasilin symbol table
        sama ast

        if (ctx.errors.hasErrors()) {

```

```

        std::cout << "Error" << std::endl;
        ctx.errors.printAll();
        // PRINT_SYMBOL_TABLE(ctx.st);
    } else {
        std::cout << "Success" << std::endl;
        PRINT_SYMBOL_TABLE(ctx.st);
        SAVE_SYMBOL_TABLE(ctx.st);
        // Print dan save AST (format baru:
indentation-based, tanpa box-drawing)
        // PRINT_AST(root, 0, ctx.st);
        // SAVE_AST(root, 0, ctx.st);
        PRINT_AST_NEW();
        SAVE_AST_NEW();
    }

    delete root;
}

return 0;
}

```

Kelas Node yang sudah ada sejak Milestone 2 berfungsi sebagai representasi setiap simpul pada parse tree hasil parser. Pada Milestone 3, kelas ini diperluas dengan menambahkan empat atribut anotasi semantik baru dan dua metode pendukungnya. Penambahan atribut ini dilakukan langsung pada kelas yang sama untuk mendukung pendekatan dekorasi in-place, yaitu parse tree yang dihasilkan parser langsung dihiasi dengan informasi semantik selama traversal tanpa perlu membangun struktur pohon baru yang terpisah.

Atribut `sem_type` bertipe `int` menyimpan kode tipe data hasil inferensi dalam bentuk nilai integer yang bersesuaian dengan nilai enum `TypeCode`. Nilai `-1` digunakan sebagai sentinel yang menandakan bahwa node tersebut belum dianotasi atau tipenya tidak relevan. Atribut `sem_tab_index` bertipe `int` menyimpan indeks entri di tabel `tab` yang berkaitan dengan node

tersebut, misalnya untuk node yang merepresentasikan identifier variabel, nilai ini adalah indeks entri variabel tersebut di tab. Untuk node yang tidak merujuk pada identifier tertentu (seperti node operator atau node struktural), nilainya adalah -1. Atribut `sem_lev` bertipe `int` menyimpan lexical level tempat node berada, di mana 0 berarti global scope dan angka yang lebih besar berarti scope yang lebih dalam (misalnya 1 untuk scope di dalam prosedur atau fungsi). Atribut `sem_initialized` bertipe `bool` merupakan flag yang menandai apakah variabel yang direpresentasikan oleh node ini sudah diinisialisasi dengan nilai sebelum digunakan.

```
#include "node.hpp"

Node::Node(const std::string& lbl)
    : label(lbl), sem_type(-1), sem_tab_index(-1),
  sem_lev(-1), sem_initialized(false) {}

void Node::addChild(Node* child) {
    this->children.push_back(child);
}

std::string Node::getLabel() const {
    return this->label;
}

Node::~~Node() {
    for (Node* child : children) {
        delete child;
    }
}

void Node::annotate(int typeCode, int tabIndex, int
lexLevel) {
    sem_type = typeCode;
    sem_tab_index = tabIndex;
```

```

    sem_lev      = lexLevel;
}

void Node::setInitialized(bool val) {
    sem_initialized = val;
}

bool Node::isAnnotated() const {
    return (sem_type != -1);
}

```

Konstruktor Node menginisialisasi keempat atribut semantik baru dengan nilai default: `sem_type = -1`, `sem_tab_index = -1`, `sem_lev = -1`, dan `sem_initialized = false`. Dengan demikian, setiap node yang baru dibuat secara otomatis berada dalam keadaan "belum dianotasi" dan programmer tidak perlu menginisialisasi secara manual. Metode `annotate(typeCode, tabIndex, lexLevel)` disediakan untuk mengisi ketiga atribut numerik sekaligus dalam satu pemanggilan, sehingga kode visitor menjadi lebih ringkas dan konsisten. Metode `isAnnotated()` mengembalikan `true` jika `sem_type != -1`, yang berarti node sudah mendapatkan anotasi tipe dari proses semantic analysis. Metode ini digunakan oleh writer untuk menentukan apakah sebuah node perlu dicetak beserta informasi anotasinya. Destruktor Node membersihkan memori secara rekursif dengan menghapus semua child node, sehingga cukup dengan memanggil `delete root` pada root node untuk membebaskan seluruh memori parse tree.

Kelas ASTNode: `src/tree/ast_node.hpp` dan `src/tree/ast_node.cpp`

```

#ifndef AST_NODE_HPP
#define AST_NODE_HPP

#include <memory>
#include <string>
#include <vector>

struct ASTNode {

```



```

    std::string kind;
    std::string text;
    int type;
    int tab_index;
    int lev;
    int block_index;
    std::vector<std::shared_ptr<ASTNode>> children;

    explicit ASTNode(const std::string& kindName, const
std::string& textValue = "");
    void addChild(const std::shared_ptr<ASTNode>& child);
};

std::shared_ptr<ASTNode> makeAstNode(const std::string&
kindName,
                                     const std::string&
textValue = "");

extern std::shared_ptr<ASTNode> g_astRoot;

#endif

```

ASTNode merupakan struktur data baru yang diperkenalkan pada Milestone 3 untuk merepresentasikan node pada Decorated AST yang lebih ringkas dan bermakna secara semantik dibandingkan parse tree. Berbeda dengan kelas Node yang label-nya adalah string dari grammar atau token verbatim (seperti "<assignment-statement>" atau "ident (Hello)"), ASTNode menggunakan field kind yang berisi jenis node semantik yang lebih ekspresif. Nilai kind yang digunakan antara lain: "Program" untuk root program, "Declarations" untuk kelompok deklarasi, "ConstDecl" untuk deklarasi konstanta, "TypeDecl" untuk deklarasi tipe, "VarDecl" untuk deklarasi variabel, "ProcedureDecl" untuk deklarasi prosedur, "FunctionDecl" untuk deklarasi fungsi, "Block" untuk blok compound statement, "Assign" untuk pernyataan assignment, "BinOp" untuk operasi biner, "UnaryOp" untuk operasi unary, "Var" untuk referensi variabel,

"Literal" untuk nilai literal, "ProcCall" untuk pemanggilan prosedur/fungsi, "If" untuk pernyataan if, "While" untuk pernyataan while, "For" untuk pernyataan for, "Repeat" untuk pernyataan repeat, "Case" untuk pernyataan case, "CaseBlock" untuk satu branch dalam case, "Condition" untuk ekspresi kondisi, "Then" dan "Else" untuk cabang if, "Limit" untuk batas akhir for, "Index" untuk subscript array, "Field" untuk akses field record, dan "Args" untuk daftar argumen pemanggilan.

Field text menyimpan informasi teks tambahan yang spesifik untuk setiap jenis node: untuk "Var" berisi nama variabel, untuk "Literal" berisi nilai literal sebagai string, untuk "BinOp" dan "UnaryOp" berisi simbol operator (misalnya "+", "-", "not"), untuk "Assign" berisi nama variabel target, untuk "ProcCall" berisi nama prosedur/fungsi, untuk "Limit" berisi arah iterasi ("to" atau "downto"), dan sebagainya. Kombinasi kind dan text memberikan informasi yang cukup untuk memahami makna setiap node tanpa perlu melihat child-child-nya.

```
#include "tree/ast_node.hpp"

std::shared_ptr<ASTNode> g_astRoot = nullptr;

ASTNode::ASTNode(const std::string& kindName, const
std::string& textValue)
    : kind(kindName), text(textValue), type(-1),
tab_index(-1), lev(-1), block_index(-1) {}

void ASTNode::addChild(const std::shared_ptr<ASTNode>&
child) {
    if (child) {
        children.push_back(child);
    }
}

std::shared_ptr<ASTNode> makeAstNode(const std::string&
kindName,
```

```

const std::string&
textValue) {
    return std::make_shared<ASTNode>(kindName,
textValue);
}

```

Field type, tab_index, lev, dan block_index menyimpan anotasi semantik yang diperoleh dari fungsi annotateFromParse() selama traversal, dengan nilai -1 sebagai sentinel untuk field yang belum terisi. Children dikelola menggunakan std::vector<std::shared_ptr<ASTNode>> sehingga manajemen memori dilakukan secara otomatis melalui reference counting tanpa perlu destructor manual. Global variable g_astRoot bertipe std::shared_ptr<ASTNode> menyimpan root AST sehingga dapat diakses dari fungsi writer tanpa perlu diteruskan sebagai parameter. Fungsi factory makeAstNode(kind, text) disediakan agar pembuatan node baru menjadi lebih ringkas dan konsisten di seluruh kode semantic_analyzer.cpp: cukup memanggil makeAstNode("BinOp", "+") untuk membuat node operasi penjumlahan.

2.7 Implementasi Symbol Table

SymbolTable: src/semantic/symbol_table.hpp dan src/semantic/symbol_table.cpp

```

#ifndef SYMBOL_TABLE_HPP
#define SYMBOL_TABLE_HPP

#include <string>
#include <vector>

enum class ObjClass {
    CONSTANT = 0,
    VARIABLE = 1,
    TYPE = 2,
    PROCEDURE = 3,
    FUNCTION = 4,
    PROGRAM = 5, // identifier program utama

```

```

    FIELD = 6
};

enum class TypeCode {
    NOTYPE = 0, // belum diketahui atau void
    INTEGER = 1,
    REAL = 2,
    CHAR = 3,
    BOOLEAN = 4,
    STRING = 5,
    ARRAY = 6, // detail ada di atab[ref]
    RECORD = 7, // detail ada di btab[ref]
    SUBRANGE = 8, // detail ada di atab[ref]
    ENUMERATED = 9 // detail ada di btab[ref]
};

struct TabEntry {
    std::string identifier;
    int link; // indeks ke identifier sebelumnya di blok
yang sama
    ObjClass obj;
    TypeCode type;
    int ref; // indeks ke atab (array/subrange) atau btab
(record/procedure block)
    int nrm; // 1 = normal, 0 = pass by reference
    int lev; // lexical level, kalau 0 = global
    int adr; // offset variabel/nilai konstanta/ukuran
lain
    TabEntry();
};

struct AtabEntry {

```

```

    int xtyp; // tipe indeks
    int etyp; // tipe elemen
    int eref; // ref untuk tipe elemen komposit
    int low; // batas bawah indeks
    int high; // batas atas indeks
    int elsz; // ukuran satu elemen (byte/unit memori)
    int size; // total ukuran array = (high-low+1)*elsz
    AtabEntry();
};

struct BtabEntry {
    int last; // indeks tab ke identifier terakhir yang
dideklarasikan di blok ini
    int lpar; // indeks tab ke parameter terakhir (0 jika
bukan prosedur/fungsi)
    int psize; // total ukuran parameter block
    int vsze; // total ukuran variabel lokal block
    BtabEntry();
};

class SymbolTable {
public:
    std::vector<TabEntry> tab; // tab[0..33] = reserved
words, tab[34+] = identifier user
    std::vector<AtabEntry> atab; // diakses via ref pada
TabEntry
    std::vector<BtabEntry> btab; // btab[0] = global
block
private:
    std::vector<int> display;
    int currentLevel; // lexical level saat ini
    int tabPtr; // indeks tab berikutnya yang bisa diisi

```

```

    int atabPtr; // indeks atab berikutnya
    int btabPtr; // indeks btab berikutnya
public:
    SymbolTable();
    int enter_block();
    void exit_block();
    int currentLev() const;
    int currentBlock() const;
        int insert(const std::string& id, ObjClass obj,
TypeCode type, int ref = 0, int nrm = 1, int adr = 0);
    int lookup(const std::string& id) const;
    int lookupCurrentBlock(const std::string& id) const;
        int insertArray(int xtyp, int etyp, int eref, int
low, int high, int elsz);
private:
    void initReservedWords();
    void initPredefined();
    int sizeof(TypeCode t) const;
};

std::string typeCodeToString(TypeCode tc);
std::string objClassToString(ObjClass oc);

#endif

```

Header `symbol_table.hpp` mendefinisikan seluruh tipe data yang digunakan dalam tiga tabel simbol. Enum `ObjClass` mendefinisikan tujuh kelas objek yang membedakan peran setiap identifier: `CONSTANT` untuk konstanta yang nilainya tidak berubah, `VARIABLE` untuk variabel yang nilainya dapat berubah, `TYPE` untuk nama tipe yang didefinisikan pengguna maupun predefined, `PROCEDURE` untuk prosedur tanpa nilai kembalian, `FUNCTION` untuk fungsi dengan nilai kembalian, `PROGRAM` untuk nama program utama, dan `FIELD` untuk field di dalam record. Enum `TypeCode` mendefinisikan sepuluh kode tipe yang merepresentasikan semua

tipe data yang didukung bahasa Arion: NOTYPE sebagai tipe kosong atau void, INTEGER, REAL, CHAR, BOOLEAN, STRING sebagai tipe primitif, serta ARRAY, RECORD, SUBRANGE, ENUMERATED sebagai tipe komposit atau terstruktur.

Struct TabEntry menyimpan semua informasi yang diperlukan untuk satu identifier: identifier sebagai nama string, link sebagai indeks ke identifier sebelumnya dalam linked list satu blok, obj sebagai kelas objek, type sebagai tipe dasar, ref sebagai indeks ke atab (untuk array/subrange) atau btab (untuk record/prosedur/fungsi), nrm sebagai flag mode passing parameter (1 = by value, 0 = by reference), lev sebagai lexical level deklarasi, dan adr yang maknanya berbeda tergantung kelas objek. Struct AtabEntry menyimpan metadata untuk satu entri array atau subrange: xtyp sebagai kode tipe indeks, etyp sebagai kode tipe elemen, eref sebagai referensi elemen komposit (untuk array multidimensi), low dan high sebagai batas, elsz sebagai ukuran satu elemen, dan size sebagai total ukuran. Struct BtabEntry menyimpan metadata untuk satu blok scope: last sebagai indeks tab identifier terakhir di blok (head linked list), lpar sebagai indeks tab parameter terakhir prosedur/fungsi, psze sebagai ukuran total parameter, dan vsze sebagai ukuran total variabel lokal yang diakumulasikan saat insert() dipanggil.

```
#include "symbol_table.hpp"
#include <iostream>
#include <iomanip>
#include <algorithm>

TabEntry::TabEntry() : identifier(""), link(0),
obj(ObjClass::VARIABLE), type(TypeCode::NOTYPE), ref(0), nrm(1),
lev(0), adr(0) {}

AtabEntry::AtabEntry() : xtyp(0), etyp(0), eref(0), low(0),
high(0), elsz(1), size(0) {}

BtabEntry::BtabEntry() : last(0), lpar(0), psze(0), vsze(0) {}

SymbolTable::SymbolTable() : currentLevel(0), tabPtr(0),
atabPtr(1), btabPtr(1)
{
```

```

    atab.push_back(AtabEntry());
    btab.push_back(BtabEntry());
    display.push_back(0);
    initReservedWords();
    initPredefined();
}

void SymbolTable::initReservedWords() {
    auto pushRW = [&](const std::string& id) {
        TabEntry e;
        e.identifier = id;
        e.link = 0;
        e.obj = ObjClass::TYPE;
        e.type = TypeCode::NOTYPE;
        e.ref = 0;
        e.nrm = 1;
        e.lev = 0;
        e.adr = 0;
        tab.push_back(e);
        tabPtr++;
    };

    pushRW("and");
    pushRW("array");
    pushRW("begin");
    pushRW("case");
    pushRW("const");
    pushRW("div");
    pushRW("downto");
    pushRW("do");
    pushRW("else");
    pushRW("end");
    pushRW("for");
    pushRW("function");
    pushRW("if");
    pushRW("mod");
    pushRW("not");

```



```

pushRW("of");
pushRW("or");
pushRW("procedure");
pushRW("program");
pushRW("record");
pushRW("repeat");
pushRW("integer");
pushRW("real");
pushRW("boolean");
pushRW("char");
pushRW("string");
pushRW("then");
pushRW("to");
pushRW("type");
pushRW("until");
pushRW("var");
pushRW("while");
// pushRW("true");
// pushRW("false");
}

```

Konstruktor `SymbolTable` melakukan tiga inisialisasi sebelum memanggil `initReservedWords()` dan `initPredefined()`. Pertama, satu `AtabEntry` dummy dimasukkan ke `atab` pada indeks 0 agar semua indeks `atab` yang valid dimulai dari 1 (indeks 0 berarti "tidak ada referensi"). Kedua, satu `BtabEntry` kosong dimasukkan ke `btab` pada indeks 0 sebagai representasi global block yang akan diisi oleh identifier-identifier global. Ketiga, `display[0] = 0` diinisialisasi agar pencarian scope pada level 0 selalu menunjuk ke global block `btab[0]`.

Fungsi `initReservedWords()` mengisi indeks 0 hingga 31 di `tab` dengan tepat 32 reserved words bahasa Arion dalam urutan tetap menggunakan lambda lokal `pushRW`. Setiap reserved word dimasukkan dengan `obj = TYPE` dan `type = NOTYPE` hanya sebagai placeholder untuk menjamin bahwa indeks user identifier selalu dimulai dari angka 32 ke atas, sehingga ada pemisahan yang jelas antara reserved words dan user-defined identifier di dalam tabel. Urutan reserved words yang dimasukkan adalah: `and`, `array`, `begin`, `case`, `const`, `div`, `downto`, `do`, `else`,

end, for, function, if, mod, not, of, or, procedure, program, record, repeat, integer, real, boolean, char, string, then, to, type, until, var, while.

```
void SymbolTable::initPredefined() {
    auto pushType = [&](const std::string& id, TypeCode
tc) {
        TabEntry e;
        e.identifier = id;
        e.link = btab[0].last;
        e.obj = ObjClass::TYPE;
        e.type = tc;
        e.ref = 0;
        e.nrm = 1;
        e.lev = 0;
        e.adr = 0;
        tab.push_back(e);
        btab[0].last = tabPtr;
        tabPtr++;
    };
    auto pushConst = [&](const std::string& id, TypeCode
tc, int val) {
        TabEntry e;
        e.identifier = id;
        e.link = btab[0].last;
        e.obj = ObjClass::CONSTANT;
        e.type = tc;
        e.ref = 0;
        e.nrm = 1;
        e.lev = 0;
        e.adr = val;
        tab.push_back(e);
        btab[0].last = tabPtr;
```

```

        tabPtr++;
    };
    auto pushProc = [&](const std::string& id) {
        TabEntry e;
        e.identifier = id;
        e.link = btab[0].last;
        e.obj = ObjClass::PROCEDURE;
        e.type = TypeCode::NOTYPE;
        e.ref = 0;
        e.nrm = 1;
        e.lev = 0;
        e.adr = 0;
        tab.push_back(e);
        btab[0].last = tabPtr;
        tabPtr++;
    };
    pushType("Integer", TypeCode::INTEGER);
    pushType("Real", TypeCode::REAL);
    pushType("Char", TypeCode::CHAR);
    pushType("Boolean", TypeCode::BOOLEAN);
    pushType("String", TypeCode::STRING);
    pushConst("True", TypeCode::BOOLEAN, 1);
    pushConst("False", TypeCode::BOOLEAN, 0);
    pushProc("writeln");
    pushProc("readln");
    pushProc("write");
    pushProc("read");
}

int SymbolTable::enter_block() {
    currentLevel++;
    BtabEntry b;

```

```

    b.last = 0;
    b.lpar = 0;
    b.psize = 0;
    b.vsize = 0;
    btab.push_back(b);
    btabPtr++;
    int newBlockIdx = static_cast<int>(btab.size()) - 1;
    if (currentLevel < static_cast<int>(display.size()))
{
    display[currentLevel] = newBlockIdx;
} else {
    display.push_back(newBlockIdx);
}
return newBlockIdx;
}

void SymbolTable::exit_block() {
    if (currentLevel > 0) {
        currentLevel--;
    }
}

```

Fungsi `initPredefined()` mengisi predefined identifier menggunakan tiga lambda lokal. Lambda `pushType` memasukkan tipe bawaan dengan `obj = TYPE`. Lambda `pushConst` memasukkan konstanta bawaan dengan `obj = CONSTANT` dan nilai di `adr`. Lambda `pushProc` memasukkan prosedur bawaan dengan `obj = PROCEDURE`. Setiap entri yang dimasukkan dihubungkan ke chain `btab[0].last` melalui field `link`, membentuk linked list identifier di global block yang dapat ditemukan oleh `lookup()` dari scope manapun. Urutan pemasukan mempengaruhi urutan pencarian: entri yang terakhir dimasukkan menjadi head dari linked list (`btab[0].last`) dan akan ditemukan pertama kali saat pencarian.

Fungsi `enter_block()` menambah `currentLevel` lalu membuat `BtabEntry` baru dengan semua field diinisialisasi nol dan menambahkannya ke vektor `btab`. Indeks blok baru ini (`btab.size() - 1`) disimpan ke `display[currentLevel]`, mengganti atau memperluas array `display` jika perlu. Fungsi mengembalikan indeks blok baru agar caller dapat menyimpannya untuk referensi ke depan. Fungsi `exit_block()` hanya mengurangi `currentLevel` tanpa menghapus data apapun dari `btab` atau `tab`. Keputusan ini sangat penting: data blok yang sudah di-exit tetap dipertahankan karena masih dibutuhkan untuk validasi akses field record (field-field record disimpan di blok yang sudah di-exit) dan akan dibutuhkan untuk code generation pada milestone berikutnya. Identifier di scope yang sudah di-exit tidak dapat ditemukan oleh `lookup()` biasa karena `display[level]` sudah tidak menunjuk ke blok tersebut, tetapi data fisiknya tetap ada.

```
int SymbolTable::insert(const std::string& id, ObjClass
obj, TypeCode type, int ref, int nrm, int adr) {
    int blk = currentBlock();
    TabEntry e;
    e.identifier = id;
    e.link = btab[blk].last;
    e.obj = obj;
    e.type = type;
    e.ref = ref;
    e.nrm = nrm;
    e.lev = currentLevel;
    e.adr = adr;
    tab.push_back(e);
    int idx = tabPtr;
    btab[blk].last = idx;
    tabPtr++;

    if (obj == ObjClass::VARIABLE || obj ==
ObjClass::FIELD) {
        btab[blk].vsze += sizeof(type);
    }
    return idx;
}
```

```

}

int SymbolTable::lookup(const std::string& id) const {
    std::string idLower = id;
    std::transform(idLower.begin(), idLower.end(),
idLower.begin(), ::tolower);

    for (int lv = currentLevel; lv >= 0; lv--) {
        int blk = display[lv];
        int k = btab[blk].last;
        while (k != 0) {
            std::string tabIdLower = tab[k].identifier;
            std::transform(tabIdLower.begin(),
tabIdLower.end(), tabIdLower.begin(), ::tolower);
            if (tabIdLower == idLower) return k;
            k = tab[k].link;
        }
    }

    for (int i = 0; i < 32 && i < (int)tab.size(); i++) {
        std::string tabIdLower = tab[i].identifier;
        std::transform(tabIdLower.begin(),
tabIdLower.end(), tabIdLower.begin(), ::tolower);
        if (tabIdLower == idLower) return i;
    }

    return -1;
}

int SymbolTable::lookupCurrentBlock(const std::string&
id) const {
    std::string idLower = id;

```

```

        std::transform(idLower.begin(), idLower.end(),
idLower.begin(), ::tolower);
        int blk = currentBlock();
        int k = btab[blk].last;
        while (k != 0) {
            std::string tabIdLower = tab[k].identifier;
            std::transform(tabIdLower.begin(),
tabIdLower.end(), tabIdLower.begin(), ::tolower);
            if (tabIdLower == idLower) return k;
            k = tab[k].link;
        }
        return -1;
    }

int SymbolTable::insertArray(int xtyp, int etyp, int
eref, int low, int high, int elsz) {
    AtabEntry a;
    a.xtyp = xtyp;
    a.etyp = etyp;
    a.eref = eref;
    a.low = low;
    a.high = high;
    a.elsz = elsz;
    a.size = (high - low + 1) * elsz;
    atab.push_back(a);
    int idx = atabPtr;
    atabPtr++;
    return idx;
}

```

Fungsi insert() membuat TabEntry baru dengan semua atribut yang diberikan. Field link diisi dengan nilai btab[currentBlock].last saat ini, yaitu indeks identifier yang sebelumnya

menjadi head dari linked list blok ini. Kemudian `btab[currentBlock].last` diperbarui ke indeks entri baru sehingga entri baru menjadi head baru dari linked list. Efeknya adalah setiap identifier baru "mendorong" dirinya sendiri ke bagian depan linked list blok, dan pencarian selanjutnya akan menemukan identifier terbaru terlebih dahulu. Jika objek yang diinsert adalah VARIABLE atau FIELD, `btab[currentBlock].vsze` diakumulasikan dengan ukuran tipe via `sizeof()` yang mengembalikan ukuran dalam unit memori (misalnya 1 untuk INTEGER, 2 untuk REAL, 4 untuk STRING). Fungsi mengembalikan indeks entri baru di tab agar caller dapat menggunakannya untuk anotasi node parse tree atau ASTNode.

Fungsi `lookup()` melakukan pencarian dari lexical level terdalam (`currentLevel`) hingga level terluar (0) dengan mengikuti rantai link pada tiap blok. Untuk setiap level `lv`, pencarian dimulai dari `btab[display[lv]].last` dan mengikuti `tab[k].link` sampai `k == 0`. Pencarian bersifat case-insensitive dengan mengkonversi kedua string ke lowercase menggunakan `std::transform` sebelum perbandingan, sesuai dengan sifat bahasa Arion yang case-insensitive. Jika tidak ditemukan di chain display manapun, pencarian dilanjutkan ke indeks 0 hingga 31 (reserved words area) sebagai fallback. Fungsi `lookupCurrentBlock()` bekerja sama dengan `lookup()` namun hanya menelusuri chain dari `btab[currentBlock].last` saja, tanpa menelusuri scope yang lebih luar. Fungsi ini digunakan khusus untuk deteksi redeclaration: suatu identifier yang sudah ada di scope yang lebih luar tidak dianggap konflik, hanya yang ada di blok yang sama persis. Fungsi `insertArray()` membuat `AtabEntry` baru dengan semua metadata yang diberikan, menghitung `size = (high - low + 1) * elsz` secara otomatis, menambahkannya ke vektor `atab`, dan mengembalikan indeks entri baru.

2.8 Implementasi Type Checker

TypeInfo dan Type Checker: `src/semantic/type_checker.hpp` dan `src/semantic/type_checker.cpp`

```
#ifndef TYPE_CHECKER_HPP
#define TYPE_CHECKER_HPP

#include "symbol_table.hpp"
#include <string>
```



```

struct TypeInfo {
    TypeCode baseType;
    int ref;
    int low;
    int high;
    TypeCode subrangeBaseType;
    bool isNamed;
    std::string namedId;

    TypeInfo();
    TypeInfo(TypeCode tc, int r = 0, int lo = 0, int hi =
0,
            bool named = false, const std::string& nid =
"");

    bool isNumeric() const;
    bool isOrdinal() const;
    bool isSimple() const;
    bool isReal() const;
    bool isInteger() const;
    bool isBoolean() const;
    bool isChar() const;
    bool isString() const;
    bool isArray() const;
    bool isRecord() const;
    bool isSubrange() const;
    bool isEnumerated() const;

    std::string toString() const;
};

```

```

bool isCompatible(const TypeInfo& t1, const TypeInfo& t2,
const SymbolTable& st);

bool isAssignmentCompatible(const TypeInfo& t1, const
TypeInfo& t2, const SymbolTable& st);

TypeInfo resultType(const std::string& op,
                    const TypeInfo& left,
                    const TypeInfo& right,
                    const SymbolTable& st);

TypeInfo resultTypeUnary(const std::string& op,
                        const TypeInfo& operand);

TypeInfo typeInfoFromTab(int tabIdx, const SymbolTable&
st);

bool isValidIndexType(const TypeInfo& t);

bool isValidSubrangeType(const TypeInfo& t);

bool isBooleanType(const TypeInfo& t);

bool isOrdinalType(const TypeInfo& t);

#endif

```

Header `type_checker.hpp` mendefinisikan struct `TypeInfo` beserta seluruh fungsi pengecekan tipe. `TypeInfo` adalah struct yang membawa informasi tipe lengkap dari suatu ekspresi atau identifier selama traversal. Dibandingkan hanya menyimpan `TypeCode` saja, `TypeInfo` menyimpan informasi yang jauh lebih kaya untuk memungkinkan validasi tipe yang akurat dan informatif. Field `ref` menyimpan indeks ke `atab` (untuk array/subrange) atau `btab`

(untuk record) sehingga detail tipe komposit dapat diakses kapanpun selama traversal tanpa perlu melakukan lookup ulang ke symbol table. Field low dan high menyimpan batas subrange atau batas indeks array secara langsung agar validasi nilai literal dalam subrange dapat dilakukan dengan cepat tanpa mengakses atab lagi. Field subrangeBaseType sangat penting untuk perbandingan kompatibilitas subrange: misalnya [1..10] memiliki baseType = SUBRANGE namun subrangeBaseType = INTEGER, sehingga perbandingan kompatibilitas dapat dilakukan terhadap INTEGER langsung tanpa kerancuan. Field isNamed dan namedId adalah kunci untuk aturan incompatibility antara anonymous record: dua TypeInfo bertipe RECORD dianggap compatible hanya jika keduanya isNamed = true dan namedId-nya sama.

```
#include "type_checker.hpp"
#include <sstream>
#include <algorithm>

TypeInfo::TypeInfo()
    :   baseType(TypeCode::NOTYPE),   ref(0),   low(0),
high(0),
    subrangeBaseType(TypeCode::NOTYPE),
    isNamed(false), namedId("") {}

TypeInfo::TypeInfo(TypeCode tc, int r, int lo, int hi,
    bool named, const std::string& nid)
    : baseType(tc), ref(r), low(lo), high(hi),
    subrangeBaseType(TypeCode::NOTYPE),
    isNamed(named), namedId(nid) {}

bool TypeInfo::isNumeric() const {
    return baseType == TypeCode::INTEGER || baseType ==
TypeCode::REAL ||
        baseType == TypeCode::SUBRANGE;
}
```

```

bool TypeInfo::isOrdinal() const {
    return baseType == TypeCode::INTEGER ||
           baseType == TypeCode::CHAR ||
           baseType == TypeCode::BOOLEAN ||
           baseType == TypeCode::SUBRANGE ||
           baseType == TypeCode::ENUMERATED;
}

bool TypeInfo::isSimple() const {
    return baseType != TypeCode::ARRAY && baseType !=
TypeCode::RECORD;
}

bool TypeInfo::isReal() const { return baseType ==
TypeCode::REAL; }

bool TypeInfo::isInteger() const { return baseType ==
TypeCode::INTEGER || baseType == TypeCode::SUBRANGE; }

bool TypeInfo::isBoolean() const { return baseType ==
TypeCode::BOOLEAN; }

bool TypeInfo::isChar() const { return baseType ==
TypeCode::CHAR; }

bool TypeInfo::isString() const { return baseType ==
TypeCode::STRING; }

bool TypeInfo::isArray() const { return baseType ==
TypeCode::ARRAY; }

bool TypeInfo::isRecord() const { return baseType ==
TypeCode::RECORD; }

bool TypeInfo::isSubrange() const { return baseType ==
TypeCode::SUBRANGE; }

bool TypeInfo::isEnumerated() const { return baseType ==
TypeCode::ENUMERATED; }

```

```

std::string TypeInfo::toString() const {
    switch (baseType) {
        case TypeCode::NOTYPE:      return "notype";
        case TypeCode::INTEGER:    return "integer";
        case TypeCode::REAL:       return "real";
        case TypeCode::CHAR:       return "char";
        case TypeCode::BOOLEAN:    return "boolean";
        case TypeCode::STRING:     return "string";
        case TypeCode::ARRAY:      return "array";
        case TypeCode::RECORD:
            if (isNamed) return "record(" + namedId +
") ";
            return "record(anonymous) ";
        case TypeCode::SUBRANGE: {
            std::ostringstream oss;
            oss << "subrange[" << low << ".." << high <<
"] ";
            return oss.str();
        }
        case TypeCode::ENUMERATED:
            if (isNamed) return "enumerated(" + namedId +
") ";
            return "enumerated";
        default: return "unknown";
    }
}

static TypeCode effectiveBaseType(const TypeInfo& t) {
    if (t.baseType == TypeCode::SUBRANGE) {
        if (t.subrangeBaseType != TypeCode::NOTYPE)
return t.subrangeBaseType;
        return TypeCode::INTEGER;
    }
}

```

```

    }
    return t.baseType;
}

bool isCompatible(const TypeInfo& t1, const TypeInfo& t2,
const SymbolTable& st) {
    if (t1.baseType == t2.baseType) {
        if (t1.baseType == TypeCode::RECORD) {
            if (!t1.isNamed || !t2.isNamed) return false;
            return t1.namedId == t2.namedId;
        }
        if (t1.baseType == TypeCode::ENUMERATED) {
            if (!t1.isNamed || !t2.isNamed) return false;
            return t1.namedId == t2.namedId;
        }
        if (t1.baseType == TypeCode::ARRAY) {
            return t1.ref == t2.ref;
        }

        if (t1.baseType == TypeCode::SUBRANGE &&
t2.baseType == TypeCode::SUBRANGE) {
            return effectiveBaseType(t1) ==
effectiveBaseType(t2);
        }
        return true;
    }

    bool t1Sub = (t1.baseType == TypeCode::SUBRANGE);
    bool t2Sub = (t2.baseType == TypeCode::SUBRANGE);

    if (t1Sub || t2Sub) {
        TypeCode base1 = effectiveBaseType(t1);
        TypeCode base2 = effectiveBaseType(t2);
    }
}

```

```

        return base1 == base2;
    }

    return false;
}

```

Fungsi `static internal effectiveBaseType()` digunakan oleh `isCompatible()` untuk mendapatkan base type yang sesungguhnya dari sebuah `TypeInfo`. Jika `baseType == SUBRANGE` dan `subrangeBaseType != NOTYPE`, maka `subrangeBaseType` dikembalikan (misalnya `INTEGER` untuk subrange `integer`). Jika `subrangeBaseType == NOTYPE` (artinya belum diset), maka `INTEGER` dikembalikan sebagai default yang aman. Untuk tipe lain, `baseType` itu sendiri dikembalikan. Fungsi ini memastikan bahwa perbandingan antara `[1..10]` dan `Integer` menghasilkan `true` karena keduanya memiliki effective base type `INTEGER`.

Fungsi `isCompatible()` mengimplementasikan logika pengecekan kompatibilitas tipe dengan beberapa kasus yang ditangani secara berurutan. Pertama, jika kedua tipe memiliki `TypeCode` yang sama, dilakukan pengecekan lebih lanjut berdasarkan jenis tipe: untuk `RECORD` harus sama-sama `isNamed` dengan `namedId` yang sama (anonymous record tidak compatible); untuk `ENUMERATED` harus sama-sama `isNamed` dengan `namedId` yang sama; untuk `ARRAY` harus memiliki `ref` yang sama (indeks atab yang sama berarti deklarasi type yang sama); untuk `SUBRANGE` keduanya dianggap compatible jika `effectiveBaseType()` sama; untuk tipe lain (primitif) langsung dianggap compatible. Kedua, jika salah satu atau keduanya adalah `SUBRANGE`, dilakukan perbandingan `effectiveBaseType()` keduanya, sehingga subrange `integer` compatible dengan `integer` dan subrange lain berbasis `integer`.

```

bool isAssignmentCompatible(const TypeInfo& t1, const
TypeInfo& t2, const SymbolTable& st) {
    TypeInfo t1_check = t1;
    TypeInfo t2_check = t2;

    // Unwrap ARRAY types to get element type for
checking

```

```

        if (t1.baseType == TypeCode::ARRAY && t1.ref > 0 &&
t1.ref < (int)st.atab.size()) {
            const AtabEntry& atabEntry = st.atab[t1.ref];
                                t1_check      =
TypeInfo(static_cast<TypeCode>(atabEntry.ety),
atabEntry.eref);
        }

        if (t2.baseType == TypeCode::ARRAY && t2.ref > 0 &&
t2.ref < (int)st.atab.size()) {
            const AtabEntry& atabEntry = st.atab[t2.ref];
                                t2_check      =
TypeInfo(static_cast<TypeCode>(atabEntry.ety),
atabEntry.eref);
        }

        // Unwrap RECORD types (keep ref for field checking
if needed)
        if (t1.baseType == TypeCode::RECORD && t1.ref > 0) {
            // RECORD ref points to btab, which represents
the record structure
            // Keep as is for now
            t1_check = t1;
        }

        if (t2.baseType == TypeCode::RECORD && t2.ref > 0) {
            t2_check = t2;
        }

        // Check REAL compatibility with INTEGER/SUBRANGE
        if (t1_check.baseType == TypeCode::REAL &&

```



```

        (t2_check.baseType == TypeCode::INTEGER ||
t2_check.baseType == TypeCode::SUBRANGE)) {
    return true;
}

    if (isCompatible(t1_check, t2_check, st)) {
        if (t1_check.baseType == TypeCode::SUBRANGE) {
            if (t2_check.baseType == TypeCode::INTEGER ||
t2_check.baseType == TypeCode::SUBRANGE) {
                return true;
            }
        }

        if (t1_check.baseType == TypeCode::STRING &&
t2_check.baseType == TypeCode::STRING) {
            return true;
        }
        return true;
    }

    return false;
}

TypeInfo resultType(const std::string& op,
                    const TypeInfo& left,
                    const TypeInfo& right,
                    const SymbolTable& st) {

    // Unwrap ARRAY types to get element type for
checking
    TypeInfo left_check = left;
    TypeInfo right_check = right;

```

```

        if (left.baseType == TypeCode::ARRAY && left.ref > 0
&& left.ref < (int)st.atab.size()) {
            const AtabEntry& atabEntry = st.atab[left.ref];
                                left_check      =
TypeInfo(static_cast<TypeCode>(atabEntry.ety),
atabEntry.eref);
        }

        if (right.baseType == TypeCode::ARRAY && right.ref >
0 && right.ref < (int)st.atab.size()) {
            const AtabEntry& atabEntry = st.atab[right.ref];
                                right_check      =
TypeInfo(static_cast<TypeCode>(atabEntry.ety),
atabEntry.eref);
        }

        if (op == "plus" || op == "minus" || op == "times") {
            if (!left_check.isNumeric() ||
!right_check.isNumeric()) return
TypeInfo(TypeCode::NOTYPE);
            if (left_check.isReal() || right_check.isReal())
return TypeInfo(TypeCode::REAL);
            return TypeInfo(TypeCode::INTEGER);
        }

        if (op == "rdiv") {
            if (!left_check.isNumeric() ||
!right_check.isNumeric()) return
TypeInfo(TypeCode::NOTYPE);
            return TypeInfo(TypeCode::REAL);
        }

```

```

    if (op == "idiv" || op == "imod") {
        if ((left_check.baseType == TypeCode::INTEGER ||
left_check.baseType == TypeCode::SUBRANGE) &&
            (right_check.baseType == TypeCode::INTEGER ||
right_check.baseType == TypeCode::SUBRANGE)) {
            return TypeInfo(TypeCode::INTEGER);
        }
        return TypeInfo(TypeCode::NOTYPE);
    }

    if (op == "eq1" || op == "neq" ||
        op == "lss" || op == "leq" ||
        op == "gtr" || op == "geq") {
        if (isCompatible(left_check, right_check, st)) {
            return TypeInfo(TypeCode::BOOLEAN);
        }

        if ((left_check.isReal() &&
right_check.isInteger()) ||
            (left_check.isInteger() &&
right_check.isReal())) {
            return TypeInfo(TypeCode::BOOLEAN);
        }
        return TypeInfo(TypeCode::NOTYPE);
    }

    if (op == "andsy" || op == "orsy") {
        if (left_check.baseType == TypeCode::BOOLEAN &&
            right_check.baseType == TypeCode::BOOLEAN) {
            return TypeInfo(TypeCode::BOOLEAN);
        }
        return TypeInfo(TypeCode::NOTYPE);
    }
}

```

```

    return TypeInfo(TypeCode::NOTYPE);
}

TypeInfo resultTypeUnary(const std::string& op, const
TypeInfo& operand) {
    if (op == "notsy") {
        if (operand.baseType == TypeCode::BOOLEAN) return
TypeInfo(TypeCode::BOOLEAN);
        return TypeInfo(TypeCode::NOTYPE);
    }
    if (op == "plus" || op == "minus") {
        if (operand.isReal()) return
TypeInfo(TypeCode::REAL);
        if (operand.isInteger()) return
TypeInfo(TypeCode::INTEGER);
        return TypeInfo(TypeCode::NOTYPE);
    }
    return TypeInfo(TypeCode::NOTYPE);
}

TypeInfo typeInfoFromTab(int tabIdx, const SymbolTable&
st) {
    if (tabIdx < 0 || tabIdx >= (int)st.tab.size()) {
        return TypeInfo(TypeCode::NOTYPE);
    }
    const TabEntry& e = st.tab[tabIdx];
    TypeInfo ti;
    ti.baseType = e.type;
    ti.ref      = e.ref;

```

```

        if (e.type == TypeCode::ARRAY && e.ref > 0 && e.ref <
(int)st.atab.size()) {
            ti.low = st.atab[e.ref].low;
            ti.high = st.atab[e.ref].high;
        }

        if (e.type == TypeCode::SUBRANGE && e.ref > 0 &&
e.ref < (int)st.atab.size()) {
            ti.low = st.atab[e.ref].low;
            ti.high = st.atab[e.ref].high;
                                ti.subrangeBaseType =
static_cast<TypeCode>(st.atab[e.ref].etyp);
        }
        if (e.type == TypeCode::RECORD) {
            ti.isNamed = true;
            ti.namedId = e.identifier;
        }
        return ti;
    }
}

```

Fungsi `isAssignmentCompatible()` pertama-tama melakukan unwrap pada tipe `ARRAY` dan `RECORD` yang memiliki `ref` valid untuk mendapatkan tipe elemen atau tipe dasar, kemudian mengecek dua aturan khusus sebelum memanggil `isCompatible()`: pertama, jika target (`t1`) bertipe `REAL` dan sumber (`t2`) bertipe `INTEGER` atau `SUBRANGE` integer maka dianggap compatible (implicit widening); kedua, aturan kompatibilitas umum dari `isCompatible()` berlaku untuk semua kasus lainnya.

Fungsi `resultType()` menghitung tipe hasil dari operasi biner berdasarkan operator dan tipe kedua operand. Untuk operator plus, minus, times: jika salah satu numeric, keduanya harus numeric, dan hasil adalah `REAL` jika salah satu `REAL`, selain itu `INTEGER`. Untuk `rdiv`: keduanya harus numeric dan hasil selalu `REAL` tanpa pengecualian. Untuk `idiv` dan `imod`: keduanya wajib `INTEGER` atau `SUBRANGE` integer, hasil `INTEGER`. Untuk `andsy` dan `orsy`: keduanya wajib `BOOLEAN`, hasil `BOOLEAN`. Untuk operator relasional (`eql`, `neq`, `lss`, `leq`, `gtr`,

geq): operand harus compatible satu sama lain, atau keduanya numerik, hasil selalu BOOLEAN. Jika aturan tidak terpenuhi, dikembalikan TypeInfo(NOTYPE) yang akan memicu error saat dicek oleh caller. Fungsi resultTypeUnary() menangani operator notsy yang hanya valid untuk BOOLEAN dan menghasilkan BOOLEAN, serta operator plus dan minus unary yang valid untuk tipe real dan integer. Fungsi typeInfoFromTab() mengkonversi TabEntry menjadi TypeInfo lengkap: mengisi low/high dari atab untuk array dan subrange, mengisi subrangeBaseType dari atab.xtyp untuk subrange, dan mengisi isNamed/namedId dari identifier untuk record.

2.9 Implementasi Semantic Error Collection

SemanticContext dan SemanticErrorCollector: src/semantic/semantic_error.hpp dan src/semantic/semantic_analyzer.hpp

```
#ifndef SEMANTIC_ERROR_HPP
#define SEMANTIC_ERROR_HPP

#include <string>
#include <vector>
#include <iostream>

// simpen satu pesan error dan konteksnya
struct SemanticError {
    std::string message;
    int line; // sumber, 0 kalau tidak tahu
    SemanticError(const std::string& msg, int ln = 0) :
message(msg), line(ln) {}
};

// kumpulin semua error lalu cetak semuanya sekaligus
class SemanticErrorCollector {
private:
    std::vector<SemanticError> errors;
```

```

public:
    void add(const std::string& message, int line = 0);
    bool hasErrors() const;
    int count() const;
    void printAll() const;
    void clear();
};

#endif

```

SemanticErrorCollector dirancang khusus untuk mengakumulasi seluruh pesan error semantik yang ditemukan selama traversal DFS, kemudian mencetak semuanya sekaligus setelah traversal selesai. Desain ini sangat berbeda dari penanganan error pada Milestone 2 di mana SyntaxError dilempar sebagai exception dan langsung menghentikan proses parsing. Pada Milestone 3, pendekatan akumulasi dipilih karena semantic analysis adalah proses yang jauh lebih kompleks: satu error semantik (misalnya tipe yang tidak kompatibel pada satu assignment) seharusnya tidak menghalangi penemuan error lain di bagian program yang berbeda. Dengan mengakumulasi semua error, pengguna mendapatkan gambaran menyeluruh tentang semua masalah yang ada dalam satu kali proses kompilasi.

```

#include "semantic_error.hpp"

void SemanticErrorCollector::add(const std::string&
message, int line) {
    errors.emplace_back(message, line);
}

bool SemanticErrorCollector::hasErrors() const {
    return !errors.empty();
}

int SemanticErrorCollector::count() const {

```

```

        return static_cast<int>(errors.size());
    }

    void SemanticErrorCollector::printAll() const {
        if (errors.empty()) return;
        for (const SemanticError& e : errors) {
            if (e.line > 0) {
                std::cerr << "Semantic error at line " <<
e.line << ": " << e.message << std::endl;
            } else {
                std::cerr << "Semantic error: " << e.message
<< std::endl;
            }
        }
    }

    void SemanticErrorCollector::clear() {
        errors.clear();
    }

```

Strukt SemanticError menyimpan dua informasi untuk setiap error: message berupa string deskripsi kesalahan yang informatif dan spesifik (misalnya "Tipe assignment tidak kompatibel: lhs adalah integer, rhs adalah string"), dan line berupa integer nomor baris sumber di mana error terjadi. Nilai line = 0 digunakan ketika nomor baris tidak tersedia, karena parse tree yang dihasilkan parser tidak menyimpan informasi baris sumber. Metode add(message, line) menambahkan SemanticError baru ke vektor errors tanpa menginterupsi eksekusi fungsi caller sama sekali; traversal dapat terus berlanjut ke node-node berikutnya. Metode hasErrors() mengembalikan true jika vektor errors tidak kosong dan digunakan di main.cpp sebagai kondisi untuk menentukan apakah output symbol table dan AST perlu dicetak. Metode printAll() mengiterasi seluruh vektor errors dan mencetak setiap error ke stderr dengan format yang berbeda tergantung apakah line > 0 atau tidak. Metode clear() disediakan untuk menghapus semua error yang terakumulasi jika diperlukan untuk pengujian atau reset.

SemanticContext adalah struct pembungkus yang menyatukan SymbolTable dan SemanticErrorCollector ke dalam satu objek tunggal. Setiap fungsi visit_* menerima SemanticContext& sebagai parameter sehingga dapat mengakses baik symbol table (untuk lookup, insert, enter_block, exit_block) maupun error collector (untuk menambahkan error) melalui satu referensi tunggal. Pendekatan ini lebih bersih dan lebih mudah dikembangkan dibandingkan melewati dua referensi terpisah atau menggunakan global variable.

2.10 Implementasi Semantic Analyzer

Visitor Functions: src/semantic/semantic_analyzer.cpp

a. Fungsi Utilitas dan AstBuildContext

```
#include "semantic/semantic_analyzer.hpp"
#include <algorithm>
#include <sstream>

std::string nodeTokenType(const Node* node) {
    const std::string& lbl = node->label;
    std::string type = "";
    int i = 0;
    while (i < (int)lbl.size() && lbl[i] != '(') {
        type.push_back(lbl[i]);
        i++;
    }
    while (!type.empty() && type.back() == ' ')
        type.pop_back();
    return type;
}

std::string nodeTokenValue(const Node* node) {
    const std::string& lbl = node->label;
    int i = 0;
```

```

    while (i < (int)lbl.size() && lbl[i] != '(') i++;
    if (i >= (int)lbl.size()) return "";
    std::string value = "";
    int j = i + 1;
    while (j < (int)lbl.size() && lbl[j] != ')') {
        value.push_back(lbl[j]);
        j++;
    }
    return value;
}

bool isNodeType(const Node* node, const std::string& label) {
    return node->label == label;
}

Node* findChild(Node* node, const std::string& label) {
    for (Node* c : node->children) {
        if (c->label == label) return c;
    }
    return nullptr;
}

Node* findChildByTokenType(Node* node, const
std::string& tokenType) {
    for (Node* c : node->children) {
        if (nodeTokenType(c) == tokenType) return c;
    }
    return nullptr;
}

struct AstBuildContext {

```

```

    std::shared_ptr<ASTNode> declarations;
    std::vector<std::shared_ptr<ASTNode>> blockStack;
    std::shared_ptr<ASTNode> attachOverride;
    std::shared_ptr<ASTNode> pendingBlockParent;
};

static AstBuildContext g_astCtx;

static void resetAstContext() {
    g_astRoot = nullptr;
    g_astCtx.declarations.reset();
    g_astCtx.blockStack.clear();
    g_astCtx.attachOverride.reset();
    g_astCtx.pendingBlockParent.reset();
}

static void annotateFromParse(const
std::shared_ptr<ASTNode>& ast, const Node* parseNode) {
    if (!ast || !parseNode) return;
    if (parseNode->sem_type >= 0) ast->type =
parseNode->sem_type;
    if (parseNode->sem_tab_index >= 0) ast->tab_index =
parseNode->sem_tab_index;
    if (parseNode->sem_lev >= 0) ast->lev =
parseNode->sem_lev;
}

static std::shared_ptr<ASTNode> currentBlockAst() {
    if (!g_astCtx.blockStack.empty()) {
        return g_astCtx.blockStack.back();
    }
    return nullptr;
}

```

```

}

static std::shared_ptr<ASTNode> selectAttachParent() {
    if (g_astCtx.attachOverride) return
g_astCtx.attachOverride;
    if (g_astCtx.pendingBlockParent) return
g_astCtx.pendingBlockParent;
    if (!g_astCtx.blockStack.empty()) return
g_astCtx.blockStack.back();
    return g_astRoot;
}

static void attachAstNode(const
std::shared_ptr<ASTNode>& node) {
    auto parent = selectAttachParent();
    if (parent) {
        parent->addChild(node);
    }
}

static std::string opToText(const std::string& tt) {
    if (tt == "plus") return "+";
    if (tt == "minus") return "-";
    if (tt == "times") return "*";
    if (tt == "rdiv") return "/";
    if (tt == "idiv") return "div";
    if (tt == "imod") return "mod";
    if (tt == "orsy") return "or";
    if (tt == "andsy") return "and";
    if (tt == "eq1") return "=";
    if (tt == "neq") return "<>";
    if (tt == "lss") return "<";

```

```

    if (tt == "leq") return "<=";
    if (tt == "gtr") return ">";
    if (tt == "geq") return ">=";
    return tt;
}

```

Terdapat empat fungsi utilitas dasar yang dipanggil ratusan kali selama traversal. Fungsi `nodeTokenType()` mengekstrak tipe token dari label node terminal dengan memotong string pada karakter '(' dan menghapus spasi trailing: misalnya label "ident (Hello)" menghasilkan "ident", dan label "intcon (5)" menghasilkan "intcon". Fungsi ini digunakan secara ekstensif untuk mengidentifikasi jenis token pada setiap node terminal tanpa harus membandingkan string lengkap. Fungsi `nodeTokenValue()` mengekstrak nilai token yang ada di antara karakter '(' dan ')': misalnya label "ident (Hello)" menghasilkan "Hello", dan label "intcon (5)" menghasilkan "5". Fungsi ini digunakan untuk mendapatkan nilai aktual dari identifier, konstanta, atau string literal. Fungsi `findChild()` mencari dan mengembalikan child pertama dari sebuah node yang labelnya persis sama dengan string yang dicari, berguna untuk menemukan node non-terminal tertentu (seperti `<expression>` atau `<variable>`) di antara children sebuah node. Fungsi `findChildByTokenType()` mencari child pertama yang merupakan node terminal dengan tipe token tertentu, menggunakan `nodeTokenType()` untuk perbandingan; berguna misalnya untuk menemukan token ident pertama dalam sebuah node `<procedure-declaration>`.

`AstBuildContext` adalah struct internal yang menyimpan seluruh state pembangunan AST selama traversal berlangsung. Field `declarations` adalah `shared_ptr<ASTNode>` ke node "Declarations" global yang menjadi parent dari semua node deklarasi. Field `blockStack` adalah `vector<shared_ptr<ASTNode>>` yang berfungsi sebagai stack berisi node "Block" yang sedang aktif; node block paling atas dari stack selalu menjadi parent attachment default untuk statement-statement yang sedang diproses. Saat `visit_compound_statement()` dipanggil, block baru di-push ke stack; saat selesai, di-pop. Field `attachOverride` adalah `shared_ptr<ASTNode>` yang ketika bernilai non-null menggantikan parent attachment default: misalnya saat memproses body dari then-branch, `attachOverride` diset ke node "Then" agar statement yang dikunjungi menempel ke "Then" bukan ke block utama. Field

pendingBlockParent menentukan parent dari block berikutnya yang akan dibuat: misalnya saat memproses body prosedur, pendingBlockParent diset ke node "ProcedureDecl" agar block body prosedur menempel ke deklarasi prosedur yang sesuai, bukan ke block global. Fungsi helper selectAttachParent() menentukan parent yang tepat berdasarkan prioritas: attachOverride > pendingBlockParent > top of blockStack > g_astRoot. Fungsi attachAstNode() memanggil selectAttachParent() dan menambahkan node baru sebagai child dari parent yang dipilih. Selain itu, terdapat fungsi static opToText() yang mengkonversi token type operator ke simbol matematika standar: "plus" → "+", "minus" → "-", "times" → "*", "rdiv" → "/", "idiv" → "div", "imod" → "mod", "orsy" → "or", "andsy" → "and", "eq1" → "=", "neq" → "≠", dan sebagainya. Konversi ini digunakan oleh semua fungsi build_*_ast() untuk mengisi field text pada node "BinOp" dan "UnaryOp".

b. Fungsi Visit Deklarasi

```
void visit_program_header(Node* node, SemanticContext&
ctx) {
    if (!node) return;
    for (Node* c : node->children) {
        if (nodeTokenType(c) == "ident") {
            std::string name = nodeTokenValue(c);
            int idx = ctx.st.insert(name,
ObjClass::PROGRAM, TypeCode::NOTYPE);
            c->annotate((int)TypeCode::NOTYPE, idx,
ctx.st.currentLev());
            if (g_astRoot) annotateFromParse(g_astRoot,
c);
            break;
        }
    }
    node->annotate((int)TypeCode::NOTYPE, -1,
ctx.st.currentLev());
}
```

```

void visit_program(Node* node, SemanticContext& ctx) {
    if (!node) return;

    std::string programName = "program";
    for (Node* c : node->children) {
        if (c->label == "<program-header>") {
            Node* idNode = findChildByTokenType(c,
"ident");
                                if (idNode) programName =
nodeTokenValue(idNode);
            break;
        }
    }

    auto programAst = makeAstNode("Program",
programName);
    g_astRoot = programAst;

                                g_astCtx.declarations =
makeAstNode("Declarations");
    programAst->addChild(g_astCtx.declarations);

    for (Node* c : node->children) {
        if (c->label == "<program-header>")
visit_program_header(c, ctx);
        else if (c->label == "<declaration-part>")
visit_declaration_part(c, ctx);
        else if (c->label == "<compound-statement>") {
                                auto prevPending =
g_astCtx.pendingBlockParent;
                                g_astCtx.pendingBlockParent = programAst;

```

```

        visit_compound_statement(c, ctx);
        g_astCtx.pendingBlockParent = prevPending;
    }
}
node->annotate((int)TypeCode::NOTYPE, -1, 0);
}

```

Fungsi `visit_program()` adalah entry point utama yang dipanggil oleh `analyze()` dan menjadi titik awal seluruh traversal DFS. Fungsi ini pertama-tama mencari nama program dengan menelusuri node `<program-header>` dan mengekstrak nilai token ident pertama yang ditemukan. Selanjutnya dibuat root `ASTNode` bertipe "Program" dengan nama program sebagai text, dan disimpan ke global `g_astRoot`. Node "Declarations" dibuat dan ditambahkan sebagai child pertama dari root, lalu disimpan ke `g_astCtx.declarations` agar semua fungsi deklarasi dapat menempelkan node deklarasinya ke sana. Fungsi ini secara sengaja tidak memanggil `enter_block()` agar seluruh deklarasi global masuk langsung ke `btav[0]` pada lexical level 0. Jika `enter_block()` dipanggil di sini, seluruh variabel global akan ter-insert dengan `lev = 1` yang akan merusak logika pencarian scope dan menghasilkan anotasi lexical level yang salah pada seluruh Decorated AST.

Fungsi `visit_program_header()` bertugas mengekstrak nama program dan menginsertnya ke symbol table. Fungsi mengiterasi children dari node `<program-header>` dan saat menemukan token ident pertama, mengekstrak nilainya lalu memanggil `ctx.st.insert(name, ObjClass::PROGRAM, TypeCode::NOTYPE)`. Indeks tab yang dikembalikan digunakan untuk menganotasi node identifier program. Fungsi `visit_declaration_part()` mendispatch ke visitor yang sesuai berdasarkan label setiap child: `<const-declaration>` ke `visit_const_declaration()`, `<type-declaration>` ke `visit_type_declaration()`, `<var-declaration>` ke `visit_var_declaration()`, dan `<subprogram-declaration>` ke `visit_subprogram_declaration()`. Fungsi `visit_block()` menangani block dalam prosedur/fungsi yang dapat mengandung deklarasi lokal dan compound statement, mendelegasikan ke `visit_declaration_part()` dan `visit_compound_statement()` secara berurutan.


```

void      visit_const_declaration(Node*      node,
SemanticContext& ctx) {
    if (!node) return;

    int i = 0;
    int n = (int)node->children.size();

    if (i < n && nodeTokenType(node->children[i]) ==
"constsy") i++;

    while (i < n) {
        Node* identNode = node->children[i];
        if (nodeTokenType(identNode) != "ident") { i++;
continue; }

        std::string name = nodeTokenValue(identNode);
        i++;

        if (i < n && nodeTokenType(node->children[i])
== "eq1") i++;

        if (i >= n) break;

        Node* valueNode = node->children[i];
        TypeInfo ti = visit_constant_value(valueNode,
ctx);
        i++;

        if (i < n && nodeTokenType(node->children[i])
== "semicolon") i++;

        if (ctx.st.lookupCurrentBlock(name) >= 0) {

```

```

        ctx.errors.add("Identifier '" + name + "'
sudah dideklarasikan dalam scope ini");
        continue;
    }

    int adr = (ti.baseType == TypeCode::INTEGER) ?
ti.low : 0;

    int idx = ctx.st.insert(name,
ObjClass::CONSTANT, ti.baseType, ti.ref, 1, adr);
    identNode->annotate((int)ti.baseType, idx,
ctx.st.currentLev());

    if (g_astCtx.declarations) {
        auto decl = makeAstNode("ConstDecl", name);
        annotateFromParse(decl, identNode);
        auto valAst =
build_constant_ast(valueNode);
        if (valAst) decl->addChild(valAst);
        g_astCtx.declarations->addChild(decl);
    }
}

    node->annotate((int)TypeCode::NOTYPE, -1,
ctx.st.currentLev());
}

```

Fungsi `visit_const_declaration()` memproses deklarasi konstanta dengan melakukan iterasi flat (bukan rekursif) pada vector children dari node <const-declaration>. Iterasi flat diperlukan karena node ini memiliki banyak children yang bercampur antara terminal (token `constsy`, `ident`, `eql`, `semicolon`) dan non-terminal (<constant>). Setiap iterasi mencari pola: skip `constsy` di awal, baca `ident` sebagai nama konstanta, skip `eql`, evaluasi <constant> via `visit_constant_value()` yang mengembalikan `TypeInfo` dengan informasi tipe dan nilai, skip

semicolon, lalu proses satu deklarasi konstanta. Sebelum insert ke symbol table, dilakukan pengecekan redeclaration via `lookupCurrentBlock(name)`. Jika tidak ada konflik, `ctx.st.insert(name, ObjClass::CONSTANT, ti.baseType, ti.ref, 1, adr)` dipanggil di mana `adr` diisi dengan `ti.low` untuk konstanta integer (menyimpan nilai konstanta langsung) atau 0 untuk tipe lain. Node ASTNode bertipe "ConstDecl" dengan nama konstanta sebagai text dibuat dan ditambahkan ke `g_astCtx.declarations`.

```
void visit_type_declaration(Node* node,
SemanticContext& ctx) {
    if (!node) return;

    int i = 0;
    int n = (int) node->children.size();

    if (i < n && nodeTokenType(node->children[i]) ==
"typesy") i++;

    while (i < n) {
        Node* identNode = node->children[i];
        if (nodeTokenType(identNode) != "ident") { i++;
continue; }

        std::string name = nodeTokenValue(identNode);
        i++;

        if (i < n && nodeTokenType(node->children[i])
== "eq1") i++;

        if (i >= n) break;

        Node* typeNode = node->children[i];
        i++;
    }
}
```

```

        if (i < n && nodeTokenType(node->children[i])
== "semicolon") i++;

        TypeInfo ti;
        if (typeNode->label == "<type>") {
            Node* inner = typeNode->children.empty() ?
nullptr : typeNode->children[0];
            if (inner && inner->label ==
"<record-type>") {
                int btabOut = 0;
                ti = visit_record_type(inner, ctx,
name, &btabOut);
                typeNode->annotate((int)ti.baseType,
btabOut, ctx.st.currentLev());
            } else {
                ti = visit_type(typeNode, ctx);
                if (ti.baseType == TypeCode::RECORD) {
ti.isNamed = true; ti.namedId = name; }
            }
        } else {
            ti = visit_type(typeNode, ctx);
        }

        if (ctx.st.lookupCurrentBlock(name) >= 0) {
            ctx.errors.add("Tipe '" + name + "' sudah
dideklarasikan dalam scope ini");
            continue;
        }

        int idx = ctx.st.insert(name, ObjClass::TYPE,
ti.baseType, ti.ref);

```

```

        identNode->annotate((int)ti.baseType, idx,
ctx.st.currentLev());

    if (g_astCtx.declarations) {
        auto decl = makeAstNode("TypeDecl", name);
        annotateFromParse(decl, identNode);
        g_astCtx.declarations->addChild(decl);
    }
}
}

```

Fungsi `visit_type_declaration()` memproses deklarasi tipe dengan pola iterasi flat yang serupa. Kunci penting di sini adalah penanganan named record: ketika `<type>` yang diproses mengandung `<record-type>`, nama tipe yang sedang dideklarasikan (misalnya "TPoint") diteruskan ke `visit_record_type()` sebagai parameter `namedId`. Di dalam `visit_record_type()`, parameter ini digunakan untuk mengisi `TypeInfo.isNamed = true` dan `TypeInfo.namedId = namedId`. Informasi ini kemudian disimpan bersama `TabEntry` dan dapat diambil kembali via `typeInfoFromTab()` saat dua variabel bertipe record dibandingkan kompatibilitasnya. Tanpa mekanisme ini, semua record akan dianggap anonymous dan tidak pernah bisa di-assign satu sama lain.

```

void visit_var_declaration(Node* node, SemanticContext&
ctx) {
    if (!node) return;

    int i = 0;
    int n = (int)node->children.size();

    if (i < n && nodeTokenType(node->children[i]) ==
"varsy") i++;

    while (i < n) {

```

```

        Node* c = node->children[i];
        if (nodeTokenType(c) == "semicolon") { i++;
continue; }
        if (c->label != "<identifier-list>") { i++;
continue; }

        std::vector<std::string> names;
        std::vector<Node*> identNodes;
        for (Node* ic : c->children) {
            if (nodeTokenType(ic) == "ident") {
                names.push_back(nodeTokenValue(ic));
                identNodes.push_back(ic);
            }
        }
        i++;

        if (i < n && nodeTokenType(node->children[i])
== "colon") i++;

        if (i >= n) break;

        Node* typeNode = node->children[i];
        i++;

        if (i < n && nodeTokenType(node->children[i])
== "semicolon") i++;

        TypeInfo ti;
        if (typeNode->label == "<type>") {
            Node* inner = typeNode->children.empty() ?
nullptr : typeNode->children[0];

```

```

        if (inner && inner->label ==
"<record-type>") {
            int btabOut = 0;
            ti = visit_record_type(inner, ctx, "",
&btabOut);

            typeNode->annotate((int)ti.baseType,
btabOut, ctx.st.currentLev());
        } else {
            ti = visit_type(typeNode, ctx);
        }
    } else {
        ti = visit_type(typeNode, ctx);
    }

    for (int ni = 0; ni < (int)names.size(); ni++)
    {
        if (ctx.st.lookupCurrentBlock(names[ni]) >=
0) {
            ctx.errors.add("Variabel '" + names[ni]
+ "'" sudah dideklarasikan dalam scope ini");
        } else {
            int idx = ctx.st.insert(names[ni],
ObjClass::VARIABLE, ti.baseType, ti.ref);
            identNodes[ni]->annotate((int)ti.baseType, idx,
ctx.st.currentLev());

            if (g_astCtx.declarations) {
                auto decl = makeAstNode("VarDecl",
names[ni]);
                annotateFromParse(decl,
identNodes[ni]);
            }
        }
    }
}

```

```

g_astCtx.declarations->addChild(decl);
        }
    }
}
}
}
}
}

```

Fungsi `visit_var_declaration()` memproses deklarasi variabel yang dapat memiliki beberapa identifier dalam satu baris (misalnya `var x, y, z: Integer`). Iterasi flat dilakukan pada `children`, mencari pola: node `<identifier-list>` yang berisi semua nama variabel dalam satu baris, diikuti token colon, diikuti node `<type>`. Setelah tipe diketahui via `visit_type()`, loop dilakukan untuk menginsert setiap nama identifier secara terpisah dengan tipe yang sama. Penanganan khusus dilakukan untuk anonymous record: jika `<type>` mengandung `<record-type>`, dipanggil `visit_record_type()` secara langsung dengan `namedId = ""` (string kosong) sehingga `TypeInfo.isNamed` tetap false. Ini memastikan bahwa dua variabel yang dideklarasikan dengan anonymous record yang sama (misalnya `var p1, p2: record x, y: Integer; end`) akan dianggap incompatible satu sama lain.

```

void      visit_procedure_declaration(Node*      node,
SemanticContext& ctx) {
    if (!node) return;
    std::string procName = "";
    Node* fplNode = nullptr, *blockNode = nullptr;
    bool foundFirst = false;
    std::shared_ptr<ASTNode> procAst;
    for (Node* c : node->children) {
        if (nodeTokenType(c) == "ident" && !foundFirst)
        { procName = nodeTokenValue(c); foundFirst = true; }
        else if (c->label == "<formal-parameter-list>")
        fplNode = c;
        else if (c->label == "block") blockNode = c;
    }
}

```



```

    }
    if (procName.empty()) return;
    if (ctx.st.lookupCurrentBlock(procName) >= 0)
        ctx.errors.add("Prosedur '" + procName + "'
sudah dideklarasikan dalam scope ini");
        int procIdx = ctx.st.insert(procName,
ObjClass::PROCEDURE, TypeCode::NOTYPE);

    if (g_astCtx.declarations) {
        procAst = makeAstNode("ProcedureDecl",
procName);
        g_astCtx.declarations->addChild(procAst);
    }

    int btabIdx = ctx.st.enter_block();
    ctx.st.tab[procIdx].ref = btabIdx;
    if (fplNode) visit_formal_parameter_list(fplNode,
ctx, btabIdx);
    if (blockNode) {
        if (procAst) g_astCtx.pendingBlockParent =
procAst;
        visit_block(blockNode, ctx);
    }
    ctx.st.exit_block();
    Node* identNode = findChildByTokenType(node,
"ident");
    if (identNode) {
        identNode->annotate((int) TypeCode::NOTYPE,
procIdx, ctx.st.currentLev());
        if (procAst) annotateFromParse(procAst,
identNode);
    }
}

```

```

}

void      visit_function_declaration(Node*      node,
SemanticContext& ctx) {
    if (!node) return;
    std::string funcName = "", retTypeName = "";
    Node* fplNode = nullptr, *blockNode = nullptr,
*retTypeNode = nullptr;
    bool foundFirst = false;
    std::shared_ptr<ASTNode> funcAst;
    for (Node* c : node->children) {
        if (nodeTokenType(c) == "ident") {
            if (!foundFirst) { funcName =
nodeTokenValue(c); foundFirst = true; }
            else { retTypeName = nodeTokenValue(c);
retTypeNode = c; }
        } else if (c->label ==
"<formal-parameter-list>") fplNode = c;
        else if (c->label == "block") blockNode = c;
    }
    if (funcName.empty()) return;
    TypeInfo retType(TypeCode::NOTYPE);
    if (!retTypeName.empty()) {
        int rtIdx = ctx.st.lookup(retTypeName);
        if (rtIdx < 0) {
            ctx.errors.add("Return type fungsi tidak
ditemukan: '" + retTypeName + "'");
        } else {
            retType = typeInfoFromTab(rtIdx, ctx.st);
            if (retTypeNode)
retTypeNode->annotate((int) retType.baseType, rtIdx,
ctx.st.currentLev());

```

```

    }

    }

    if (ctx.st.lookupCurrentBlock(funcName) >= 0)
        ctx.errors.add("Fungsi '" + funcName + "' sudah
dideklarasikan dalam scope ini");
        int funcIdx = ctx.st.insert(funcName,
ObjClass::FUNCTION, retType.baseType, retType.ref);

    if (g_astCtx.declarations) {
        funcAst = makeAstNode("FunctionDecl",
funcName);
        g_astCtx.declarations->addChild(funcAst);
    }

    int btabIdx = ctx.st.enter_block();
    ctx.st.tab[funcIdx].ref = btabIdx;
    if (fplNode) visit_formal_parameter_list(fplNode,
ctx, btabIdx);
    if (blockNode) {
        if (funcAst) g_astCtx.pendingBlockParent =
funcAst;
        visit_block(blockNode, ctx);
    }
    ctx.st.exit_block();
    Node* identNode = findChildByTokenType(node,
"ident");
    if (identNode) {
        identNode->annotate((int) retType.baseType,
funcIdx, ctx.st.currentLev());
        if (funcAst) annotateFromParse(funcAst,
identNode);
    }
}

```

```
}
```

Fungsi `visit_procedure_declaration()` memproses deklarasi prosedur melalui beberapa langkah berurutan. Pertama, nama prosedur diekstrak dari token ident pertama dan dicek apakah sudah terdeklasikan di scope saat ini via `lookupCurrentBlock()`. Jika tidak ada konflik, prosedur diinsert ke symbol table dengan `obj = PROCEDURE` dan mendapatkan indeks `tab`. Kemudian `enter_block()` dipanggil untuk membuat scope baru bagi parameter dan variabel lokal prosedur, dan indeks `btabs` yang dikembalikan disimpan ke `ctx.st.tab[procIdx].ref`. Penyimpanan ini krusial karena nantinya `visit_procedure_function_call()` akan mengakses `btabs[tab[procIdx].ref]` untuk mendapatkan informasi parameter formal saat memvalidasi pemanggilan prosedur. Jika ada `<formal-parameter-list>`, dipanggil `visit_formal_parameter_list(btabIdx)` dalam scope baru. Kemudian `visit_block()` dipanggil untuk memproses body prosedur. Terakhir, `exit_block()` dipanggil untuk keluar dari scope prosedur.

Fungsi `visit_function_declaration()` bekerja serupa namun dengan dua perbedaan utama. Pertama, sebelum `enter_block()`, dilakukan lookup return type dari identifier yang muncul setelah tanda titik dua dalam header fungsi, dan tipe ini digunakan sebagai type pada `TabEntry` fungsi. Kedua, di dalam scope fungsi, fungsi `ctx.st.insert(funcName, ObjClass::FUNCTION, refType.baseType, ...)` dipanggil lagi (selain yang sudah diinsert di scope luar) untuk mendukung penugasan nilai return via `functionName := expression` di dalam body fungsi. Tanpa insert ini, statement seperti `CalculateSum := total` di dalam body fungsi `CalculateSum` akan menghasilkan error "identifier tidak ditemukan".

```
void      visit_formal_parameter_list(Node*      node,
SemanticContext& ctx, int btabIdx) {
    if (!node) return;
    int paramCount = 0;
    for (Node* c : node->children) {
        if (c->label != "<parameter-group>") continue;
        TypeInfo paramType (TypeCode::NOTYPE);
        std::vector<std::string> paramNames;
```

```

        std::vector<Node*> paramIdentNodes;
        for (Node* pc : c->children) {
            if (pc->label == "<identifier-list>") {
                for (Node* ic : pc->children) {
                    if (nodeTokenType(ic) == "ident") {
                        paramNames.push_back(nodeTokenValue(ic));
                        paramIdentNodes.push_back(ic);
                    }
                }
            } else if (pc->label == "<array-type>") {
                paramType = visit_array_type(pc, ctx);
            } else if (pc->label == "<type>") {
                paramType = visit_type(pc, ctx);
            } else if (nodeTokenType(pc) == "ident") {
                std::string tv = nodeTokenValue(pc);
                int idx = ctx.st.lookup(tv);
                if (idx < 0) {
                    ctx.errors.add("Tipe parameter
tidak ditemukan: '" + tv + "'");
                } else {
                    paramType = typeInfoFromTab(idx,
ctx.st);
                }
            }
            pc->annotate((int)paramType.baseType, idx,
ctx.st.currentLev());
        }
    }
    for (int ni = 0; ni < (int)paramNames.size();
ni++) {

```

```

if
    (ctx.st.lookupCurrentBlock(paramNames[ni]) >= 0) {
        ctx.errors.add("Parameter '" +
paramNames[ni] + "' sudah dideklarasikan");
    } else {
        int idx = ctx.st.insert(paramNames[ni],
ObjClass::VARIABLE,
paramType.baseType, paramType.ref, 1, paramCount);

paramIdentNodes[ni]->annotate((int)paramType.baseType,
idx, ctx.st.currentLev());
        paramCount++;
    }
}

ctx.st.btab[btabIdx].lpar =
ctx.st.btab[btabIdx].last;
ctx.st.btab[btabIdx].psize = paramCount;
}

```

Fungsi `visit_formal_parameter_list()` mengiterasi setiap node <parameter-group> di dalam <formal-parameter-list>. Untuk setiap group, fungsi mengumpulkan nama parameter dari <identifier-list> dan menentukan tipe parameter dari child berikutnya (bisa berupa token ident sebagai nama tipe, node <array-type> untuk parameter array, atau node <type> untuk tipe lain). Setiap parameter diinsert ke symbol table di dalam scope blok prosedur/fungsi yang sudah di-enter, dengan adr berisi urutan offset parameter (0, 1, 2, ...) untuk keperluan code generation. Setelah semua parameter diproses, `btab[btabIdx].lpar` diperbarui ke `btab[btabIdx].last` (indeks parameter terakhir yang menjadi head linked list parameter) dan `btab[btabIdx].psize` diisi dengan total jumlah parameter. Kedua nilai ini digunakan oleh `visit_procedure_function_call()` untuk mengumpulkan dan memvalidasi parameter formal saat prosedur/fungsi dipanggil.

c. Fungsi Visit Type

```
TypeInfo visit_array_type(Node* node, SemanticContext&
ctx) {
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    TypeInfo indexType(TypeCode::NOTYPE);
    TypeInfo elemType(TypeCode::NOTYPE);

    for (Node* c : node->children) {
        if (c->label == "<range>") {
            indexType = visit_range(c, ctx);
        } else if (c->label == "<type>") {
            elemType = visit_type(c, ctx);
        } else if (nodeTokenType(c) == "ident") {
            std::string tv = nodeTokenValue(c);
            int idx = ctx.st.lookup(tv);
            if (idx < 0) {
                ctx.errors.add("Tipe index array tidak
ditemukan: '" + tv + "'");
            } else {
                indexType = typeInfoFromTab(idx,
ctx.st);
                c->annotate((int) indexType.baseType,
idx, ctx.st.currentLev());
            }
        }
    }

    if (!isValidIndexType(indexType)) {
        ctx.errors.add("Index array harus bertipe
ordinal bukan Real: " + indexType.toString());
    }
}
```

```

        indexType = TypeInfo(TypeCode::INTEGER);
    }

    int elemRef = 0;
    if (elemType.baseType == TypeCode::ARRAY ||
elemType.baseType == TypeCode::SUBRANGE ||
elemType.baseType == TypeCode::RECORD) {
        elemRef = elemType.ref;
    }

    int atabIdx = ctx.st.insertArray(
                                (int) indexType.baseType,
(int) elemType.baseType,
        elemRef, indexType.low, indexType.high, 1
    );

    TypeInfo result(TypeCode::ARRAY, atabIdx,
indexType.low, indexType.high);
    node->annotate((int) TypeCode::ARRAY, atabIdx,
ctx.st.currentLev());
    return result;
}

TypeInfo visit_type(Node* node, SemanticContext& ctx) {
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    for (Node* c : node->children) {
        if (c->label == "<array-type>") {
            TypeInfo ti = visit_array_type(c, ctx);
            node->annotate((int) ti.baseType, ti.ref,
ctx.st.currentLev());
            return ti;
        }
    }
}

```



```

    }
    if (c->label == "<record-type>") {
        int btabOut = 0;
        TypeInfo ti = visit_record_type(c, ctx, "",
&btabOut);
        node->annotate((int)ti.baseType, btabOut,
ctx.st.currentLev());
        return ti;
    }
    if (c->label == "<range>") {
        TypeInfo ti = visit_range(c, ctx);
        node->annotate((int)ti.baseType, ti.ref,
ctx.st.currentLev());
        return ti;
    }
    if (c->label == "<enumerated>") {
        TypeInfo ti = visit_enumerated(c, ctx);
        node->annotate((int)ti.baseType, ti.ref,
ctx.st.currentLev());
        return ti;
    }
    if (nodeTokenType(c) == "ident") {
        std::string tv = nodeTokenValue(c);
        int idx = ctx.st.lookup(tv);
        if (idx < 0) {
            ctx.errors.add("Tipe tidak ditemukan:
'" + tv + "'");
            return TypeInfo(TypeCode::NOTYPE);
        }
        const TabEntry& e = ctx.st.tab[idx];
        if (e.obj != ObjClass::TYPE) {

```

```

        ctx.errors.add("'" + tv + "'" bukan
sebuah tipe");
        return TypeInfo(TypeCode::NOTYPE);
    }
    TypeInfo ti = typeInfoFromTab(idx, ctx.st);
    if (e.type == TypeCode::RECORD) {
        ti.isNamed = true;
        ti.namedId = tv;
    }

    c->annotate((int)ti.baseType, idx,
ctx.st.currentLev());
    node->annotate((int)ti.baseType, idx,
ctx.st.currentLev());
    return ti;
}
}

return TypeInfo(TypeCode::NOTYPE);
}

```

Fungsi `visit_type()` berfungsi sebagai dispatcher yang mendelegasikan ke sub-visitor yang sesuai berdasarkan label child pertama node `<type>`. Untuk `<array-type>` didelegasikan ke `visit_array_type()`; untuk `<record-type>` ke `visit_record_type()`; untuk `<range>` ke `visit_range()`; untuk `<enumerated>` ke `visit_enumerated()`; dan untuk token ident dilakukan `lookup()` untuk menemukan tipe yang sudah dideklarasikan. Pada kasus ident, validasi tambahan dilakukan bahwa identifier yang ditemukan harus memiliki `obj == TYPE` (bukan variabel atau prosedur). Jika tipe yang ditemukan adalah record, `isNamed` dan `namedId` dari `TypeInfo` diisi dari nama identifier tipe tersebut sehingga perbandingan named record dapat bekerja dengan benar.

Fungsi `visit_array_type()` memproses node `<array-type>` dengan mengiterasi children untuk menemukan tipe indeks dan tipe elemen. Tipe indeks bisa berasal dari node `<range>`

(subrange eksplisit) atau token ident (nama tipe yang sudah dideklarasikan). Setelah keduanya diketahui, isValidIndexType() dipanggil untuk memvalidasi bahwa tipe indeks adalah ordinal dan bukan REAL atau STRING. Jika tidak valid, error ditambahkan namun proses tetap berlanjut dengan menggunakan INTEGER sebagai fallback agar validasi tipe elemen dan konteks lainnya dapat terus diproses. Fungsi insertArray() kemudian dipanggil dengan tipe indeks, tipe elemen, referensi elemen komposit, batas indeks, dan ukuran elemen untuk membuat entri baru di atab dan mendapatkan indeksinya.

```
TypeInfo visit_range(Node* node, SemanticContext& ctx)
{
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    TypeInfo loTi, hiTi;
    int loVal = 0, hiVal = 0;

    std::vector<Node*> constants;
    for (Node* c : node->children) {
        if (c->label == "<constant>")
            constants.push_back(c);
    }

    bool negLo = false, negHi = false;

    auto extractConst = [&](Node* cn, bool& neg,
TypeInfo& ti, int& val) {
        neg = false;
        if (cn->children.empty()) { ti =
TypeInfo(TypeCode::NOTYPE); return; }
        int ci = 0;
        std::string ft =
nodeTokenType(cn->children[ci]);
        if (ft == "plus") { ci++; }
```

```

        else if (ft == "minus") { neg = true; ci++; }
        if (ci >= (int)cn->children.size()) { ti =
TypeInfo(TypeCode::NOTYPE); return; }
        Node* v      = cn->children[ci];
        std::string ttype = nodeTokenType(v);
        std::string tval  = nodeTokenValue(v);
        if (ttype == "intcon") {
            val = parseIntValue(tval);
            if (neg) val = -val;
            ti = TypeInfo(TypeCode::INTEGER);
            ti.low = val; ti.high = val;
        } else if (ttype == "charcon") {
            val = (int)tval[0];
            ti = TypeInfo(TypeCode::CHAR);
        } else if (ttype == "ident") {
            int idx = ctx.st.lookup(tval);
            if (idx < 0) {
                ctx.errors.add("Identifler tidak
ditemukan di range: '" + tval + "'");
                ti = TypeInfo(TypeCode::NOTYPE);
return;
            }
            ti = typeInfoFromTab(idx, ctx.st);
            val = ctx.st.tab[idx].adr;
            v->annotate((int)ti.baseType, idx,
ctx.st.currentLev());
        } else if (ttype == "realcon") {
            ctx.errors.add("Subrange tidak boleh
bertipe Real");
            ti = TypeInfo(TypeCode::NOTYPE);
        } else {
            ti = TypeInfo(TypeCode::NOTYPE);

```

```

    }

};

if (constants.size() >= 2) {
    extractConst(constants[0], negLo, loTi, loVal);
    extractConst(constants[1], negHi, hiTi, hiVal);
} else {
    ctx.errors.add("Range tidak lengkap");
    return TypeInfo(ErrorCode::NOTYPE);
}

    if (loTi.baseType == ErrorCode::NOTYPE ||
hiTi.baseType == ErrorCode::NOTYPE)
    return TypeInfo(ErrorCode::NOTYPE);

    if (loTi.baseType != hiTi.baseType) {
        ctx.errors.add("Tipe batas bawah dan atas range
harus sama: " +
                                loTi.toString() + " vs " +
hiTi.toString());
        return TypeInfo(ErrorCode::NOTYPE);
    }

    if (!isValidSubrangeType(loTi)) {
        ctx.errors.add("Subrange tidak boleh bertipe
Real atau komposit");
        return TypeInfo(ErrorCode::NOTYPE);
    }

    if (loVal > hiVal) {
        ctx.errors.add("Batas bawah range tidak boleh
lebih besar dari batas atas: " +

```

```

                                std::to_string(loVal) + " > " +
std::to_string(hiVal));
    }

    int atabIdx = ctx.st.insertArray(
        (int)loTi.baseType, (int)loTi.baseType,
        0, loVal, hiVal, 1
    );

    TypeInfo result(TypeCode::SUBRANGE, atabIdx, loVal,
hiVal);
    result.subrangeBaseType = loTi.baseType;
    node->annotate((int)TypeCode::SUBRANGE, atabIdx,
ctx.st.currentLev());
    return result;
}

TypeInfo visit_record_type(Node* node, SemanticContext&
ctx, const std::string& namedId, int* btabIdxOut) {
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    int btabIdx = ctx.st.enter_block();
    if (btabIdxOut) *btabIdxOut = btabIdx;

    for (Node* c : node->children) {
        if (c->label != "<field-list>") continue;
        for (Node* fp : c->children) {
            if (fp->label != "<field-part>") continue;

            TypeInfo fieldType(TypeCode::NOTYPE);
            std::vector<std::string> fieldNames;
            std::vector<Node> fieldIdentNodes;

```

```

        for (Node* fc : fp->children) {
            if (fc->label == "<identifier-list>") {
                for (Node* ic : fc->children) {
                    if (nodeTokenType(ic) ==
"ident") {

fieldNames.push_back(nodeTokenValue(ic));

fieldIdentNodes.push_back(ic);

                    }
                }
            } else if (fc->label == "<type>") {
                fieldType = visit_type(fc, ctx);
            }
        }

        for (int ni = 0; ni <
(int)fieldNames.size(); ni++) {
            if
(ctx.st.lookupCurrentBlock(fieldNames[ni]) >= 0) {
                ctx.errors.add("Field record sudah
dideklarasikan: '" + fieldNames[ni] + "'");
            } else {
                int idx =
ctx.st.insert(fieldNames[ni], ObjClass::FIELD,
fieldType.baseType, fieldType.ref);

fieldIdentNodes[ni]->annotate((int)fieldType.baseType,
idx, ctx.st.currentLev());
            }
        }

```

```

    }
}

ctx.st.exit_block();

TypeInfo result(TypeCode::RECORD, btabIdx);
result.isNamed = !namedId.empty();
result.namedId = namedId;
    node->annotate((int)TypeCode::RECORD, btabIdx,
ctx.st.currentLev());
return result;
}

```

Fungsi `visit_record_type()` adalah salah satu fungsi paling kompleks karena harus mengelola scope khusus untuk field record sambil membangun informasi yang dibutuhkan untuk akses field di kemudian hari. Fungsi memanggil `enter_block()` untuk membuat scope field record dan menyimpan indeks btab yang dikembalikan. Kemudian setiap <field-part> dalam <field-list> diproses: identifier-identifier field dikumpulkan dari <identifier-list>, tipe field diproses via `visit_type()`, lalu setiap field diinsert ke symbol table dengan obj = FIELD. Setelah semua field selesai, `exit_block()` dipanggil. Indeks btab dari blok field record ini sangat krusial: disimpan ke ref pada TabEntry tipe record dan nantinya digunakan oleh `visit_component_variable()` untuk mencari field dengan mengakses `btab[ref].last` dan mengikuti chain link.

Fungsi `visit_range()` memproses node <range> dengan menggunakan lambda `extractConst` untuk mengekstrak tipe dan nilai dari setiap <constant>. Lambda ini menangani tiga kemungkinan: `intcon` (nilai integer langsung), `charcon` (nilai ordinal dari karakter), dan `ident` (konstanta bernama yang dilookup dari symbol table). Tanda unary plus dan minus sebelum nilai juga ditangani. Setelah kedua batas diketahui, tiga validasi dilakukan: tipe batas kiri dan kanan harus sama, tipe tidak boleh REAL (via `isValidSubrangeType()`), dan `loVal <= hiVal`. Jika `loVal > hiVal`, error ditambahkan namun pemrosesan dilanjutkan.

Metadata subrange kemudian disimpan ke atab via insertArray() dengan xtyp = loTi.baseType dan etyp = loTi.baseType.

```
TypeInfo visit_constant_value(Node* node,
SemanticContext& ctx) {
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    if (node->label == "<constant>") {
        if (node->children.empty()) return
TypeInfo(TypeCode::NOTYPE);

        bool negative = false;
        int ci = 0;

        std::string firstType =
nodeTokenType(node->children[ci]);
        if (firstType == "plus") {
            ci++;
        } else if (firstType == "minus") {
            negative = true;
            ci++;
        }

        if (ci >= (int)node->children.size()) return
TypeInfo(TypeCode::NOTYPE);

        Node* valNode = node->children[ci];
        std::string tt = nodeTokenType(valNode);
        std::string tv = nodeTokenValue(valNode);

        if (tt == "intcon") {
            int v = parseIntValue(tv);
            if (negative) v = -v;
        }
    }
}
```

```

        TypeInfo ti (TypeCode::INTEGER);
        ti.low = v; ti.high = v;
        return ti;
    }

    if (tt == "realcon") return
TypeInfo (TypeCode::REAL);
    if (tt == "charcon") return
TypeInfo (TypeCode::CHAR);
    if (tt == "string") return
TypeInfo (TypeCode::STRING);
    if (tt == "ident") {
        int idx = ctx.st.lookup(tv);
        if (idx < 0) {
            ctx.errors.add("Identifier tidak
ditemukan: '" + tv + "'");
            return TypeInfo (TypeCode::NOTYPE);
        }
        TypeInfo ti = typeInfoFromTab(idx, ctx.st);
        valNode->annotate((int)ti.baseType, idx,
ctx.st.currentLev());
        return ti;
    }
    return TypeInfo (TypeCode::NOTYPE);
}

std::string tt = nodeTokenType(node);
std::string tv = nodeTokenValue(node);

if (tt == "intcon") {
    TypeInfo ti (TypeCode::INTEGER);
    int v = parseIntValue(tv);
    ti.low = v; ti.high = v;

```

```

        return ti;
    }

    if (tt == "realcon") return
TypeInfo(TypeCode::REAL);
    if (tt == "charcon") return
TypeInfo(TypeCode::CHAR);
    if (tt == "string") return
TypeInfo(TypeCode::STRING);
    if (tt == "ident") {
        int idx = ctx.st.lookup(tv);
        if (idx < 0) {
            ctx.errors.add("Identifier tidak ditemukan:
'" + tv + "'");
            return TypeInfo(TypeCode::NOTYPE);
        }
        TypeInfo ti = typeInfoFromTab(idx, ctx.st);
        node->annotate((int)ti.baseType, idx,
ctx.st.currentLev());
        return ti;
    }
    return TypeInfo(TypeCode::NOTYPE);
}

TypeInfo visit_enumerated(Node* node, SemanticContext&
ctx) {
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    int ordinal = 0;
    for (Node* c : node->children) {
        if (nodeTokenType(c) == "ident") {
            std::string name = nodeTokenValue(c);
            if (ctx.st.lookupCurrentBlock(name) >= 0) {

```

```

        ctx.errors.add("Identifier enumerated
sudah dideklarasikan: '" + name + "'");
    } else {
        int idx = ctx.st.insert(name,
ObjClass::CONSTANT,
TypeCode::ENUMERATED, 0, 1, ordinal);
        c->annotate((int)TypeCode::ENUMERATED,
idx, ctx.st.currentLev());
        ordinal++;
    }
}
}

node->annotate((int)TypeCode::ENUMERATED, 0,
ctx.st.currentLev());
return TypeInfo(TypeCode::ENUMERATED);
}

```

Fungsi `visit_enumerated()` mendaftarkan setiap identifier dalam tipe enumerated sebagai konstanta dengan nilai ordinal yang dimulai dari 0 dan bertambah 1 untuk setiap identifier berikutnya. Setiap identifier dicek dengan `lookupCurrentBlock()` untuk mendeteksi redeclaration, lalu diinsert dengan `obj = CONSTANT`, `type = ENUMERATED`, dan `adr = ordinal`. Fungsi `visit_constant_value()` menentukan tipe dan nilai dari sebuah konstanta. Fungsi ini menangani dua kemungkinan struktur: `node <constant>` (dengan kemungkinan tanda unary di depan) dan `node terminal langsung`. Untuk `intcon`, `TypeInfo(INTEGER)` dibuat dengan `low = high = nilai` sehingga nilai konstanta tersimpan dan dapat digunakan untuk validasi batas subrange. Untuk `ident`, dilakukan lookup ke symbol table dan `TypeInfo` dibangun dari `TabEntry` yang ditemukan.

d. Fungsi Visit Ekspresi

```

TypeInfo visit_expression(Node* node, SemanticContext&
ctx) {
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    if (node->label != "<expression>") {
        return visit_simple_expression(node, ctx);
    }

    if (node->children.empty()) return
TypeInfo(TypeCode::NOTYPE);

    // Kumpulkan simple-expression dan
relational-operator
    std::vector<Node*> simpleExprs;
    std::string relOp = "";

    for (Node* c : node->children) {
        if (c->label == "<simple-expression>") {
            simpleExprs.push_back(c);
        } else if (c->label == "<relational-operator>")
{
            // Ambil operator dari dalam node
<relational-operator>
            for (Node* rc : c->children) {
                std::string rtt = nodeTokenType(rc);
                if (rtt == "eq1") { relOp = "eq1";
break; }

                if (rtt == "neq") { relOp = "neq";
break; }

                if (rtt == "lss") { relOp = "lss";
break; }

```

```

        if (rtt == "leq")    { relOp = "leq";
break; }

        if (rtt == "gtr")    { relOp = "gtr";
break; }

        if (rtt == "geq")    { relOp = "geq";
break; }

    }
    } else {
        // Operator relasional mungkin langsung
        sebagai token (tanpa wrapper)
        std::string tt = nodeTokenType(c);
        if (tt == "eq" || tt == "neq" || tt ==
"lss" || tt == "leq" ||
            tt == "gtr" || tt == "geq") {
            relOp = tt;
        }
    }
}

// Hanya satu simple-expression, tidak ada operasi
relasional
if (simpleExprs.size() == 1) {
    TypeInfo      ti      =
visit_simple_expression(simpleExprs[0], ctx);
    node->annotate((int)ti.baseType,  ti.ref,
ctx.st.currentLev());
    return ti;
}

// Dua simple-expression dengan operator relasional
if (simpleExprs.size() == 2) {

```

```

                                TypeInfo    leftTi    =
visit_simple_expression(simpleExprs[0], ctx);
                                TypeInfo    rightTi    =
visit_simple_expression(simpleExprs[1], ctx);

    if (relOp.empty()) {
        ctx.errors.add("Operator relasional tidak
ditemukan dalam expression");
        node->annotate((int) TypeCode::BOOLEAN, -1,
ctx.st.currentLev());
        return TypeInfo(TypeCode::BOOLEAN);
    }

    // Validasi kompatibilitas operand
    bool compatible = isCompatible(leftTi, rightTi,
ctx.st)

                                || (leftTi.isNumeric() &&
rightTi.isNumeric());

    if (!compatible) {
        ctx.errors.add("Operand operator '" + relOp
+ "' tidak kompatibel: "
                                + leftTi.toString() + " vs "
+ rightTi.toString());
    }

    // Hasil operator relasional selalu Boolean
    node->annotate((int) TypeCode::BOOLEAN, -1,
ctx.st.currentLev());
    return TypeInfo(TypeCode::BOOLEAN);
}

```

```

        // Fallback: lebih dari 2 simple-expression
        (seharusnya tidak terjadi)
        if (!simpleExprs.empty()) {
            TypeInfo ti =
visit_simple_expression(simpleExprs[0], ctx);
            node->annotate((int)ti.baseType, ti.ref,
ctx.st.currentLev());
            return ti;
        }

        return TypeInfo(TypeCode::NOTYPE);
    }

```

Fungsi `visit_expression()` menangani node `<expression>` yang merepresentasikan ekspresi paling umum dalam grammar. Fungsi pertama-tama memeriksa apakah node tersebut adalah `<expression>`; jika tidak (misalnya dipanggil dengan node `<simple-expression>`), langsung didelegasikan ke `visit_simple_expression()`. Untuk node `<expression>` yang sesungguhnya, fungsi mengumpulkan semua `<simple-expression>` dan mengekstrak operator relasional dari node `<relational-operator>` atau dari terminal langsung. Jika hanya ada satu `<simple-expression>`, tipe langsung diteruskan tanpa perubahan. Jika ada dua `<simple-expression>` dengan operator relasional, kedua tipe diproses via `visit_simple_expression()`, kemudian validasi kompatibilitas dilakukan: dua operand dianggap valid jika `isCompatible(leftTi, rightTi)` atau jika keduanya numerik (untuk mendukung perbandingan Integer > Real). Jika tidak valid, error ditambahkan dengan pesan yang menyebutkan kedua tipe yang bermasalah. Hasil selalu bertipe `BOOLEAN` dan node dianotasi dengan demikian.

```

TypeInfo visit_simple_expression(Node* node,
SemanticContext& ctx) {
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    if (node->label != "<simple-expression>") {

```



```

        return visit_term(node, ctx);
    }

    if (node->children.empty()) return
TypeInfo(TypeCode::NOTYPE);

    TypeInfo current(TypeCode::NOTYPE);
    std::string pendingOp = "";
    bool firstOperand = true;
    bool unaryMinus    = false;
    bool unaryPlus     = false;

    for (Node* c : node->children) {
        std::string tt;
        if (c->getLabel() == "<additive-operator>") {
            tt = nodeTokenType (c->children[0]);
        }

        // Operator additive
        if (tt == "plus" || tt == "minus" || tt ==
"orsy") {
            if (firstOperand && current.baseType ==
TypeInfo::NOTYPE) {
                // Ini unary operator
                if (tt == "minus") unaryMinus = true;
                else if (tt == "plus") unaryPlus =
true;

                // orsy tidak valid sebagai unary
            } else {
                if (tt == "plus") pendingOp = "plus";
                if (tt == "minus") pendingOp = "minus";
                if (tt == "orsy") pendingOp = "orsy";
            }
        }
    }

```

```

    }
    continue;
}

// Operand
TypeInfo operandTi (TypeCode::NOTYPE);
if (c->label == "<term>") {
    operandTi = visit_term(c, ctx);
    // } else if (c->label ==
"<simple-expression>") {
    // operandTi = visit_simple_expression(c,
ctx);
} else {
    continue;
}

// Terapkan unary operator pada operand pertama
if (firstOperand) {
    if (unaryMinus || unaryPlus) {
        std::string uop = unaryMinus ? "minus"
: "plus";
        TypeInfo uRes = resultTypeUnary(uop,
operandTi);
        if (uRes.baseType == TypeCode::NOTYPE)
        {
            ctx.errors.add("Operator unary '" +
uop + "' tidak bisa diterapkan pada tipe: "
+
operandTi.toString());
        }
        current = uRes.baseType !=
TypeCode::NOTYPE ? uRes : operandTi;

```

```

        } else {
            current = operandTi;
        }
        firstOperand = false;
        continue;
    }

    // Operand selanjutnya (biner)
    if (pendingOp.empty()) {
        ctx.errors.add("Operator tidak ditemukan di
antara operand dalam simple-expression");
        continue;
    }

    if (pendingOp == "orsy") {
        if (!current.isBoolean() ||
!operandTi.isBoolean()) {
            ctx.errors.add("Operator 'or'
membutuhkan operand bertipe Boolean, "
                        "ditemukan: " +
current.toString() + " dan " + operandTi.toString());
            current = TypeInfo(TypeCode::NOTYPE);
            pendingOp = "";
            continue;
        }
    }

    TypeInfo res = resultType(pendingOp, current,
operandTi, ctx.st);
    if (res.baseType == TypeCode::NOTYPE) {
        ctx.errors.add("Operasi '" + pendingOp + "'
tidak kompatibel untuk tipe: "

```

```

                                + current.toString() + " dan
" + operandTi.toString());
    }
    current    = res;
    pendingOp  = "";
}

    node->annotate((int)current.baseType, current.ref,
ctx.st.currentLev());
    return current;
}

```

Fungsi `visit_simple_expression()` menangani operator additive dengan kompleksitas tambahan karena harus membedakan unary operator di awal ekspresi dari binary operator di tengah. Iterasi dilakukan dari kiri ke kanan pada children node `<simple-expression>`. Untuk node `<additive-operator>`, tipe operator diambil dari child pertamanya. Untuk token additive langsung (tanpa wrapper), tipe diambil via `nodeTokenType()`. Jika operator ditemukan dan belum ada operand pertama, diperlakukan sebagai unary. Jika operator ditemukan setelah operand pertama, disimpan sebagai `pendingOp`. Untuk setiap node `<term>`, `visit_term()` dipanggil. Operand pertama mungkin dikenai unary operator via `resultTypeUnary()`. Operand berikutnya diproses dengan `resultType(pendingOp, current, operandTi)`. Khusus untuk orsy, kedua operand divalidasi secara eksplisit harus bertipe `BOOLEAN` sebelum `resultType()` dipanggil, dan error yang lebih spesifik ditambahkan jika tidak sesuai.

```

TypeInfo visit_term(Node* node, SemanticContext& ctx) {
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    if (node->label != "<term>") {
        // Mungkin langsung dipanggil dengan child
factor
        return visit_factor(node, ctx);
    }
}

```

```

        if (node->children.empty()) return
TypeInfo(TypeCode::NOTYPE);

    // <term> bisa punya anak flat:
    // <factor> + (<mulop> <factor>)*
    // Karena parse tree bisa di-flatten, kita iterasi
left-to-right
    TypeInfo current(TypeCode::NOTYPE);
    std::string pendingOp = "";

    for (Node* c : node->children) {
        std::string tt;

        if (c->getLabel() ==
"<multiplicative-operator>") {
            tt = nodeTokenType(c->children[0]);
        }

        // Operator: times(*), rdiv(/), idiv(div),
imod(mod), andsy(and)
        if (tt == "times" || tt == "rdiv" ||
            tt == "idiv" || tt == "imod" ||
            tt == "andsy") {
            // Normalisasi nama operator agar cocok
dengan resultType()
            if (tt == "times") pendingOp = "times";
            else if (tt == "rdiv") pendingOp = "rdiv";
            else if (tt == "idiv") pendingOp = "idiv";
            else if (tt == "imod") pendingOp = "imod";
            else if (tt == "andsy") pendingOp =
"andsy";

            continue;

```

```

    }

    // Operand: <factor> atau <term>
    TypeInfo operandTi (TypeCode::NOTYPE);
    if (c->label == "<factor>") {
        operandTi = visit_factor(c, ctx);
    // } else if (c->label == "<term>") {
    //     operandTi = visit_term(c, ctx);
    } else {
        // Skip non-operand nodes (terminal lain)
        continue;
    }

    if (current.baseType == TypeCode::NOTYPE) {
        // Ini operand pertama
        current = operandTi;
    } else {
        // Ada operator + operand kedua
        if (pendingOp.empty()) {
            std::cout << "Label: " <<
node->getLabel() << std::endl;
            ctx.errors.add("Operator tidak
ditemukan di antara operand dalam term");
            continue;
        }

        // Validasi khusus per operator
        if (pendingOp == "idiv" || pendingOp ==
"imod") {
            if (!current.isInteger() ||
!operandTi.isInteger()) {

```

```

                                ctx.errors.add("Operator '" +
pendingOp + "' membutuhkan operand bertipe Integer, "
                                "ditemukan: " +
current.toString() + " dan " + operandTi.toString());
                                current =
TypeInfo(TypeCode::NOTYPE);
                                pendingOp = "";
                                continue;
                        }
                }
                if (pendingOp == "andsy") {
                        if (!current.isBoolean() ||
!operandTi.isBoolean()) {
                                ctx.errors.add("Operator 'and'
membutuhkan operand bertipe Boolean, "
                                "ditemukan: " +
current.toString() + " dan " + operandTi.toString());
                                current =
TypeInfo(TypeCode::NOTYPE);
                                pendingOp = "";
                                continue;
                        }
                }

                TypeInfo res = resultType(pendingOp,
current, operandTi, ctx.st);
                if (res.baseType == TypeCode::NOTYPE) {
                        ctx.errors.add("Operasi '" + pendingOp
+ "' tidak kompatibel untuk tipe: "
                                + current.toString() + "
dan " + operandTi.toString());
                }

```

```

        current      = res;
        pendingOp    = "";
    }
}

    node->annotate((int)current.baseType, current.ref,
ctx.st.currentLev());
    return current;
}

```

Fungsi `visit_term()` bekerja dengan pola yang lebih sederhana dibandingkan `visit_simple_expression()` karena tidak ada unary operator pada level ini. Iterasi dilakukan pada children node `<term>`. Untuk node `<multiplicative-operator>`, tipe operator diambil dari child pertamanya. Untuk setiap node `<factor>`, `visit_factor()` dipanggil. Operand pertama langsung disimpan sebagai `current`. Operand berikutnya diproses dengan `resultType(pendingOp, current, operandTi)` setelah validasi khusus per operator: untuk `idiv` dan `imod`, kedua operand wajib integer; untuk `andsy`, kedua operand wajib boolean. Jika validasi gagal, error spesifik ditambahkan dan tipe sementara diset ke `NOTYPE` agar tidak memicu cascade error yang tidak relevan.

```

TypeInfo visit_factor(Node* node, SemanticContext& ctx)
{
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    // Jika node ini adalah <factor>, telusuri
child-nya
    if (node->label == "<factor>") {
        if (node->children.empty()) return
TypeInfo(TypeCode::NOTYPE);

        Node* first = node->children[0];
        std::string tt = nodeTokenType(first);
    }
}

```



```

        // NOT <factor>
        if (tt == "notsy") {
            if (node->children.size() < 2) {
                ctx.errors.add("Ekspresi NOT tidak
lengkap");
                return TypeInfo(TypeCode::NOTYPE);
            }

            TypeInfo operand =
visit_factor(node->children[1], ctx);
            if (operand.baseType != TypeCode::BOOLEAN)
{
                ctx.errors.add("Operand NOT harus
bertipe Boolean, ditemukan: " + operand.toString());
                node->annotate((int)TypeCode::NOTYPE,
-1, ctx.st.currentLev());
                return TypeInfo(TypeCode::NOTYPE);
            }
            node->annotate((int)TypeCode::BOOLEAN, -1,
ctx.st.currentLev());
            return TypeInfo(TypeCode::BOOLEAN);
        }

        // ( expression )
        if (tt == "lparent") {
            // cari child <expression>
            for (Node* c : node->children) {
                if (c->label == "<expression>") {
                    TypeInfo ti = visit_expression(c,
ctx);
                    node->annotate((int)ti.baseType,
ti.ref, ctx.st.currentLev());
                }
            }
        }
    }
}

```

```

        return ti;
    }
}

return TypeInfo(TypeCode::NOTYPE);
}

// <variable> - akses variabel (termasuk
array/record)
if (first->label == "<variable>") {
    TypeInfo ti = visit_variable(first, ctx);
    node->annotate((int)ti.baseType, ti.ref,
ctx.st.currentLev());
    return ti;
}

// <procedure/function-call> yang berupa
function (punya nilai kembalian)
if (first->label ==
"<procedure/function-call>") {
    // Kita perlu tahu return type-nya
    // Cari ident dari call node
    Node* callIdentNode =
findChildByTokenType(first, "ident");
    if (callIdentNode) {
        std::string funcName =
nodeTokenValue(callIdentNode);
        int idx = ctx.st.lookup(funcName);
        if (idx >= 0 && ctx.st.tab[idx].obj ==
ObjClass::FUNCTION) {
visit_procedure_function_call(first, ctx);

```

```

        TypeInfo ti = typeInfoFromTab(idx,
ctx.st);

        node->annotate((int)ti.baseType,
ti.ref, ctx.st.currentLev());

        return ti;
    }

    }

    visit_procedure_function_call(first, ctx);
    return TypeInfo(TypeCode::NOTYPE);
}

// Literal: intcon, realcon, charcon, string
if (tt == "intcon") {
    TypeInfo ti(TypeCode::INTEGER);

    try { int v =
std::stoi(nodeTokenValue(first)); ti.low = v; ti.high =
v; }

    catch (...) {}

    first->annotate((int)TypeCode::INTEGER, -1,
ctx.st.currentLev());

    node->annotate((int)TypeCode::INTEGER, -1,
ctx.st.currentLev());

    return ti;
}

if (tt == "realcon") {
    first->annotate((int)TypeCode::REAL, -1,
ctx.st.currentLev());

    node->annotate((int)TypeCode::REAL, -1,
ctx.st.currentLev());

    return TypeInfo(TypeCode::REAL);
}

if (tt == "charcon") {

```

```

        first->annotate((int)TypeCode::CHAR, -1,
ctx.st.currentLev());
        node->annotate((int)TypeCode::CHAR, -1,
ctx.st.currentLev());
        return TypeInfo(TypeCode::CHAR);
    }
    if (tt == "string") {
        first->annotate((int)TypeCode::STRING, -1,
ctx.st.currentLev());
        node->annotate((int)TypeCode::STRING, -1,
ctx.st.currentLev());
        return TypeInfo(TypeCode::STRING);
    }

    // Identifier langsung (True, False, konstanta,
dll)
    if (tt == "ident") {
        std::string name = nodeTokenValue(first);
        int idx = ctx.st.lookup(name);
        if (idx < 0) {
            ctx.errors.add("Identifier tidak
ditemukan: '" + name + "'");
            node->annotate((int)TypeCode::NOTYPE,
-1, ctx.st.currentLev());
            return TypeInfo(TypeCode::NOTYPE);
        }
        TypeInfo ti = typeInfoFromTab(idx, ctx.st);
        first->annotate((int)ti.baseType, idx,
ctx.st.currentLev());
        node->annotate((int)ti.baseType, idx,
ctx.st.currentLev());
        return ti;
    }

```

```

    }

    // Fallback: delegate ke child pertama
    return visit_factor(first, ctx);
}

// Jika node ini langsung terminal (dipanggil
rekursif dari NOT)
    std::string tt = nodeTokenType(node);
    if (tt == "intcon") {
        TypeInfo ti(TypeCode::INTEGER);
        try { int v = std::stoi(nodeTokenValue(node));
ti.low = v; ti.high = v; }
        catch (...) {}
        node->annotate((int) TypeCode::INTEGER, -1,
ctx.st.currentLev());
        return ti;
    }

        if (tt == "realcon") {
node->annotate((int) TypeCode::REAL, -1,
ctx.st.currentLev()); return TypeInfo(TypeCode::REAL);
        }

        if (tt == "charcon") {
node->annotate((int) TypeCode::CHAR, -1,
ctx.st.currentLev()); return TypeInfo(TypeCode::CHAR);
        }

        if (tt == "string") {
node->annotate((int) TypeCode::STRING, -1,
ctx.st.currentLev()); return
TypeInfo(TypeCode::STRING); }
    if (tt == "ident") {
        std::string name = nodeTokenValue(node);

```

```

        int idx = ctx.st.lookup(name);
        if (idx < 0) {
            ctx.errors.add("Identifier tidak ditemukan:
'" + name + "'");
            return TypeInfo(TypeCode::NOTYPE);
        }
        TypeInfo ti = typeInfoFromTab(idx, ctx.st);
        node->annotate((int)ti.baseType, idx,
ctx.st.currentLev());
        return ti;
    }

    return TypeInfo(TypeCode::NOTYPE);
}

```

Fungsi `visit_factor()` menangani unit terkecil ekspresi dengan banyak kasus yang harus dibedakan. Untuk node `<factor>`, child pertama diperiksa tipenya: `notsy` memicu pengambilan operand via `visit_factor()` pada child kedua dan validasi harus `BOOLEAN`; `lparent` memicu pencarian node `<expression>` di children dan rekursi via `visit_expression()`; `<variable>` mendelegasikan ke `visit_variable()` untuk lookup dan validasi variabel; `<procedure/function-call>` mendelegasikan ke `visit_procedure_function_call()` dan mengambil return type dari symbol table jika identifier adalah fungsi; literal `intcon/realcon/charcon/string` langsung menghasilkan `TypeInfo` yang sesuai dengan nilai disimpan di `low/high` untuk `intcon`; `ident` melakukan `lookup()` dan membangun `TypeInfo` dari `TabEntry`. Setiap node yang diproses dianotasi dengan tipe yang dihasilkan.

```

TypeInfo visit_variable(Node* node, SemanticContext&
ctx) {
    if (!node) return TypeInfo(TypeCode::NOTYPE);

    // Kumpulkan semua child yang relevan
    Node* identNode = nullptr;

```

```

        Node*      componentNode      =      nullptr;      //
<component-variable> jika ada

        for (Node* c : node->children) {
            if (nodeTokenType(c) == "ident" && !identNode)
identNode = c;
                if (c->label == "<component-variable>")
componentNode = c;
        }

        if (!identNode) {
            // Mungkin satu-satunya child adalah
<component-variable>
            if (componentNode)
return
visit_component_variable(componentNode,      ctx,
TypeInfo(TypeCode::NOTYPE));
            return TypeInfo(TypeCode::NOTYPE);
        }

        std::string name = nodeTokenValue(identNode);
        int idx = ctx.st.lookup(name);
        if (idx < 0) {
            ctx.errors.add("Variabel tidak ditemukan: '" +
name + "'");
            node->annotate((int) TypeInfo::NOTYPE, -1,
ctx.st.currentLev());
            return TypeInfo(TypeCode::NOTYPE);
        }

        const TabEntry& e = ctx.st.tab[idx];

```

```

        // Pastikan ini bukan tipe atau prosedur yang
dipakai sebagai variabel
        if (e.obj == ObjClass::TYPE) {
            ctx.errors.add("'" + name + "' adalah tipe,
bukan variabel");
            node->annotate((int) TypeCode::NOTYPE, idx,
ctx.st.currentLev());
            return TypeInfo(TypeCode::NOTYPE);
        }
        if (e.obj == ObjClass::PROCEDURE) {
            ctx.errors.add("'" + name + "' adalah prosedur,
tidak bisa digunakan sebagai nilai");
            node->annotate((int) TypeCode::NOTYPE, idx,
ctx.st.currentLev());
            return TypeInfo(TypeCode::NOTYPE);
        }

        TypeInfo baseType = typeInfoFromTab(idx, ctx.st);
        identNode->annotate((int) baseType.baseType, idx,
ctx.st.currentLev());

        // Jika ada akses komponen (array subscript atau
record field), lanjutkan
        if (componentNode) {
                                TypeInfo    resultTi    =
visit_component_variable(componentNode, ctx, baseType);
                                node->annotate((int) resultTi.baseType,
resultTi.ref, ctx.st.currentLev());
                                return resultTi;
        }

```



```

        node->annotate( (int)baseType.baseType, idx,
ctx.st.currentLev() );
    return baseType;
}

```

Fungsi `visit_variable()` memproses akses variabel yang bisa berupa akses sederhana (hanya nama) atau akses komponen (dengan subscript array atau field record). Pertama, node ident dicari di children. Jika ditemukan, `lookup()` dipanggil dengan nama identifier. Jika tidak ditemukan (indeks -1), error "variabel tidak ditemukan" ditambahkan. Jika ditemukan, dilakukan validasi bahwa objek bukan TYPE dan bukan PROCEDURE (keduanya tidak bisa digunakan sebagai nilai). Kemudian `typeInfoFromTab()` dipanggil untuk mendapatkan `TypeInfo` lengkap dari entri tab. Jika ada node <component-variable> di children, `visit_component_variable()` dipanggil dengan tipe dasar ini untuk memproses akses lebih lanjut, dan hasilnya menggantikan tipe dasar. Node ident dan node variabel keseluruhan dianotasi dengan tipe final.

```

TypeInfo visit_component_variable(Node* node,
SemanticContext& ctx, TypeInfo baseType) {
    if (!node) return baseType;

    // Gunakan indexed for loop untuk handle
multidimensional subscript dengan benar
    for (int i = 0; i < (int)node->children.size();
i++) {
        Node* c = node->children[i];

        // ---- Akses array: arr[expression] ----
        if (nodeTokenType(c) == "lbrack") {
            // Pastikan baseType adalah array
            if (baseType.baseType != TypeCode::ARRAY) {
                ctx.errors.add("Subscript hanya bisa
dilakukan pada tipe Array, ditemukan: "

```

```

        + baseType.toString());
    return TypeInfo(TypeCode::NOTYPE);
}

// Ambil info array dari atab
if (baseType.ref <= 0 || baseType.ref >=
(int) ctx.st.atab.size()) {
    ctx.errors.add("Referensi array tidak
valid");
    return TypeInfo(TypeCode::NOTYPE);
}

const AtabEntry& arrayInfo =
ctx.st.atab[baseType.ref];

TypeInfo
indexType(static_cast<TypeCode>(arrayInfo.xtyp));

// Cari expression di antara '[' dan ']'
// Loop dari i+1 sampai menemukan
expression
bool foundExpr = false;
for (int j = i + 1; j <
(int) node->children.size(); j++) {
    Node* sub = node->children[j];

    if (sub->label == "<expression>") {
        TypeInfo idxTi =
visit_expression(sub, ctx);
        foundExpr = true;
    }

    // Index harus kompatibel dengan
tipe indeks array

```

```

        if (!isCompatible(idxTi, indexType,
ctx.st)) {
            // Integer ke subrange integer
            masih ok
            if (!(idxTi.isInteger() &&
indexType.isInteger()) &&
                !(idxTi.isInteger() &&
indexType.isSubrange())) {
                ctx.errors.add("Tipe indeks
array tidak kompatibel: ditemukan "
                                +
idxTi.toString() + ", diharapkan "
                                +
indexType.toString());
            }
        }
        i = j; // Skip ke expression yang
baru saja diproses
        break;
    }

    if (nodeTokenType(sub) == "rbrack") {
        i = j; // Skip ke rbrack
        break;
    }
}

// Tipe hasil = tipe elemen array
                                TypeInfo
elemType(static_cast<TypeCode>(arrayInfo.ety),
arrayInfo.eref);
    baseType = elemType;

```

```

        node->annotate((int)elemType.baseType,
elemType.ref, ctx.st.currentLev());
        continue;
    }

    // ---- Akses field record: rec.field ----
    if (nodeTokenType(c) == "period") {
        // Pastikan baseType adalah record, atau
        ARRAY dari record
        bool isValidForFieldAccess = false;

        if (baseType.baseType == TypeCode::RECORD)
        {
            isValidForFieldAccess = true;
        } else if (baseType.baseType ==
TypeCode::ARRAY) {
            // Cek tipe elemen array
            if (baseType.ref > 0 && baseType.ref <
(int)ctx.st.atab.size()) {
                const AtabEntry& arrayInfo =
ctx.st.atab[baseType.ref];

                TypeCode elemType =
static_cast<TypeCode>(arrayInfo.ety);
                if (elemType == TypeCode::RECORD) {
                    isValidForFieldAccess = true;
                    // Update baseType ke tipe
                    elemen (RECORD)

                    baseType =
TypeInfo(TypeCode::RECORD, arrayInfo.eref);
                }
            }
        }
    }

```

```

    }

    if (!isValidForFieldAccess) {
        ctx.errors.add("Akses field '.' hanya
bisa dilakukan pada tipe Record, ditemukan: "
                        + baseType.toString());
        return TypeInfo(TypeCode::NOTYPE);
    }
}

if (nodeTokenType(c) == "ident" &&
    baseType.baseType == TypeCode::RECORD) {
    std::string fieldName = nodeTokenValue(c);

    // Cari field di btab[ref] record ini
    if (baseType.ref <= 0 || baseType.ref >=
(int) ctx.st.btab.size()) {
        ctx.errors.add("Referensi record tidak
valid untuk field '" + fieldName + "'");
        return TypeInfo(TypeCode::NOTYPE);
    }

    int fieldIdx = -1;
    int k = ctx.st.btab[baseType.ref].last;
    while (k != 0 && k <
(int) ctx.st.tab.size()) {
        std::string tabId =
ctx.st.tab[k].identifier;
        // case-insensitive compare
        std::string tabIdLow = tabId, fnLow =
fieldName;

```

```

        std::transform(tabIdLow.begin(),
tabIdLow.end(), tabIdLow.begin(), ::tolower);
        std::transform(fnLow.begin(),
fnLow.end(), fnLow.begin(), ::tolower);
        if (tabIdLow == fnLow) { fieldIdx = k;
break; }

        k = ctx.st.tab[k].link;
    }

    if (fieldIdx < 0) {
        ctx.errors.add("Field '" + fieldName +
"' tidak ditemukan dalam record '"
+ baseType.namedId +
"'");
        return TypeInfo(TypeCode::NOTYPE);
    }

    TypeInfo fieldType =
typeInfoFromTab(fieldIdx, ctx.st);
    c->annotate((int)fieldType.baseType,
fieldIdx, ctx.st.currentLev());
    baseType = fieldType;

    node->annotate((int)fieldType.baseType,
fieldType.ref, ctx.st.currentLev());
    continue;
}

// Akses komponen berlapis (array of record,
dll) – rekursif
if (c->label == "<component-variable>") {

```

```

        baseType = visit_component_variable(c, ctx,
baseType);
    }
}

return baseType;
}

```

Fungsi `visit_component_variable()` menangani akses komponen berlapis menggunakan indexed for loop untuk mendukung skenario seperti `matrix[i][j]` (array 2D) dan `points[1].x` (array of record). Iterasi dilakukan pada children dengan mengakses indeks secara eksplisit. Saat token lbrack ditemukan, proses akses array dimulai: `baseType` divalidasi harus ARRAY, info array diambil dari `ctx.st.atab[baseType.ref]`, lalu dicari node `<expression>` setelah lbrack untuk mendapatkan tipe indeks. Tipe indeks divalidasi kompatibel dengan `xtp` dari `atab`. Setelah pemrosesan, `baseType` diperbarui menjadi tipe elemen array (`TypeInfo(ety, eref)`). Saat token period ditemukan, proses akses field dimulai: `baseType` divalidasi harus RECORD. Kemudian saat token ident berikutnya ditemukan, nama field dicari di chain `btab[baseType.ref].last` secara case-insensitive. Jika tidak ditemukan, error "field tidak ditemukan dalam record" ditambahkan. Jika ditemukan, `baseType` diperbarui menjadi tipe field.

```

void      visit_procedure_function_call(Node*      node,
SemanticContext& ctx) {
    if (!node) return;

    // Ambil nama procedure/function dari ident pertama
    Node*  identNode = findChildByTokenType(node,
"ident");
    if (!identNode) {
        ctx.errors.add("Pemanggilan prosedur/fungsi
tanpa nama");
        return;
    }
}

```

```

    }

    std::string name = nodeTokenValue(identNode);
    int idx = ctx.st.lookup(name);

    if (idx < 0) {
        ctx.errors.add("Prosedur/fungsi tidak
ditemukan: '" + name + "'");
        node->annotate((int)TypeCode::NOTYPE, -1,
ctx.st.currentLev());
        return;
    }

    const TabEntry& e = ctx.st.tab[idx];
    if (e.obj != ObjClass::PROCEDURE && e.obj !=
ObjClass::FUNCTION) {
        ctx.errors.add("'" + name + "' bukan prosedur
atau fungsi");
        return;
    }

    // Anotasi identifier call
    identNode->annotate((int)e.type, idx,
ctx.st.currentLev());

    // Kumpulkan argumen dari <parameter-list> atau
<actual-parameter-list>
    std::vector<Node*> args;
    for (Node* c : node->children) {
        if (c->label == "<parameter-list>" ||
            c->label == "<actual-parameter-list>") {
            for (Node* pc : c->children) {

```



```

        if (pc->label == "<expression>" ||
            pc->label == "<actual-parameter>")
    {
        args.push_back(pc);
    }
}

// Parameter bisa juga langsung sebagai
<expression> child
if (c->label == "<expression>") {
    args.push_back(c);
}

// Untuk predefined procedures (writeln, readln,
write, read):
// tidak perlu validasi ketat jumlah/tipe parameter
bool isPredefined = (name == "writeln" || name ==
"readln" ||
                    name == "write" || name ==
"read");
if (isPredefined) {
    // Tetap visit semua argumen agar teranotasi
    for (Node* arg : args) {
        if (arg->label == "<actual-parameter>") {
            for (Node* ac : arg->children) {
                if (ac->label == "<expression>")
visit_expression(ac, ctx);
            }
        } else {
            visit_expression(arg, ctx);
        }
    }
}

```

```

    }

    node->annotate((int)e.type, idx,
ctx.st.currentLev());
    return;
}

// Untuk prosedur/fungsi user-defined: kumpulkan
info parameter formal
std::vector<TypeInfo> formalParams;
if (e.ref > 0 && e.ref < (int)ctx.st.btab.size()) {
    int lparIdx = ctx.st.btab[e.ref].lpar;
    // Kumpulkan parameter dari lpar ke belakang
(via link)
    std::vector<int> paramIndices;
    int k = lparIdx;
    while (k != 0 && k < (int)ctx.st.tab.size()) {
        paramIndices.push_back(k);
        k = ctx.st.tab[k].link;
    }

    // Balik urutan agar sesuai dengan urutan
deklarasi
    std::reverse(paramIndices.begin(),
paramIndices.end());
    for (int pi : paramIndices) {
        formalParams.push_back(typeInfoFromTab(pi,
ctx.st));
    }
}

// Cek jumlah argumen
if (args.size() != formalParams.size()) {

```

```

        ctx.errors.add("Jumlah argumen tidak sesuai
untuk '" + name + "': "
                                "diharapkan " +
std::to_string(formalParams.size()) +
                                ", diberikan " +
std::to_string(args.size()));
    }

    // Cek tipe tiap argumen
    int checkCount = std::min(args.size(),
formalParams.size());
    for (int i = 0; i < checkCount; i++) {
        TypeInfo argType (TypeCode::NOTYPE);

        if (args[i]->label == "<actual-parameter>") {
            for (Node* ac : args[i]->children) {
                if (ac->label == "<expression>") {
                    argType = visit_expression(ac,
ctx);
                    break;
                }
            }
        } else {
            argType = visit_expression(args[i], ctx);
        }

        if (!isAssignmentCompatible(formalParams[i],
argType, ctx.st)) {
            ctx.errors.add("Tipe argumen ke-" +
std::to_string(i + 1) +
                                " untuk '" + name + "' tidak
kompatibel: "

```

```

                                "diharapkan " +
formalParams[i].toString() +
                                ", diberikan " +
argType.toString());
    }
}

// Visit argumen yang sisa (jika jumlahnya lebih)
for (int i = (int)checkCount; i < (int)args.size();
i++) {
    if (args[i]->label == "<actual-parameter>") {
        for (Node* ac : args[i]->children) {
            if (ac->label == "<expression>")
visit_expression(ac, ctx);
        }
    } else {
        visit_expression(args[i], ctx);
    }
}

TypeInfo retTi = typeInfoFromTab(idx, ctx.st);
node->annotate((int)retTi.baseType, idx,
ctx.st.currentLev());
}

```

Fungsi `visit_procedure_function_call()` adalah salah satu fungsi `visit` yang paling panjang karena harus menangani berbagai skenario pemanggilan. Setelah lookup nama dan validasi objek harus `PROCEDURE` atau `FUNCTION`, argumen aktual dikumpulkan dari node `<parameter-list>` atau `<actual-parameter-list>` (jika ada). Untuk predefined procedures (`writeln`, `readln`, `write`, `read`), semua argumen dikunjungi via `visit_expression()` untuk dianotasi, namun tidak ada validasi jumlah atau tipe argumen karena sifat variadiknya. Untuk user-defined procedures/functions, parameter formal dikumpulkan dengan mengikuti

chain dari `btab[e.ref].lpar` ke belakang menggunakan field `link`. Chain ini di-reverse agar urutan sesuai dengan urutan deklarasi (karena linked list di symbol table adalah LIFO). Jumlah argumen aktual dibandingkan dengan jumlah parameter formal, dan error ditambahkan jika berbeda. Untuk setiap pasangan argumen-parameter, `visit_expression()` dipanggil untuk mendapatkan tipe argumen, lalu `isAssignmentCompatible(formalType, argType)` dipanggil untuk memvalidasi kompatibilitas tipe.

e. Fungsi Visit Statement

```
void visit_compound_statement(Node* node,
SemanticContext& ctx) {
    // std::cout << "masuk compound statement" <<
std::endl;
    if (!node) return;

    auto blockAst = makeAstNode("Block");
    blockAst->block_index = ctx.st.currentBlock();
    blockAst->lev = ctx.st.currentLev();
    attachAstNode(blockAst);

    auto prevPending = g_astCtx.pendingBlockParent;
    g_astCtx.pendingBlockParent.reset();
    g_astCtx.blockStack.push_back(blockAst);

    // anak: beginsy <statement-list> endsy
    for (Node* c : node->children) {
        if (c->label == "<statement-list>") {
            visit_statement_list(c, ctx);
        }
    }
}
```

```

        if (!g_astCtx.blockStack.empty())
g_astCtx.blockStack.pop_back();
        g_astCtx.pendingBlockParent = prevPending;

        node->annotate((int)TypeCode::NOTYPE, -1,
ctx.st.currentLev());
    }

void visit_statement_list(Node* node, SemanticContext&
ctx) {
    // std::cout << "masuk statement list" <<
std::endl;
    if (!node) return;

    // anak: <statement> (semicolon <statement>)*
    for (Node* c : node->children) {
        if (c->label == "<statement>") {
            visit_statement(c, ctx);
        }
    }

    node->annotate((int)TypeCode::NOTYPE, -1,
ctx.st.currentLev());
}

void visit_statement_list(Node* node, SemanticContext&
ctx) {
    // std::cout << "masuk statement list" <<
std::endl;
    if (!node) return;

    // anak: <statement> (semicolon <statement>)*
    for (Node* c : node->children) {

```

```

        if (c->label == "<statement>") {
            visit_statement(c, ctx);
        }
    }

    node->annotate( (int) TypeCode::NOTYPE,    -1,
ctx.st.currentLev() );
}

```

Fungsi `visit_compound_statement()` membuat `ASTNode` bertipe "Block" dengan `block_index` dari `ctx.st.currentBlock()` (indeks btabs blok saat ini) dan `lev` dari `ctx.st.currentLev()` (lexical level saat ini). Node block ini di-attach ke parent yang ditentukan oleh `selectAttachParent()`, yang mempertimbangkan `attachOverride`, `pendingBlockParent`, dan `top of blockStack` secara berurutan. Kemudian `pendingBlockParent` di-reset ke null agar tidak mempengaruhi block berikutnya, dan block baru di-push ke `blockStack`. Setelah `visit_statement_list()` selesai memproses semua statement di dalam block, block di-pop dari `blockStack` dan `pendingBlockParent` dikembalikan ke nilai sebelumnya. Dengan mekanisme push/pop ini, nested compound statement (misalnya compound statement di dalam body if di dalam while) selalu menempel ke block yang tepat. Fungsi `visit_statement_list()` mengiterasi semua child dari node `<statement-list>` dan memanggil `visit_statement()` untuk setiap node `<statement>` yang ditemukan, melewati token semicolon.

```

void visit_statement(Node* node, SemanticContext& ctx)
{
    // std::cout << "masuk statement" << std::endl;
    if (!node) return;

    // anak : salah satu dari
    //    <assignment-statement>,    <if-statement>,
    <while-statement>,
    //    <repeat-statement>,    <for-statement>,
    <case-statement>,

```

```

// <procedure/function-call>, <compound-statement>

if (node->children.empty()) {
    return;
}

Node* child = node->children[0];
if (child->label == "<assignment-statement>") {
    visit_assignment_statement(child, ctx);
} else if (child->label == "<if-statement>") {
    visit_if_statement(child, ctx);
} else if (child->label == "<while-statement>") {
    visit_while_statement(child, ctx);
} else if (child->label == "<repeat-statement>") {
    visit_repeat_statement(child, ctx);
} else if (child->label == "<for-statement>") {
    visit_for_statement(child, ctx);
} else if (child->label == "<case-statement>") {
    visit_case_statement(child, ctx);
} else if (child->label ==
"<procedure/function-call>") {
    visit_procedure_function_call(child, ctx);
    auto callAst = build_proc_call_ast(child);
    if (callAst) attachAstNode(callAst);
} else if (child->label == "<compound-statement>")
{
    auto prevPending = g_astCtx.pendingBlockParent;
    g_astCtx.pendingBlockParent =
g_astCtx.attachOverride ? g_astCtx.attachOverride :
currentBlockAst();
    visit_compound_statement(child, ctx);
    g_astCtx.pendingBlockParent = prevPending;
}

```



```

    }

    node->annotate((int)TypeCode::NOTYPE, -1,
ctx.st.currentLev());
}

void visit_assignment_statement(Node* node,
SemanticContext& ctx) {
    // std::cout << "masuk assignment statement" <<
std::endl;
    if (!node) return;

    // anak: <variable> becomes <expression>
    // Validasi: tipe lhs dan rhs harus
assignment-compatible

    Node* varNode = nullptr;
    Node* exprNode = nullptr;

    for (Node* c : node->children) {
        if (c->label == "<variable>" && !varNode)
varNode = c;
        else if (c->label == "<expression>" &&
!exprNode) exprNode = c;
    }

    if (!varNode || !exprNode) {
        ctx.errors.add("Assignment statement tidak
lengkap");
        return;
    }
}

```

```

    TypeInfo lhsType = visit_variable(varNode, ctx);
    TypeInfo rhsType = visit_expression(exprNode, ctx);

    if (!isAssignmentCompatible(lhsType, rhsType,
ctx.st)) {
        ctx.errors.add("Tipe assignment tidak
kompatibel: "
                        "lhs adalah " +
lhsType.toString() +
                        ", rhs adalah " +
rhsType.toString());
    }

    Node* nameNode = findChildByTokenType(varNode,
"ident");
    auto assignAst = makeAstNode("Assign", nameNode ?
nodeTokenValue(nameNode) : "");
    assignAst->addChild(build_variable_ast(varNode));
assignAst->addChild(build_expression_ast(exprNode));
    attachAstNode(assignAst);

    node->annotate((int)TypeCode::NOTYPE, -1,
ctx.st.currentLev());
}

```

Fungsi `visit_statement()` adalah dispatcher utama untuk semua jenis statement. Jika `node->children` kosong, fungsi langsung return tanpa melakukan apapun (empty statement). Jika tidak kosong, child pertama diambil dan labelnya diperiksa. Setiap label yang dikenali didelegasikan ke visitor yang sesuai. Untuk `<procedure/function-call>`, setelah `visit_procedure_function_call()` selesai, `build_proc_call_ast()` dipanggil untuk membangun

ASTNode dan di-attach. Untuk <compound-statement>, pendingBlockParent diperbarui sebelum memanggil visit_compound_statement().

Fungsi visit_assignment_statement() mengekstrak node <variable> (LHS) dan node <expression> (RHS) dari children. Keduanya diproses untuk mendapatkan TypeInfo: LHS via visit_variable() dan RHS via visit_expression(). isAssignmentCompatible(lhsType, rhsType) kemudian dipanggil. Jika tidak compatible, error ditambahkan dengan pesan yang menyebutkan tipe LHS dan RHS secara spesifik. Terlepas dari hasil validasi, ASTNode bertipe "Assign" tetap dibangun menggunakan build_variable_ast() dan build_expression_ast(), dan di-attach ke parent aktif. Keputusan ini memastikan bahwa AST tetap dibangun meskipun ada error, sehingga output AST tetap tersedia untuk debugging meskipun ada kesalahan semantik.

```
void visit_if_statement(Node* node, SemanticContext&
ctx) {
    if (!node) return;

    // anak: ifsy <expression> thensy <statement>
    (elsesy <statement>)?
    Node* condNode = nullptr;
    Node* thenStmt = nullptr;
    Node* elseStmt = nullptr;

    for (Node* c : node->children) {
        if (c->label == "<expression>" && !condNode) {
            condNode = c;
        } else if (c->label == "<statement>" &&
!thenStmt) {
            thenStmt = c;
        } else if (c->label == "<statement>" &&
thenStmt && !elseStmt) {
            elseStmt = c;
        }
    }
}
```

```

    }

}

if (!condNode || !thenStmt) {
    ctx.errors.add("If statement tidak lengkap");
    return;
}

TypeInfo condType = visit_expression(condNode,
ctx);

if (condType.baseType != TypeCode::BOOLEAN) {
    ctx.errors.add("Kondisi if harus bertipe
Boolean, ditemukan: " + condType.toString());
}

auto ifAst = makeAstNode("If");
auto condAst = makeAstNode("Condition");
condAst->addChild(build_expression_ast(condNode));
ifAst->addChild(condAst);

auto thenAst = makeAstNode("Then");
ifAst->addChild(thenAst);

auto prevOverride = g_astCtx.attachOverride;
g_astCtx.attachOverride = thenAst;
visit_statement(thenStmt, ctx);
g_astCtx.attachOverride = prevOverride;

if (elseStmt) {
    auto elseAst = makeAstNode("Else");
    ifAst->addChild(elseAst);
}

```

```

        g_astCtx.attachOverride = elseAst;
        visit_statement(elseStmt, ctx);
        g_astCtx.attachOverride = prevOverride;
    }

    attachAstNode(ifAst);

    node->annotate( (int) TypeCode::NOTYPE, -1,
ctx.st.currentLev() );
}

```

Fungsi `visit_if_statement()` mengekstrak tiga komponen dari children node `<if-statement>`: node `<expression>` pertama sebagai kondisi, node `<statement>` pertama sebagai then-branch, dan node `<statement>` kedua (jika ada) sebagai else-branch. Kondisi divalidasi harus bertipe `BOOLEAN` via `visit_expression()`. `ASTNode` "If" dibangun dengan child "Condition" yang berisi ekspresi kondisi. Kemudian `ASTNode` "Then" dibuat dan ditambahkan sebagai child dari "If". Untuk memastikan statement yang dikunjungi saat memproses then-branch menempel ke "Then" bukan ke block aktif, `g_astCtx.attachOverride` diset ke node "Then" sebelum `visit_statement(thenStmt)` dipanggil, lalu di-reset setelahnya. Pola yang sama diterapkan untuk else-branch jika ada, dengan `attachOverride` diset ke node "Else". Setelah kedua branch selesai, node "If" di-attach ke parent aktif via `attachAstNode()`.

```

void visit_while_statement(Node* node, SemanticContext&
ctx) {
    if (!node) return;

    //      anak:      whilesy      <expression>      dosy
<compound-statement> semicolon
    Node* condNode = nullptr;
    Node* compoundStmt = nullptr;

    for (Node* c : node->children) {

```

```

        if (c->label == "<expression>" && !condNode) {
            condNode = c;
        } else if (c->label == "<compound-statement>"
&& !compoundStmt) {
            compoundStmt = c;
        }
    }

    if (!condNode || !compoundStmt) {
        ctx.errors.add("While statement tidak
lengkap");
        return;
    }

    TypeInfo condType = visit_expression(condNode,
ctx);

    if (condType.baseType != TypeCode::BOOLEAN) {
        ctx.errors.add("Kondisi while harus bertipe
Boolean, ditemukan: " + condType.toString());
    }

    auto whileAst = makeAstNode("While");
    auto condAst = makeAstNode("Condition");
    condAst->addChild(build_expression_ast(condNode));
    whileAst->addChild(condAst);

    auto prevPending = g_astCtx.pendingBlockParent;
    g_astCtx.pendingBlockParent = whileAst;
    visit_compound_statement(compoundStmt, ctx);
    g_astCtx.pendingBlockParent = prevPending;

```

```

        attachAstNode(whileAst);

        node->annotate((int)TypeCode::NOTYPE, -1,
ctx.st.currentLev());
    }

void visit_repeat_statement(Node* node,
SemanticContext& ctx) {
    if (!node) return;

    // anak: repeatsy <statement-list> untilsy
<expression>
    Node* stmtListNode = nullptr;
    Node* condNode = nullptr;

    for (Node* c : node->children) {
        if (c->label == "<statement-list>" &&
!stmtListNode) {
            stmtListNode = c;
        } else if (c->label == "<expression>" &&
!condNode) {
            condNode = c;
        }
    }

    if (!stmtListNode || !condNode) {
        ctx.errors.add("Repeat statement tidak
lengkap");
        return;
    }

    auto repeatAst = makeAstNode("Repeat");

```

```

    auto bodyAst = makeAstNode("Block");
    bodyAst->block_index = ctx.st.currentBlock();
    bodyAst->lev = ctx.st.currentLev();
    repeatAst->addChild(bodyAst);

    auto prevOverride = g_astCtx.attachOverride;
    g_astCtx.attachOverride = bodyAst;
    visit_statement_list(stmtListNode, ctx);
    g_astCtx.attachOverride = prevOverride;

    TypeInfo condType = visit_expression(condNode,
ctx);

    if (condType.baseType != TypeCode::BOOLEAN) {
        ctx.errors.add("Kondisi until harus bertipe
Boolean, ditemukan: " + condType.toString());
    }

    auto condAst = makeAstNode("Condition");
    condAst->addChild(build_expression_ast(condNode));
    repeatAst->addChild(condAst);

    attachAstNode(repeatAst);

    node->annotate((int) TypeCode::NOTYPE, -1,
ctx.st.currentLev());
}

```

Fungsi `visit_while_statement()` mengharapkan body berupa <compound-statement> (bukan arbitrary <statement>) karena parser `parseWhileStatement()` secara khusus memanggil `parseCompoundStatement()` bukan `parseStatement()`. Ini adalah batasan implementasi parser: body while dan for wajib menggunakan `begin...end`. Kondisi divalidasi

harus BOOLEAN. ASTNode "While" dibangun dengan child "Condition". Untuk memproses body compound statement agar menempel ke node "While", pendingBlockParent diset ke whileAst sebelum visit_compound_statement() dipanggil. Fungsi visit_repeat_statement() berbeda dari visit_while_statement() dalam dua hal: body diproses terlebih dahulu sebelum kondisi (sesuai semantik repeat...until), dan body adalah <statement-list> bukan <compound-statement>. Body diproses dengan attachOverride diset ke ASTNode "Block" internal, lalu kondisi diproses dan divalidasi harus BOOLEAN.

```
void visit_for_statement(Node* node, SemanticContext&
ctx) {
    if (!node) return;

    // anak: forsy ident becomes <expression>
    (tosy|downtosy) <expression> dosy <compound-statement>
    semicolon

    Node* loopVarNode = nullptr;
    Node* initExprNode = nullptr;
    Node* finalExprNode = nullptr;
    Node* compoundStmt = nullptr;

    std::string direction = ""; // ini "to" ato
    "downto"

    int exprCount = 0;
    for (Node* c : node->children) {
        if (nodeTokenType(c) == "ident" &&
!loopVarNode) {
            loopVarNode = c;
        } else if (c->label == "<expression>" &&
exprCount == 0) {
            initExprNode = c;
            exprCount++;
        }
    }
    if (loopVarNode == nullptr || initExprNode == nullptr ||
compoundStmt == nullptr) {
        return;
    }
    loopVarNode->attachOverride(loopVarNode->internalBlock);
    initExprNode->attachOverride(initExprNode->internalBlock);
    compoundStmt->attachOverride(compoundStmt->internalBlock);
    visit_compound_statement(compoundStmt, ctx);
}
```

```

        } else if (c->label == "<expression>" &&
exprCount == 1) {
            finalExprNode = c;
            exprCount++;
        } else if (c->label == "<compound-statement>"
&& !compoundStmt) {
            compoundStmt = c;
        } else if (nodeTokenType(c) == "tosy") {
            direction = "to";
        } else if (nodeTokenType(c) == "downtosy") {
            direction = "downto";
        }
    }

    if (!loopVarNode || !initExprNode || !finalExprNode
|| !compoundStmt || direction.empty()) {
        ctx.errors.add("For statement tidak lengkap");
        return;
    }

    // Lookup loop variable
    std::string loopVarName =
nodeTokenValue(loopVarNode);
    int loopVarIdx = ctx.st.lookup(loopVarName);
    if (loopVarIdx < 0) {
        ctx.errors.add("Variabel loop '" + loopVarName
+ "'" tidak ditemukan");
        return;
    }

    TypeInfo loopVarType = typeInfoFromTab(loopVarIdx,
ctx.st);

```

```

    // Validasi: loop variable harus ordinal
    if (!loopVarType.isOrdinal()) {
        ctx.errors.add("Variabel loop harus bertipe
ordinal, ditemukan: " + loopVarType.toString());
    }

    TypeInfo initType = visit_expression(initExprNode,
ctx);

    TypeInfo finalType =
visit_expression(finalExprNode, ctx);

    if (!isAssignmentCompatible(loopVarType, initType,
ctx.st)) {
        ctx.errors.add("Tipe initial expression tidak
kompatibel dengan loop variable: "
            + "loop var adalah " +
loopVarType.toString() +
            ", initial adalah " +
initType.toString());
    }

    if (!isAssignmentCompatible(loopVarType, finalType,
ctx.st)) {
        ctx.errors.add("Tipe final expression tidak
kompatibel dengan loop variable: "
            + "loop var adalah " +
loopVarType.toString() +
            ", final adalah " +
finalType.toString());
    }

```

```

    auto forAst = makeAstNode("For");

        auto    initAssign    =    makeAstNode("Assign",
loopVarName);
    auto targetVar = makeAstNode("Var", loopVarName);
    annotateFromParse(targetVar, loopVarNode);
    initAssign->addChild(targetVar);

initAssign->addChild(build_expression_ast(initExprNode)
);
    forAst->addChild(initAssign);

    auto limitAst = makeAstNode("Limit", direction);

limitAst->addChild(build_expression_ast(finalExprNode))
;
    forAst->addChild(limitAst);

    auto prevPending = g_astCtx.pendingBlockParent;
    g_astCtx.pendingBlockParent = forAst;
    visit_compound_statement(compoundStmt, ctx);
    g_astCtx.pendingBlockParent = prevPending;

    attachAstNode(forAst);

        node->annotate( (int) TypeCode::NOTYPE,    -1,
ctx.st.currentLev() );
}

```

Fungsi `visit_for_statement()` memiliki lebih banyak komponen yang harus diekstrak dibandingkan statement lain: variabel loop (token `ident`), ekspresi initial (node `<expression>`)

pertama), arah iterasi (token tosy atau downtosy), ekspresi final (node <expression> kedua), dan body (node <compound-statement>). Variabel loop dicari via lookup() dan divalidasi harus bertipe ordinal. Ekspresi initial dan final diproses via visit_expression() dan masing-masing divalidasi assignment-compatible dengan tipe variabel loop. ASTNode "For" dibangun dengan child "Assign" (initial assignment ke variabel loop), "Limit" dengan text = "to" atau "downto" (batas akhir), dan "Block" (body loop). Untuk body, pendingBlockParent diset ke forAst sebelum visit_compound_statement() dipanggil.

```
void visit_case_statement(Node* node, SemanticContext&
ctx) {
    if (!node) return;

    // anak: casesy <expression> ofsy <case-list> endsy
    Node* exprNode = nullptr;
    Node* caseListNode = nullptr;

    for (Node* c : node->children) {
        if (c->label == "<expression>" && !exprNode) {
            exprNode = c;
        } else if (c->label == "<case-list>" &&
!caseListNode) {
            caseListNode = c;
        }
    }

    if (!exprNode || !caseListNode) {
        ctx.errors.add("Case statement tidak lengkap");
        return;
    }

    TypeInfo exprType = visit_expression(exprNode,
ctx);
```

```

    if (!exprType.isOrdinal()) {
        ctx.errors.add("Case expression harus bertipe ordinal, ditemukan: " + exprType.toString());
    }

    auto caseAst = makeAstNode("Case");
    auto exprAst = makeAstNode("Expression");
    exprAst->addChild(build_expression_ast(exprNode));
    caseAst->addChild(exprAst);

    auto prevOverride = g_astCtx.attachOverride;
    g_astCtx.attachOverride = caseAst;
    visit_case_list(caseListNode, ctx, exprType);
    g_astCtx.attachOverride = prevOverride;

    attachAstNode(caseAst);

    node->annotate((int)TypeCode::NOTYPE, -1,
ctx.st.currentLev());
}

void visit_case_list(Node* node, SemanticContext& ctx,
const TypeInfo& exprType) {
    if (!node) return;

    // anak: <case-branch> (semicolon <case-branch>)*
    for (Node* c : node->children) {
        if (c->label == "<case-branch>") {
            visit_case_branch(c, ctx, exprType);
        }
    }
}

```

```

        node->annotate((int) TypeCode::NOTYPE, -1,
ctx.st.currentLev());
}

void visit_case_branch(Node* node, SemanticContext&
ctx, const TypeInfo& exprType) {
    if (!node) return;

    // anak: <constant-list> colon <statement>
    Node* constListNode = nullptr;
    Node* stmtNode = nullptr;

    for (Node* c : node->children) {
        if (c->label == "<constant-list>" &&
!constListNode) {
            constListNode = c;
        } else if (c->label == "<statement>" &&
!stmtNode) {
            stmtNode = c;
        }
    }

    if (!constListNode || !stmtNode) {
        ctx.errors.add("Case branch tidak lengkap");
        return;
    }

    if (constListNode) {
        for (Node* c : constListNode->children) {
            if (c->label == "<constant>") {

```

```

                                TypeInfo constType =
visit_constant_value(c, ctx);
                                if (!isCompatible(constType, exprType,
ctx.st)) {
                                    ctx.errors.add("Tipe konstanta case
tidak kompatibel dengan case expression: "
                                                "konstanta adalah "
+ constType.toString() +
                                                ", expression adalah
" + exprType.toString());
                                    }
                                }
                            }
                    }

    auto blockAst = makeAstNode("CaseBlock");
    if (constListNode) {
        for (Node* c : constListNode->children) {
            if (c->label == "<constant>") {
blockAst->addChild(build_constant_ast(c));
            }
        }
    }

    auto prevOverride = g_astCtx.attachOverride;
    g_astCtx.attachOverride = blockAst;
    visit_statement(stmtNode, ctx);
    g_astCtx.attachOverride = prevOverride;

    attachAstNode(blockAst);

```



```

        node->annotate( (int) TypeCode::NOTYPE, -1,
ctx.st.currentLev() );
}

```

Fungsi `visit_case_statement()` mengekstrak ekspresi selektor dan node `<case-list>`. Selektor divalidasi harus bertipe ordinal, dan tipe selektor diteruskan ke `visit_case_list()` sebagai parameter tambahan agar setiap branch dapat memvalidasi kompatibilitas label konstantanya. ASTNode "Case" dibangun dengan child "Expression", dan `attachOverride` diset ke `caseAst` saat `visit_case_list()` dipanggil agar node-node branch menempel ke "Case".

Fungsi `visit_case_list()` mengiterasi setiap `<case-branch>` dan mendelegasikan ke `visit_case_branch()` dengan tipe selektor. Fungsi `visit_case_branch()` mengekstrak `<constant-list>` (daftar label nilai) dan `<statement>` (body branch). Untuk setiap konstanta di `<constant-list>`, `visit_constant_value()` dipanggil dan hasilnya divalidasi compatible dengan tipe selektor. ASTNode "CaseBlock" dibangun dengan node literal dari setiap konstanta sebagai child pertama, diikuti statement body yang diproses dengan `attachOverride` diset ke "CaseBlock".

f. Fungsi Build AST

```

static std::shared_ptr<ASTNode>
build_simple_expression_ast(Node* node) {
    if (!node) return nullptr;
    if (node->label != "<simple-expression>") return
build_term_ast(node);

    std::shared_ptr<ASTNode> current = nullptr;
    std::string pendingOp;
    bool firstOperand = true;
    bool unaryMinus = false;
    bool unaryPlus = false;

    for (Node* c : node->children) {

```

```

        std::string tt;
        if (c->getLabel() == "<additive-operator>" &&
!c->children.empty()) {
            tt = nodeTokenType(c->children[0]);
        }

        if (tt == "plus" || tt == "minus" || tt ==
"orsy") {
            if (firstOperand && !current) {
                if (tt == "minus") unaryMinus = true;
                if (tt == "plus") unaryPlus = true;
            } else {
                pendingOp = tt;
            }
            continue;
        }

        if (c->label == "<term>") {
            auto operand = build_term_ast(c);
            if (firstOperand) {
                if (unaryMinus || unaryPlus) {
                    auto un = makeAstNode("UnaryOp",
unaryMinus ? "-" : "+");
                    un->addChild(operand);
                    current = un;
                } else {
                    current = operand;
                }
                firstOperand = false;
            } else if (!pendingOp.empty()) {
                auto bin = makeAstNode("BinOp",
opToText(pendingOp));

```

```

        bin->addChild(current);
        bin->addChild(operand);
        annotateFromParse(bin, node);
        current = bin;
        pendingOp.clear();
    }
}

return current;
}

static                                     std::shared_ptr<ASTNode>
build_expression_ast(Node* node) {
    if (!node) return nullptr;

    if (node->label != "<expression>") return
build_simple_expression_ast(node);

    std::vector<Node*> simpleExprs;
    std::string relOp;

    for (Node* c : node->children) {
        if (c->label == "<simple-expression>") {
            simpleExprs.push_back(c);
        } else if (c->label == "<relational-operator>")
    {
        for (Node* rc : c->children) {
            std::string rtt = nodeTokenType(rc);
            if (rtt == "eq1" || rtt == "neq" || rtt
== "lss" || rtt == "leq" ||
                rtt == "gtr" || rtt == "geq") {

```

```

        relOp = rtt;
        break;
    }
}
} else {
    std::string tt = nodeTokenType(c);
    if (tt == "eq1" || tt == "neq" || tt ==
"lss" || tt == "leq" ||
        tt == "gtr" || tt == "geq") {
        relOp = tt;
    }
}

if (simpleExprs.size() == 1) {
                                                                    return
build_simple_expression_ast(simpleExprs[0]);
}

if (simpleExprs.size() == 2 && !relOp.empty()) {
        auto    left    =
build_simple_expression_ast(simpleExprs[0]);
        auto    right   =
build_simple_expression_ast(simpleExprs[1]);
        auto    bin     = makeAstNode("BinOp",
opToText(relOp));
        bin->addChild(left);
        bin->addChild(right);
        annotateFromParse(bin, node);
        return bin;
}

```

```

        if (!simpleExprs.empty()) return
build_simple_expression_ast(simpleExprs[0]);
    return nullptr;
}

```

Fungsi-fungsi `build_*_ast()` bertugas membangun subtree `ASTNode` dari parse tree setelah validasi semantik selesai. Fungsi-fungsi ini tidak melakukan validasi apapun (validasi sudah dilakukan oleh fungsi `visit_*`) melainkan hanya mengkonversi struktur parse tree yang verbose menjadi representasi AST yang ringkas dan bermakna.

Fungsi `build_expression_ast()` mengumpulkan `<simple-expression>` dan operator relasional dari children. Jika satu `simple-expression`, didelegasikan ke `build_simple_expression_ast()`. Jika dua dengan operator relasional, dibuat node "BinOp" dengan operator dikonversi via `opToText()`, left child dari `simple-expression` pertama, dan right child dari `simple-expression` kedua. Anotasi semantik disalin dari parse node via `annotateFromParse()`.

Fungsi `build_simple_expression_ast()` mengidentifikasi operator additive dan operand `<term>` secara bergantian. Operator sebelum operand pertama diperlakukan sebagai unary dan menghasilkan "UnaryOp". Operator antara dua operand menghasilkan "BinOp" yang dibentuk secara left-associative: node "BinOp" baru dibuat dengan current sebagai left child dan operand baru sebagai right child, lalu current diperbarui ke node baru. Dengan cara ini, ekspresi `a + b + c` menghasilkan pohon `BinOp('+', BinOp('+', a, b), c)` yang benar secara evaluasi.

```

static std::shared_ptr<ASTNode> build_factor_ast(Node*
node) {
    if (!node) return nullptr;

    if (node->label == "<factor>") {
        if (node->children.empty()) return nullptr;
        Node* first = node->children[0];
        std::string tt = nodeTokenType(first);

```

```

        if (tt == "notsy") {
            if (node->children.size() < 2) return
nullptr;

            auto un = makeAstNode("UnaryOp", "not");

un->addChild(build_factor_ast(node->children[1]));
            annotateFromParse(un, node);
            return un;
        }

        if (tt == "lparent") {
            for (Node* c : node->children) {
                if (c->label == "<expression>") {
                    return build_expression_ast(c);
                }
            }
        }

        if (first->label == "<variable>") {
            return build_variable_ast(first);
        }

        if (first->label ==
"<procedure/function-call>") {
            return build_proc_call_ast(first);
        }

        if (tt == "intcon" || tt == "realcon" || tt ==
"charcon" || tt == "string") {
            std::string lit = nodeTokenValue(first);
            if (tt == "string" || tt == "charcon") {

```

```

        lit = "'" + lit + "'";
    }
    auto litNode = makeAstNode("Literal", lit);
    annotateFromParse(litNode, first);
    return litNode;
}

    if (tt == "ident") {
        auto v = makeAstNode("Var",
nodeTokenValue(first));
        annotateFromParse(v, first);
        return v;
    }

    return build_factor_ast(first);
}

    std::string tt = nodeTokenType(node);
    if (tt == "intcon" || tt == "realcon" || tt ==
"charcon" || tt == "string") {
        std::string lit = nodeTokenValue(node);
        if (tt == "string" || tt == "charcon") {
            lit = "'" + lit + "'";
        }
        auto litNode = makeAstNode("Literal", lit);
        annotateFromParse(litNode, node);
        return litNode;
    }
    if (tt == "ident") {
        auto v = makeAstNode("Var",
nodeTokenValue(node));
        annotateFromParse(v, node);
    }

```

```

        return v;
    }

    return nullptr;
}

static std::shared_ptr<ASTNode> build_term_ast(Node*
node) {
    if (!node) return nullptr;
        if (node->label != "<term>") return
build_factor_ast(node);

    std::shared_ptr<ASTNode> current = nullptr;
    std::string pendingOp;

    for (Node* c : node->children) {
        std::string tt;

        if (c->getLabel() ==
"<multiplicative-operator>" && !c->children.empty()) {
            tt = nodeTokenType(c->children[0]);
        }

        if (tt == "times" || tt == "rdiv" || tt ==
"idiv" || tt == "imod" || tt == "andsy") {
            pendingOp = tt;
            continue;
        }

        if (c->label == "<factor>") {
            auto operand = build_factor_ast(c);
            if (!current) {
                current = operand;
            } else if (!pendingOp.empty()) {

```



```

        auto bin = makeAstNode("BinOp",
opToText(pendingOp));
        bin->addChild(current);
        bin->addChild(operand);
        annotateFromParse(bin, node);
        current = bin;
        pendingOp.clear();
    }
}

return current;
}

```

Fungsi `build_term_ast()` bekerja identik dengan `build_simple_expression_ast()` namun untuk operator multiplicative, membangun pohon biner left-associative. Fungsi `build_factor_ast()` menangani berbagai kemungkinan jenis faktor: `notsy` menghasilkan "UnaryOp" dengan `text = "not"`; `lparent` mendelegasikan ke `build_expression_ast()` untuk ekspresi dalam kurung; `<variable>` mendelegasikan ke `build_variable_ast()`; literal `intcon/realcon/charcon/string` menghasilkan "Literal" dengan nilai diformat (string dan `charcon` diberi tanda kutip tunggal); `ident` menghasilkan "Var" dengan nama identifier.

```

static std::shared_ptr<ASTNode>
build_variable_ast(Node* node) {
    if (!node) return nullptr;
    Node* identNode = nullptr;
    Node* componentNode = nullptr;
    for (Node* c : node->children) {
        if (nodeTokenType(c) == "ident" && !identNode)
identNode = c;
        if (c->label == "<component-variable>")
componentNode = c;
    }
}

```

```

        std::string name = identNode ?
nodeTokenValue(identNode) : "";
        auto varNode = makeAstNode("Var", name);
        annotateFromParse(varNode, identNode ? identNode :
node);

        if (componentNode) {
            for (int i = 0; i <
(int) componentNode->children.size(); i++) {
                Node* c = componentNode->children[i];
                if (nodeTokenType(c) == "lbrack") {
                    for (int j = i + 1; j <
(int) componentNode->children.size(); j++) {
                        Node* sub =
componentNode->children[j];
                        if (sub->label == "<expression>") {
                            auto idxAst =
makeAstNode("Index");
idxAst->addChild(build_expression_ast(sub));
                            varNode->addChild(idxAst);
                            i = j;
                            break;
                        }
                        if (nodeTokenType(sub) == "rbrack")
{ i = j; break; }
                    }
                } else if (nodeTokenType(c) == "period") {
                    if (i + 1 <
(int) componentNode->children.size() &&

```

```

nodeTokenType(componentNode->children[i + 1]) ==
"ident") {
    Node* fieldNode =
componentNode->children[i + 1];
    auto fieldAst =
makeAstNode("Field", nodeTokenValue(fieldNode));
    annotateFromParse(fieldAst,
fieldNode);
    varNode->addChild(fieldAst);
}
}
}
}

return varNode;
}

static std::shared_ptr<ASTNode>
build_proc_call_ast(Node* node) {
    if (!node) return nullptr;
    Node* identNode = findChildByTokenType(node,
"ident");
    std::string name = identNode ?
nodeTokenValue(identNode) : "";
    auto call = makeAstNode("ProcCall", name);
    annotateFromParse(call, identNode ? identNode :
node);

    std::vector<Node*> args;
    for (Node* c : node->children) {
        if (c->label == "<parameter-list>" || c->label
== "<actual-parameter-list>") {

```

```

        for (Node* pc : c->children) {
            if (pc->label == "<expression>" ||
pc->label == "<actual-parameter>") {
                args.push_back(pc);
            }
        }

        if (c->label == "<expression>")
args.push_back(c);
    }

    auto argsNode = makeAstNode("Args");
    for (Node* arg : args) {
        if (arg->label == "<actual-parameter>") {
            for (Node* ac : arg->children) {
                if (ac->label == "<expression>") {

argsNode->addChild(build_expression_ast(ac));

                }
            }
        } else {

argsNode->addChild(build_expression_ast(arg));

        }
    }

    call->addChild(argsNode);
    return call;
}

```

Fungsi `build_variable_ast()` membuat node "Var" dengan nama identifier yang diambil dari token ident pertama. Kemudian iterasi dilakukan pada node `<component-variable>` untuk menambahkan child yang merepresentasikan akses komponen. Untuk setiap token

lbrack dalam <component-variable>, node "Index" dibuat dengan ekspresi indeks sebagai child. Untuk setiap token period, node "Field" dibuat dengan nama field sebagai text. Hasilnya adalah "Var" node yang memiliki children "Index" dan/atau "Field" yang merepresentasikan seluruh ekspresi akses variabel secara hierarkis.

Fungsi `build_proc_call_ast()` membuat node "ProcCall" dengan nama prosedur/fungsi. Semua argumen aktual dikumpulkan dari <parameter-list> dan dimasukkan ke node "Args" yang menjadi satu-satunya child dari "ProcCall". Setiap argumen dibangun via `build_expression_ast()`.

```
static                                     std::shared_ptr<ASTNode>
build_compound_statement_ast(Node*      node,      const
SymbolTable& st) {
    if (!node) return nullptr;
    auto block = makeAstNode("Block");
    block->block_index = st.currentBlock();
    block->lev = st.currentLev();

    for (Node* c : node->children) {
        if (c->label == "<statement-list>") {
            for (Node* sc : c->children) {
                if (sc->label == "<statement>") {
                    auto stmt = build_statement_ast(sc,
st);

                    if (stmt) block->addChild(stmt);
                }
            }
        }
    }

    return block;
}
```

```

static                                     std::shared_ptr<ASTNode>
build_statement_ast(Node* node, const SymbolTable& st)
{
    if (!node || node->children.empty()) return
nullptr;
    Node* child = node->children[0];

    if (child->label == "<assignment-statement>") {
        Node* varNode = nullptr;
        Node* exprNode = nullptr;
        for (Node* c : child->children) {
            if (c->label == "<variable>" && !varNode)
varNode = c;
            else if (c->label == "<expression>" &&
!exprNode) exprNode = c;
        }
        if (!varNode || !exprNode) return nullptr;
        Node* nameNode = findChildByTokenType(varNode,
"ident");
        auto assign = makeAstNode("Assign", nameNode ?
nodeTokenValue(nameNode) : "");
        assign->addChild(build_variable_ast(varNode));
assign->addChild(build_expression_ast(exprNode));
        return assign;
    }

    if (child->label == "<procedure/function-call>") {
        return build_proc_call_ast(child);
    }
}

```

```

    if (child->label == "<compound-statement>") {
        return build_compound_statement_ast(child, st);
    }

    if (child->label == "<if-statement>") {
        auto ifNode = makeAstNode("If");
        Node* condNode = nullptr;
        Node* thenStmt = nullptr;
        Node* elseStmt = nullptr;
        for (Node* c : child->children) {
            if (c->label == "<expression>" &&
!condNode) condNode = c;
            else if (c->label == "<statement>" &&
!thenStmt) thenStmt = c;
            else if (c->label == "<statement>" &&
thenStmt && !elseStmt) elseStmt = c;
        }
        auto cond = makeAstNode("Condition");
        cond->addChild(build_expression_ast(condNode));
        ifNode->addChild(cond);
        auto thenNode = makeAstNode("Then");
        thenNode->addChild(build_statement_ast(thenStmt, st));
        ifNode->addChild(thenNode);
        if (elseStmt) {
            auto elseNode = makeAstNode("Else");
            elseNode->addChild(build_statement_ast(elseStmt, st));
            ifNode->addChild(elseNode);
        }
        return ifNode;
    }
}

```

```

    if (child->label == "<while-statement>") {
        auto whileNode = makeAstNode("While");
        Node* condNode = nullptr;
        Node* bodyNode = nullptr;
        for (Node* c : child->children) {
            if (c->label == "<expression>" &&
!condNode) condNode = c;
            else if (c->label == "<compound-statement>"
&& !bodyNode) bodyNode = c;
        }
        auto cond = makeAstNode("Condition");
        cond->addChild(build_expression_ast(condNode));
        whileNode->addChild(cond);

whileNode->addChild(build_compound_statement_ast(bodyNode, st));
        return whileNode;
    }

    if (child->label == "<repeat-statement>") {
        auto repeatNode = makeAstNode("Repeat");
        Node* stmtListNode = nullptr;
        Node* condNode = nullptr;
        for (Node* c : child->children) {
            if (c->label == "<statement-list>" &&
!stmtListNode) stmtListNode = c;
            else if (c->label == "<expression>" &&
!condNode) condNode = c;
        }
        auto body = makeAstNode("Block");
        if (stmtListNode) {

```



```

        for (Node* sc : stmtListNode->children) {
            if (sc->label == "<statement>") {
body->addChild(build_statement_ast(sc, st));
            }
        }
    }

    repeatNode->addChild(body);
    auto cond = makeAstNode("Condition");
    cond->addChild(build_expression_ast(condNode));
    repeatNode->addChild(cond);
    return repeatNode;
}

if (child->label == "<for-statement>") {
    auto forNode = makeAstNode("For");
    Node* loopVarNode = nullptr;
    Node* initExprNode = nullptr;
    Node* finalExprNode = nullptr;
    Node* compoundStmt = nullptr;
    std::string direction;
    int exprCount = 0;
    for (Node* c : child->children) {
        if (nodeTokenType(c) == "ident" &&
!loopVarNode) loopVarNode = c;
        else if (c->label == "<expression>" &&
exprCount == 0) { initExprNode = c; exprCount++; }
        else if (c->label == "<expression>" &&
exprCount == 1) { finalExprNode = c; exprCount++; }
        else if (c->label == "<compound-statement>"
&& !compoundStmt) compoundStmt = c;

```

```

        else if (nodeTokenType(c) == "tosy")
direction = "to";
        else if (nodeTokenType(c) == "downtosy")
direction = "downto";
    }
    if (loopVarNode && initExprNode) {
        auto assign = makeAstNode("Assign",
nodeTokenValue(loopVarNode));
        auto target = makeAstNode("Var",
nodeTokenValue(loopVarNode));
        annotateFromParse(target, loopVarNode);
        assign->addChild(target);

assign->addChild(build_expression_ast(initExprNode));
        forNode->addChild(assign);
    }
    if (finalExprNode) {
        auto limit = makeAstNode("Limit",
direction);

limit->addChild(build_expression_ast(finalExprNode));
        forNode->addChild(limit);
    }

forNode->addChild(build_compound_statement_ast(compound
Stmt, st));
    return forNode;
}

if (child->label == "<case-statement>") {
    auto caseNode = makeAstNode("Case");
    Node* exprNode = nullptr;

```

```

    Node* caseListNode = nullptr;
    for (Node* c : child->children) {
        if (c->label == "<expression>" &&
!exprNode) exprNode = c;
        else if (c->label == "<case-list>" &&
!caseListNode) caseListNode = c;
    }
    auto exprAst = makeAstNode("Expression");
exprAst->addChild(build_expression_ast(exprNode));
    caseNode->addChild(exprAst);
    if (caseListNode) {
        for (Node* cb : caseListNode->children) {
            if (cb->label != "<case-branch>")
continue;

            auto block = makeAstNode("CaseBlock");
            Node* constListNode = nullptr;
            Node* stmtNode = nullptr;
            for (Node* c : cb->children) {
                if (c->label == "<constant-list>"
&& !constListNode) constListNode = c;
                else if (c->label == "<statement>"
&& !stmtNode) stmtNode = c;
            }
            if (constListNode) {
                for (Node* cc :
constListNode->children) {
                    if (cc->label == "<constant>")
{
                        auto lit =
build_constant_ast(cc);
                        block->addChild(lit);

```

```

    }
    }
    }
    if (stmtNode) {
block->addChild(build_statement_ast(stmtNode, st));
    }
    caseNode->addChild(block);
    }
}
return caseNode;
}

return nullptr;
}

```

Fungsi `build_statement_ast()` mendispatch ke builder yang sesuai berdasarkan label child pertama dari `<statement>`. Setiap jenis statement dikonversi menjadi `ASTNode` yang merepresentasikan struktur semantiknya: assignment menjadi "Assign" dengan target variabel dan ekspresi nilai, if menjadi "If" dengan "Condition"/"Then"/opsional "Else", while menjadi "While" dengan "Condition" dan body block, for menjadi "For" dengan "Assign" initial/"Limit" akhir/body block, repeat menjadi "Repeat" dengan body block dan "Condition", case menjadi "Case" dengan "Expression" selektor dan "CaseBlock" untuk setiap branch.

Fungsi `build_compound_statement_ast()` membuat node "Block" dengan `block_index` dan `lev` dari symbol table saat ini, lalu mengiterasi semua `<statement>` dalam `<statement-list>` dan memanggil `build_statement_ast()` untuk setiap satu. Hasilnya adalah representasi AST yang bersih dari blok program yang hanya berisi statement-statement bermakna tanpa token `beginsy`, `endsy`, dan `semicolon`.

2.11 Implementasi Writer dan `main.cpp`

Writer dan main.cpp: src/io/writer.hpp, src/io/writer.cpp, src/main.cpp

```
void PRINT_SYMBOL_TABLE(const SymbolTable& st) {
    // TAB
    cout << "[tab]" << endl;
    cout << setw(6) << "Index"
        << setw(20) << "Identifier"
        << setw(15) << "Object"
        << setw(15) << "Type"
        << setw(10) << "Ref"
        << setw(8) << "Nrm"
        << setw(8) << "Lev"
        << setw(10) << "Address" << endl;

    for (int i = 0; i < (int)st.tab.size(); i++) {
        const TabEntry& entry = st.tab[i];
        cout << setw(6) << i
            << setw(20) << entry.identifier.substr(0,
19)
                << setw(15) << objClassToString(entry.obj)
                << setw(15) << typeCodeToString(entry.type)
                << setw(10) << entry.ref
                << setw(8) << entry.nrm
                << setw(8) << entry.lev
                << setw(10) << entry.adr << endl;
    }

    // ATAB
    cout << "[atab]" << endl;
    cout << setw(6) << "Index"
        << setw(15) << "Index Type"
        << setw(15) << "Element Type"
```

```

        << setw(10) << "ERef"
        << setw(8) << "Low"
        << setw(8) << "High"
        << setw(10) << "ElSize"
        << setw(10) << "Size" << endl;

    for (int i = 0; i < (int)st.atab.size(); i++) {
        const AtabEntry& entry = st.atab[i];
        cout << setw(6) << i
                                     << setw(15) <<
typeCodeToString((TypeCode)entry.xtyp)
                                     << setw(15) <<
typeCodeToString((TypeCode)entry.etyt)
        << setw(10) << entry.eref
        << setw(8) << entry.low
        << setw(8) << entry.high
        << setw(10) << entry.elsz
        << setw(10) << entry.size << endl;
    }

    // BTAB
    cout << "[btab]" << endl;
    cout << setw(6) << "Index"
        << setw(15) << "Last"
        << setw(15) << "Last Param"
        << setw(15) << "Param Size"
        << setw(15) << "Var Size" << endl;

    for (int i = 0; i < (int)st.btab.size(); i++) {
        const BtabEntry& entry = st.btab[i];
        cout << setw(6) << i
            << setw(15) << entry.last

```

```

        << setw(15) << entry.lpar
        << setw(15) << entry.psize
        << setw(15) << entry.vsize << endl;
    }
}

```

Fungsi `PRINT_SYMBOL_TABLE()` mencetak ketiga tabel simbol ke terminal dalam format tabular yang rapi menggunakan `std::setw()` dari `<iomanip>` untuk alignment kolom. Untuk tabel tab, dicetak header dengan kolom Index, Identifier, Object, Type, Ref, Nrm, Lev, dan Address, diikuti satu baris per entri mulai dari indeks 0. Nilai enum `ObjClass` dikonversi ke string yang mudah dibaca ("CONSTANT", "VARIABLE", "TYPE", dst.) via `objClassToString()`, dan nilai `TypeCode` dikonversi via `typeCodeToString()`. Identifier yang panjang dipotong maksimal 19 karakter menggunakan `substr(0, 19)` agar kolom tidak rusak. Untuk tabel atab, dicetak kolom Index, Index Type, Element Type, ERef, Low, High, ElSize, dan Size; nilai `xtyp` dan `etyp` dikonversi dari integer ke `TypeCode` via `cast` lalu ke string. Untuk tabel btab, dicetak kolom Index, Last, Last Param, Param Size, dan Var Size. Fungsi `SAVE_SYMBOL_TABLE()` bekerja identik namun output ditujukan ke `ofstream` yang disimpan di direktori `test/milestone-3/`. Format yang sama di terminal dan file memudahkan perbandingan antara output langsung dan output yang tersimpan.

```

void PRINT_AST_NEW() {
    if (!g_astRoot) {
        cout << "AST root kosong" << endl;
        return;
    }

    auto formatNode = [] (const std::shared_ptr<ASTNode>&
node) -> std::string {
        std::ostringstream oss;
        const std::string& k = node->kind;

```

```

    if (k == "Program") {
        oss << "ProgramNode(name: '" << node->text <<
"' )";
    } else if (k == "Declarations") {
        oss << "Declarations";
    } else if (k == "ConstDecl") {
        oss << "ConstDecl(name: '" << node->text <<
"' )";
    } else if (k == "TypeDecl") {
        oss << "TypeDecl(name: '" << node->text <<
"' )";
    } else if (k == "VarDecl") {
        oss << "VarDecl(name: '" << node->text <<
"' )";
    } else if (k == "ProcedureDecl") {
        oss << "ProcedureDecl(name: '" << node->text
<< "' )";
    } else if (k == "FunctionDecl") {
        oss << "FunctionDecl(name: '" << node->text
<< "' )";
    } else if (k == "Block") {
        oss << "Block";
    } else if (k == "Assign") {
        oss << "Assign('" << node->text << "' :=
...)" ;
    } else if (k == "BinOp") {
        oss << "BinOp '" << node->text << "'";
    } else if (k == "UnaryOp") {
        oss << "UnaryOp '" << node->text << "'";
    } else if (k == "Var") {
        oss << "Var('" << node->text << "' )";
    } else if (k == "Literal") {

```



```

        oss << "Literal(" << node->text << ")";
    } else if (k == "ProcCall") {
        oss << "procedure/function-call-statement";
        if (!node->text.empty()) {
            oss << " name: " << node->text;
        }
    } else {
        oss << k;
        if (!node->text.empty()) {
            oss << "(" << node->text << ")";
        }
    }
}

bool hasAny = false;
std::ostringstream ann;
if (node->block_index >= 0) {
    ann << "block_index:" << node->block_index;
    hasAny = true;
}

if (node->tab_index >= 0) {
    if (hasAny) ann << ", ";
    ann << "tab_index:" << node->tab_index;
    hasAny = true;
}

if (node->type >= 0) {
    if (hasAny) ann << ", ";
    ann << "type:" <<
typeCodeToString((TypeCode) node->type);
    hasAny = true;
}

if (node->lev >= 0) {
    if (hasAny) ann << ", ";

```

```

        ann << "lev:" << node->lev;
        hasAny = true;
    }

    if (hasAny) {
        oss << " -> " << ann.str();
    }

    return oss.str();
};

std::function<void(const std::shared_ptr<ASTNode>&,
int)> printRec;
printRec = [&](const std::shared_ptr<ASTNode>& node,
int depth) {
    if (!node) return;
    cout << string(depth * 4, ' ') <<
formatNode(node) << endl;
    for (const auto& child : node->children) {
        printRec(child, depth + 1);
    }
};

printRec(g_astRoot, 0);
}

void SAVE_AST_NEW() {
    if (!g_astRoot) {
        cout << "AST root kosong" << endl;
        return;
    }
}

```

```

    cout << "Masukkan nama file untuk menyimpan AST : ";
    string fileName;
    getline(cin, fileName);

    ofstream outputStream("test/milestone-3/" + fileName
+ ".txt");
    if (!outputStream.is_open()) {
        cerr << "Error: Tidak dapat membuka file untuk
penulisan" << endl;
        return;
    }

    auto formatNode = [] (const std::shared_ptr<ASTNode>&
node) -> std::string {
        std::ostringstream oss;
        const std::string& k = node->kind;

        if (k == "Program") {
            oss << "ProgramNode (name: '" << node->text <<
"' ) ";
        } else if (k == "Declarations") {
            oss << "Declarations";
        } else if (k == "ConstDecl") {
            oss << "ConstDecl (name: '" << node->text <<
"' ) ";
        } else if (k == "TypeDecl") {
            oss << "TypeDecl (name: '" << node->text <<
"' ) ";
        } else if (k == "VarDecl") {
            oss << "VarDecl (name: '" << node->text <<
"' ) ";
        } else if (k == "ProcedureDecl") {

```

```

        oss << "ProcedureDecl(name: '" << node->text
<< "'"");
    } else if (k == "FunctionDecl") {
        oss << "FunctionDecl(name: '" << node->text
<< "'"");
    } else if (k == "Block") {
        oss << "Block";
    } else if (k == "Assign") {
        oss << "Assign('" << node->text << "' :=
...)";
    } else if (k == "BinOp") {
        oss << "BinOp '" << node->text << "'";
    } else if (k == "UnaryOp") {
        oss << "UnaryOp '" << node->text << "'";
    } else if (k == "Var") {
        oss << "Var('" << node->text << "')";
    } else if (k == "Literal") {
        oss << "Literal(" << node->text << ")";
    } else if (k == "ProcCall") {
        oss << "procedure/function-call-statement";
        if (!node->text.empty()) {
            oss << " name: " << node->text;
        }
    } else {
        oss << k;
        if (!node->text.empty()) {
            oss << "(" << node->text << ")";
        }
    }
}

bool hasAny = false;
std::ostringstream ann;

```

```

        if (node->block_index >= 0) {
            ann << "block_index:" << node->block_index;
            hasAny = true;
        }
        if (node->tab_index >= 0) {
            if (hasAny) ann << ", ";
            ann << "tab_index:" << node->tab_index;
            hasAny = true;
        }
        if (node->type >= 0) {
            if (hasAny) ann << ", ";
            ann << "type:" <<
typeCodeToString((TypeCode) node->type);
            hasAny = true;
        }
        if (node->lev >= 0) {
            if (hasAny) ann << ", ";
            ann << "lev:" << node->lev;
            hasAny = true;
        }

        if (hasAny) {
            oss << " -> " << ann.str();
        }

        return oss.str();
    };

    std::function<void(const std::shared_ptr<ASTNode>&,
int, std::ofstream&)> saveRec;
    saveRec = [&](const std::shared_ptr<ASTNode>& node,
int depth, std::ofstream& os) {

```

```

        if (!node) return;
        os << string(depth * 4, ' ') << formatNode(node)
<< endl;
        for (const auto& child : node->children) {
            saveRec(child, depth + 1, os);
        }
    };

    saveRec(g_astRoot, 0, outputStream);
    outputStream.close();
}

```

Fungsi PRINT_AST_NEW() pertama memeriksa apakah g_astRoot valid; jika tidak, dicetak pesan "AST root kosong". Jika valid, lambda formatNode dipanggil untuk setiap node yang dikunjungi secara rekursif. Lambda ini menghasilkan string representasi berbeda berdasarkan kind node: untuk "Program" menghasilkan "ProgramNode(name: 'Hello')", untuk "VarDecl" menghasilkan "VarDecl(name: 'a')", untuk "Assign" menghasilkan "Assign('a' := ...)", untuk "BinOp" menghasilkan "BinOp '+'", untuk "Var" menghasilkan "Var('a')", untuk "Literal" menghasilkan "Literal(5)", untuk "ProcCall" menghasilkan "procedure/function-call-statement name: writeln", dan untuk jenis lain menghasilkan kind(text) sebagai fallback. Setelah format utama, anotasi semantik dicetak dalam format "-> block_index:X, tab_index:Y, type:Z, lev:W" menggunakan ostream yang diisi secara kondisional: setiap field hanya ditambahkan jika nilainya >= 0 (artinya sudah terisi). Fungsi rekursif printRec kemudian mencetak setiap node dengan indentasi empat spasi per level kedalaman. Fungsi SAVE_AST_NEW() bekerja identik namun menggunakan ofstream untuk menulis ke file di direktori test/milestone-3/.

```

#include "lexer/lexer.hpp"
#include "io/reader.hpp"
#include "io/writer.hpp"
#include "parser/parser.hpp"
#include "parser/utils/error.hpp"
#include "semantic/semantic_analyzer.hpp"

```

```

int main () {

    /* ===== LEXER ===== */
    INIT_DICTIONARY();
    INPUT_FILE();
    READ_ALL_FILE();
    PRINT_TOKEN_LIST();
    SAVE_TOKEN_LIST();

    /* ===== PARSER ===== */
    Node* root = nullptr;
    try{
        Parser parser(token_list);
        root = parser.parse();

        PRINT_PARSE_TREE (root, 0);

        SAVE_PARSE_TREE (root, 0);
    } catch (const SyntaxError& e) {
        std::cerr << e.what() << std::endl;
        return 0;
    }

    /* ===== SEMANTIC ANALYSIS ===== */
    if (root != nullptr) {
        SemanticContext ctx;
        analyze(root, ctx);

        // Kalo ada error ga bisa hasilin symbol table
        sama ast

        if (ctx.errors.hasErrors()) {

```

```

        std::cout << "Error" << std::endl;
        ctx.errors.printAll();
        // PRINT_SYMBOL_TABLE(ctx.st);
    } else {
        std::cout << "Success" << std::endl;
        PRINT_SYMBOL_TABLE(ctx.st);
        SAVE_SYMBOL_TABLE(ctx.st);
        // Print dan save AST (format baru:
indentation-based, tanpa box-drawing)
        // PRINT_AST(root, 0, ctx.st);
        // SAVE_AST(root, 0, ctx.st);
        PRINT_AST_NEW();
        SAVE_AST_NEW();
    }

    delete root;
}

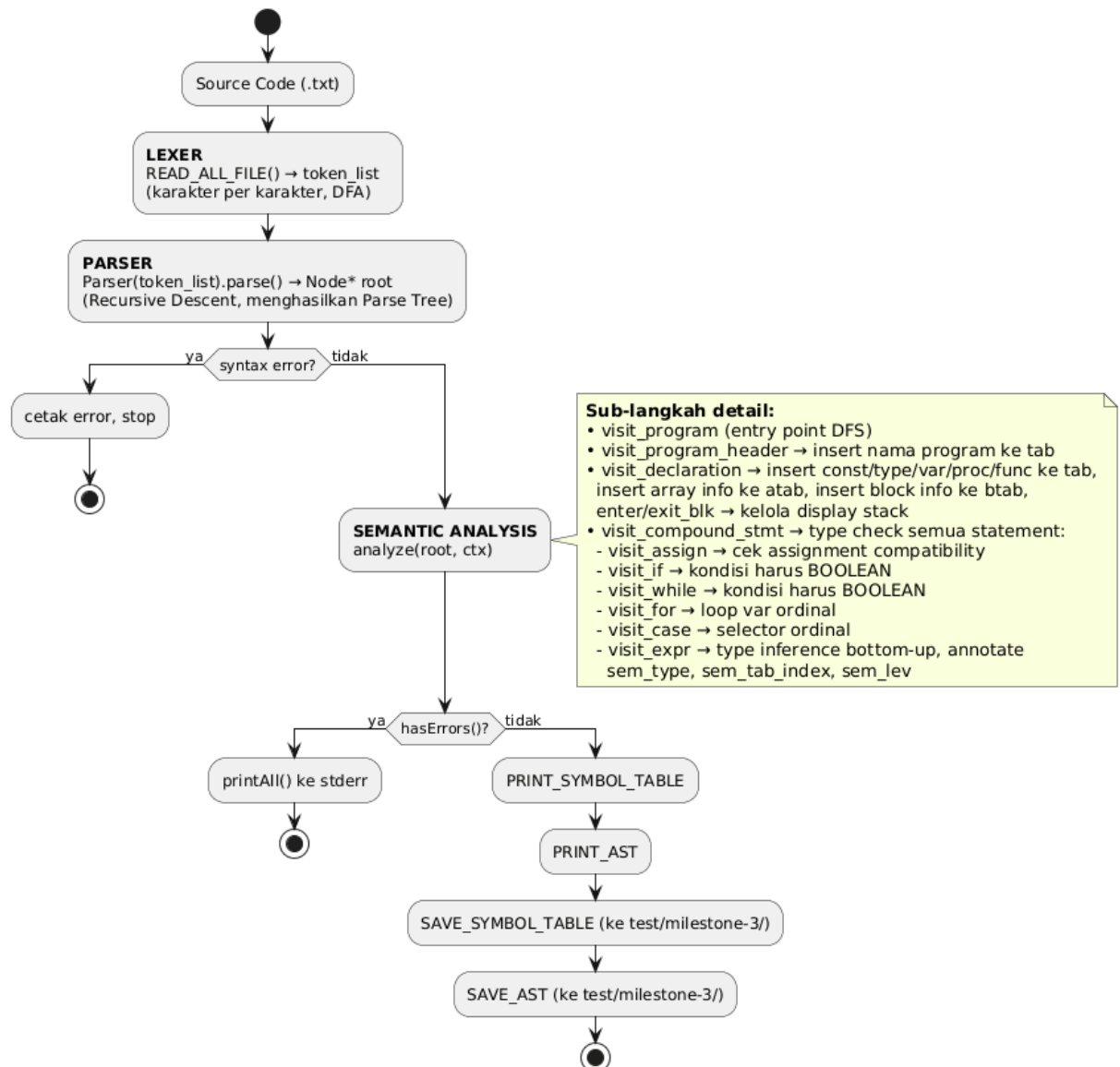
return 0;
}

```

File main.cpp mengorkestrasi ketiga tahap kompilasi dengan alur yang linear dan jelas. Tahap pertama adalah lexer: INIT_DICTIONARY() menginisialisasi dictionary keyword, INPUT_FILE() membuka file input dari direktori test/milestone-3/, READ_ALL_FILE() membaca dan mentokenisasi seluruh isi file mengisi token_list, kemudian PRINT_TOKEN_LIST() dan SAVE_TOKEN_LIST() mencetak dan menyimpan hasil tokenisasi. Tahap kedua adalah parser: objek Parser dibuat dengan token_list, parse() dipanggil untuk membangun parse tree, dan hasilnya dicetak serta disimpan. Seluruh tahap parser dibungkus dalam blok try-catch yang menangkap SyntaxError: jika terjadi syntax error, pesan error dicetak ke stderr dan program langsung return tanpa memasuki tahap semantic analysis, karena parse tree yang tidak valid tidak bisa diproses secara semantik dengan benar. Tahap ketiga adalah semantic analysis: objek SemanticContext ctx dibuat (konstruktornya otomatis menginisialisasi

symbol table dengan reserved words dan predefined identifier), kemudian `analyze(root, ctx)` dipanggil yang memulai seluruh traversal DFS. Setelah `analyze()` selesai, `ctx.errors.hasErrors()` diperiksa. Jika ada error, "Error" dicetak diikuti semua pesan error via `ctx.errors.printAll()`. Jika tidak ada error, "Success" dicetak diikuti symbol table via `PRINT_SYMBOL_TABLE()` dan `SAVE_SYMBOL_TABLE()`, serta Decorated AST via `PRINT_AST_NEW()` dan `SAVE_AST_NEW()`. Terakhir, delete root dipanggil untuk membebaskan seluruh memori parse tree secara rekursif via destructor Node.

2.12 Alur Kerja Program



Alur eksekusi program secara keseluruhan dikendalikan dari file main.cpp dan terdiri atas tiga tahap utama yang berjalan secara berurutan.

Tahap pertama adalah Lexer, di mana seluruh source code dibaca karakter per karakter dan ditokenisasi, hasilnya disimpan ke dalam token_list yang berisi string-string token dengan format "tipe (nilai)" atau "tipe" saja untuk token tanpa nilai. Tahap kedua adalah Parser, di mana Recursive Descent Parsing dijalankan terhadap token_list untuk menghasilkan parse tree yang direpresentasikan oleh Node* root. Jika terjadi syntax error, exception SyntaxError dilempar, ditangkap di main.cpp, dan program berhenti di sini. Tahap ketiga adalah Semantic Analysis, di

mana objek `SemanticContext` diinisialisasi, konstruktor `SymbolTable` di dalamnya otomatis mengisi reserved words dan predefined identifiers, lalu fungsi `analyze(root, ctx)` dipanggil yang selanjutnya meneruskan eksekusi ke `visit_program`. Selama traversal DFS berlangsung, setiap node diproses dengan pola berikut: child-child node yang relevan dikunjungi sesuai urutan semantik (synthesized attributes dihitung dari bawah ke atas, inherited attributes diteruskan dari atas ke bawah), identifier di-lookup atau di-insert ke symbol table, tipe data diinferensikan dan divalidasi, lalu node dianotasi dengan informasi semantik yang diperoleh. Error yang ditemukan tidak menghentikan traversal melainkan dikumpulkan di `SemanticErrorCollector`.

Setelah traversal selesai, apabila terdapat error, seluruh pesan error dicetak sekaligus ke `stderr`. Sementara itu, apabila analisis berhasil tanpa error, symbol table yang mencakup `tab`, `btabs`, dan `atab` beserta Decorated AST dicetak ke terminal dan disimpan ke file output di direktori `test/milestone-3/`.

2.13 Justifikasi Pilihan Implementasi

Pendekatan dekorasi *in place* dipilih dengan pertimbangan bahwa menambahkan atribut semantik langsung ke kelas `Node` yang sudah ada jauh lebih efisien dibandingkan membangun struktur pohon AST baru yang terpisah. Pendekatan ini menghemat penggunaan memori dan mengurangi kompleksitas kode karena tidak perlu menjaga dua representasi pohon secara paralel selama proses analisis berlangsung. Selain itu, dengan satu pohon tunggal, seluruh informasi baik sintaksis maupun semantik dapat diakses dari node yang sama tanpa perlu mekanisme pemetaan antar-pohon.

Mekanisme pengumpulan error secara akumulatif dipilih agar seluruh kesalahan semantik dapat dilaporkan sekaligus di akhir proses, bukan berhenti pada error pertama yang ditemukan. Pendekatan ini memberikan feedback yang jauh lebih informatif kepada pengguna karena semua kesalahan dapat dilihat dan diperbaiki dalam satu siklus kompilasi, tidak perlu menjalankan ulang program berkali-kali hanya untuk menemukan error berikutnya satu per satu.

Pemisahan informasi ke dalam tiga tabel simbol yang berbeda, yaitu `tab`, `btabs`, dan `atab`, mengikuti desain compiler klasik yang telah terbukti efektif. Pemisahan ini menjaga setiap tabel tetap ramping dengan informasi yang homogen sehingga memudahkan pencarian dan

pengelolaan data lintas jenis struktur tanpa overhead yang tidak perlu. Setiap tabel memiliki peran yang terdefinisi dengan jelas: tab untuk semua identifier, btab untuk metadata blok scope, dan atab untuk metadata tipe komposit (array dan subrange).

Penggunaan array display untuk manajemen scope dipilih karena representasi ini secara langsung mencerminkan konsep lexical scoping. Dengan menyimpan indeks btab aktif di tiap lexical level, pencarian identifier dari scope terdalam ke terluar dapat dilakukan secara efisien hanya dengan mengikuti rantai link pada entri tab tanpa perlu traversal ulang pada seluruh tabel. Hal ini jauh lebih efisien dibanding pendekatan alternatif seperti menghapus identifier saat keluar scope (yang menghilangkan informasi yang dibutuhkan code generation) atau menyimpan semua identifier dalam flat list dan melakukan linear scan setiap lookup.

TypeInfo dirancang sebagai value struct yang dioper *by value* antar fungsi visit untuk menyederhanakan manajemen lifetime objek. Keputusan ini menghindari masalah pointer dangling yang rentan terjadi apabila TypeInfo diimplementasikan sebagai pointer ke objek di heap, mengingat fungsi visit bersifat rekursif dan sering menciptakan TypeInfo temporer dalam jumlah banyak. *Trade off* berupa overhead copy sangat minimal mengingat ukuran struct yang kecil (kurang dari 64 byte).

Selain itu, visit_program tidak memanggil enter_block karena deklarasi global harus masuk ke btab[0] dengan lev = 0, level 0 sebagai global. Memanggil enter_block di visit_program akan menyebabkan seluruh variabel global salah ter-insert dengan lev = 1, yang akan merusak mekanisme lookup dari scope nested dan menghasilkan anotasi lexical level yang salah pada Decorated AST.

Lookup case-insensitive diimplementasikan dengan mengkonversi identifier ke lowercase sebelum perbandingan menggunakan std::transform yang menyatakan bahwa keyword dan identifier bersifat case-insensitive di bahasa Arion. Pendekatan ini memastikan bahwa integer, Integer, dan INTEGER semuanya merujuk ke entri yang sama di symbol table.

Predefined procedures writeln, readln, write, read tidak divalidasi jumlah dan tipe argumennya secara ketat karena keempat prosedur ini bersifat variadic (dapat menerima jumlah dan tipe argumen yang beragam). Validasi tetap dilakukan dalam arti semua argumen dikunjungi

dan dianotasi, namun pengecekan kompatibilitas tipe dilewati agar penggunaan prosedur I/O yang fleksibel tidak menghasilkan false positive error.

Pengujian

3.1 Test Case 1

Pengujian kasus sederhana secara keseluruhan sesuai pada file input sesuai contoh pada dokumen spesifikasi.

input-1.txt							
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>							
output-1-symtab.txt							
[tab]							
Index	Identifier	Object	Type	Ref	Nrm	Lev	Address
0	and	TYPE	NOTYPE	0	1	0	0
1	array	TYPE	NOTYPE	0	1	0	0
2	begin	TYPE	NOTYPE	0	1	0	0
3	case	TYPE	NOTYPE	0	1	0	0
4	const	TYPE	NOTYPE	0	1	0	0
5	div	TYPE	NOTYPE	0	1	0	0
6	downto	TYPE	NOTYPE	0	1	0	0
7	do	TYPE	NOTYPE	0	1	0	0
8	else	TYPE	NOTYPE	0	1	0	0
9	end	TYPE	NOTYPE	0	1	0	0
10	for	TYPE	NOTYPE	0	1	0	0
11	function	TYPE	NOTYPE	0	1	0	0
12	if	TYPE	NOTYPE	0	1	0	0
13	mod	TYPE	NOTYPE	0	1	0	0
14	not	TYPE	NOTYPE	0	1	0	0
15	of	TYPE	NOTYPE	0	1	0	0
16	or	TYPE	NOTYPE	0	1	0	0
17	procedure	TYPE	NOTYPE	0	1	0	0
18	program	TYPE	NOTYPE	0	1	0	0
19	record	TYPE	NOTYPE	0	1	0	0
20	repeat	TYPE	NOTYPE	0	1	0	0
21	integer	TYPE	NOTYPE	0	1	0	0
22	real	TYPE	NOTYPE	0	1	0	0
23	boolean	TYPE	NOTYPE	0	1	0	0
24	char	TYPE	NOTYPE	0	1	0	0
25	string	TYPE	NOTYPE	0	1	0	0
26	then	TYPE	NOTYPE	0	1	0	0
27	to	TYPE	NOTYPE	0	1	0	0
28	type	TYPE	NOTYPE	0	1	0	0
29	until	TYPE	NOTYPE	0	1	0	0
30	var	TYPE	NOTYPE	0	1	0	0
31	while	TYPE	NOTYPE	0	1	0	0

32	Integer	TYPE	INTEGER	0	1	0	0
33	Real	TYPE	REAL	0	1	0	0
34	Char	TYPE	CHAR	0	1	0	0
35	Boolean	TYPE	BOOLEAN	0	1	0	0
36	String	TYPE	STRING	0	1	0	0
37	True	CONSTANT	BOOLEAN	0	1	0	1
38	False	CONSTANT	BOOLEAN	0	1	0	0
39	writeln	PROCEDURE	NOTYPE	0	1	0	0
40	readln	PROCEDURE	NOTYPE	0	1	0	0
41	write	PROCEDURE	NOTYPE	0	1	0	0
42	read	PROCEDURE	NOTYPE	0	1	0	0
43	Hello	PROGRAM	NOTYPE	0	1	0	0
44	a	VARIABLE	INTEGER	0	1	0	0
45	b	VARIABLE	INTEGER	0	1	0	0
[atab]							
Index	Index	Type	Element Type	ERef	Low	High	ElSize
0		NOTYPE	NOTYPE	0	0	0	1
[btab]							
Index	Last	Last Param	Param Size		Var Size		
0	45	0	0		2		

output-1-AST.txt

```

ProgramNode(name: 'Hello') -> tab_index:43, type:NOTYPE, lev:0
  Declarations
    VarDecl(name: 'a') -> tab_index:44, type:INTEGER, lev:0
    VarDecl(name: 'b') -> tab_index:45, type:INTEGER, lev:0
  Block -> block_index:0, lev:0
    Assign('a' := ...)
      Var('a') -> tab_index:44, type:INTEGER, lev:0
      Literal(5) -> type:INTEGER, lev:0
    Assign('b' := ...)
      Var('b') -> tab_index:45, type:INTEGER, lev:0
      BinOp '+' -> tab_index:0, type:INTEGER, lev:0
        Var('a') -> tab_index:44, type:INTEGER, lev:0
        Literal(10) -> type:INTEGER, lev:0
      procedure/function-call-statement name: writeln -> tab_index:39,
type:NOTYPE, lev:0
    Args
      Literal('Result = ') -> type:STRING, lev:0
      Var('b') -> tab_index:45, type:INTEGER, lev:0

```

3.2 Test Case 2

Pengujian kasus error karena *assignment* yang tidak sesuai dan akses variabel yang tidak dideklarasikan

input-2.txt

```

program wrongType;

var
  a, b: integer;

```

```

begin
  a := 'hi';
  halodunia := 'halo';
  b := a + 10;
  writeln('Result = ', b);
end.

```

output terminal

```

Semantic error: Tipe assignment tidak kompatibel: lhs adalah
integer, rhs adalah string
Semantic error: Variabel tidak ditemukan: 'halodunia'
Semantic error: Tipe assignment tidak kompatibel: lhs adalah
notype, rhs adalah string

```

3.3 Test Case 3

Pengujian untuk kasus sukses yang meng-*cover* deklarasi tipe baru, penggunaan array, dan penggunaan subprogram

input-3.txt

```

program TypeArraySubprogram;

type
  TPoint == record
    x: integer;
    y: integer
  end;
  TNumbers == array[1..100] of integer;
  TMatrix == array[1..10] of array[1..10] of real;

var
  points: array[1..5] of TPoint;
  numbers: TNumbers;
  matrix: TMatrix;
  name: string;
  count: integer;

procedure PrintPoint(p: TPoint);
begin
  writeln('Point:');
end;

```



```

procedure InitializeArray(arr: TNumbers; size: integer);
var
    i: integer;
begin
    for i := 1 to size do
        begin
            arr[i] := i * 10;
        end;
    end;

function CalculateSum(arr: TNumbers; size: integer): integer;
var
    i: integer;
    total: integer;
begin
    total := 0;
    for i := 1 to size do
        begin
            total := total + arr[i];
        end;
    CalculateSum := total;
end;

procedure FillMatrix(m: TMatrix; rows: integer; cols: integer);
var
    i: integer;
    j: integer;
begin
    for i := 1 to rows do
        begin
            for j := 1 to cols do
                begin
                    m[i][j] := i + j;
                end;
            end;
        end;
    end;

begin
    count := 5;

    { Initialize points }
    points[1].x := 1;
    points[1].y := 2;
    points[2].x := 3;
    points[2].y := 4;

```

```

{ PrintPoint(points[1]); }

{ Initialize and process numbers array }
InitializeArray(numbers, 10);
writeln('Sum = ', CalculateSum(numbers, 10));

{ Fill matrix }
FillMatrix(matrix, 5, 5);

{ Test string }
name := 'TestProgram';
writeln('Program name: ', name);
end.

```

output-3-symtab.txt

[tab]							
Index	Identifier	Object	Type	Ref	Nrm	Lev	Address
0	and	TYPE	NOTYPE	0	1	0	0
1	array	TYPE	NOTYPE	0	1	0	0
2	begin	TYPE	NOTYPE	0	1	0	0
3	case	TYPE	NOTYPE	0	1	0	0
4	const	TYPE	NOTYPE	0	1	0	0
5	div	TYPE	NOTYPE	0	1	0	0
6	downto	TYPE	NOTYPE	0	1	0	0
7	do	TYPE	NOTYPE	0	1	0	0
8	else	TYPE	NOTYPE	0	1	0	0
9	end	TYPE	NOTYPE	0	1	0	0
10	for	TYPE	NOTYPE	0	1	0	0
11	function	TYPE	NOTYPE	0	1	0	0
12	if	TYPE	NOTYPE	0	1	0	0
13	mod	TYPE	NOTYPE	0	1	0	0
14	not	TYPE	NOTYPE	0	1	0	0
15	of	TYPE	NOTYPE	0	1	0	0
16	or	TYPE	NOTYPE	0	1	0	0
17	procedure	TYPE	NOTYPE	0	1	0	0
18	program	TYPE	NOTYPE	0	1	0	0
19	record	TYPE	NOTYPE	0	1	0	0
20	repeat	TYPE	NOTYPE	0	1	0	0
21	integer	TYPE	NOTYPE	0	1	0	0
22	real	TYPE	NOTYPE	0	1	0	0
23	boolean	TYPE	NOTYPE	0	1	0	0
24	char	TYPE	NOTYPE	0	1	0	0
25	string	TYPE	NOTYPE	0	1	0	0
26	then	TYPE	NOTYPE	0	1	0	0
27	to	TYPE	NOTYPE	0	1	0	0
28	type	TYPE	NOTYPE	0	1	0	0
29	until	TYPE	NOTYPE	0	1	0	0
30	var	TYPE	NOTYPE	0	1	0	0
31	while	TYPE	NOTYPE	0	1	0	0
32	Integer	TYPE	INTEGER	0	1	0	0
33	Real	TYPE	REAL	0	1	0	0
34	Char	TYPE	CHAR	0	1	0	0
35	Boolean	TYPE	BOOLEAN	0	1	0	0
36	String	TYPE	STRING	0	1	0	0
37	True	CONSTANT	BOOLEAN	0	1	0	1
38	False	CONSTANT	BOOLEAN	0	1	0	0
39	writeln	PROCEDURE	NOTYPE	0	1	0	0
40	readln	PROCEDURE	NOTYPE	0	1	0	0
41	write	PROCEDURE	NOTYPE	0	1	0	0
42	read	PROCEDURE	NOTYPE	0	1	0	0

43	TypeArraySubprogram	PROGRAM	NOTYPE	0	1	0	0	
44	x	FIELD	INTEGER	0	1	1	0	
45	y	FIELD	INTEGER	0	1	1	0	
46	TPoint	TYPE	RECORD	1	1	0	0	
47	TNumbers	TYPE	ARRAY	2	1	0	0	
48	TMatrix	TYPE	ARRAY	6	1	0	0	
49	points	VARIABLE	ARRAY	8	1	0	0	
50	numbers	VARIABLE	ARRAY	2	1	0	0	
51	matrix	VARIABLE	ARRAY	6	1	0	0	
52	name	VARIABLE	STRING	0	1	0	0	
53	count	VARIABLE	INTEGER	0	1	0	0	
54	PrintPoint	PROCEDURE	NOTYPE	2	1	0	0	
55	p	VARIABLE	RECORD	1	1	1	0	
56	InitializeArray	PROCEDURE	NOTYPE	3	1	0	0	
57	arr	VARIABLE	ARRAY	2	1	1	0	
58	size	VARIABLE	INTEGER	0	1	1	1	
59	i	VARIABLE	INTEGER	0	1	1	0	
60	CalculateSum	FUNCTION	INTEGER	4	1	0	0	
61	arr	VARIABLE	ARRAY	2	1	1	0	
62	size	VARIABLE	INTEGER	0	1	1	1	
63	i	VARIABLE	INTEGER	0	1	1	0	
64	total	VARIABLE	INTEGER	0	1	1	0	
65	FillMatrix	PROCEDURE	NOTYPE	5	1	0	0	
66	m	VARIABLE	ARRAY	6	1	1	0	
67	rows	VARIABLE	INTEGER	0	1	1	1	
68	cols	VARIABLE	INTEGER	0	1	1	2	
69	i	VARIABLE	INTEGER	0	1	1	0	
70	j	VARIABLE	INTEGER	0	1	1	0	
[atab]								
Index	Index	Type	Element Type	ERef	Low	High	ElSize	Size
0		NOTYPE	NOTYPE	0	0	0	1	0
1		INTEGER	INTEGER	0	1	100	1	100
2		SUBRANGE	INTEGER	0	1	100	1	100
3		INTEGER	INTEGER	0	1	10	1	10
4		INTEGER	INTEGER	0	1	10	1	10
5		SUBRANGE	REAL	0	1	10	1	10
6		SUBRANGE	ARRAY	5	1	10	1	10
7		INTEGER	INTEGER	0	1	5	1	5
8		SUBRANGE	RECORD	1	1	5	1	5
[btab]								
Index	Last	Last Param	Param Size	Var Size				
0	65	0	0	5				
1	45	0	0	2				
2	55	55	1	0				
3	59	58	2	2				
4	64	62	2	3				
5	70	68	3	4				

output-3-AST.txt

ProgramNode(name: 'TypeArraySubprogram') -> tab_index:43, type:NOTYPE, lev:0

Declarations

TypeDecl(name: 'TPoint') -> tab_index:46, type:RECORD, lev:0

TypeDecl(name: 'TNumbers') -> tab_index:47, type:ARRAY, lev:0

TypeDecl(name: 'TMatrix') -> tab_index:48, type:ARRAY, lev:0

VarDecl(name: 'points') -> tab_index:49, type:ARRAY, lev:0

VarDecl(name: 'numbers') -> tab_index:50, type:ARRAY, lev:0

VarDecl(name: 'matrix') -> tab_index:51, type:ARRAY, lev:0

VarDecl(name: 'name') -> tab_index:52, type:STRING, lev:0

VarDecl(name: 'count') -> tab_index:53, type:INTEGER, lev:0

ProcedureDecl(name: 'PrintPoint') -> tab_index:54, type:NOTYPE, lev:0

Block -> block_index:2, lev:1

procedure/function-call-statement name: writeln -> tab_index:39,

type:NOTYPE, lev:1

Args

Literal('Point:') -> type:STRING, lev:1

ProcedureDecl(name: 'InitializeArray') -> tab_index:56, type:NOTYPE, lev:0

Block -> block_index:3, lev:1

For

```

Assign('i' := ...)
  Var('i')
  Literal(1) -> type:INTEGER, lev:1
Limit(to)
  Var('size') -> tab_index:58, type:INTEGER, lev:1
Block -> block_index:3, lev:1
  Assign('arr' := ...)
    Var('arr') -> tab_index:57, type:ARRAY, lev:1
    BinOp '*' -> tab_index:0, type:INTEGER, lev:1
    Var('i') -> tab_index:59, type:INTEGER, lev:1
    Literal(10) -> type:INTEGER, lev:1
VarDecl(name: 'i') -> tab_index:59, type:INTEGER, lev:1
FunctionDecl(name: 'CalculateSum') -> tab_index:60, type:INTEGER, lev:0
  Block -> block_index:4, lev:1
    Assign('total' := ...)
      Var('total') -> tab_index:64, type:INTEGER, lev:1
      Literal(0) -> type:INTEGER, lev:1
    For
      Assign('i' := ...)
        Var('i')
        Literal(1) -> type:INTEGER, lev:1
      Limit(to)
        Var('size') -> tab_index:62, type:INTEGER, lev:1
      Block -> block_index:4, lev:1
        Assign('total' := ...)
          Var('total') -> tab_index:64, type:INTEGER, lev:1
          BinOp '+' -> tab_index:0, type:INTEGER, lev:1
          Var('total') -> tab_index:64, type:INTEGER, lev:1
          Var('arr') -> tab_index:61, type:ARRAY, lev:1
        Assign('CalculateSum' := ...)
          Var('CalculateSum') -> tab_index:60, type:INTEGER, lev:1
          Var('total') -> tab_index:64, type:INTEGER, lev:1
VarDecl(name: 'i') -> tab_index:63, type:INTEGER, lev:1
VarDecl(name: 'total') -> tab_index:64, type:INTEGER, lev:1
ProcedureDecl(name: 'FillMatrix') -> tab_index:65, type:NOTYPE, lev:0
  Block -> block_index:5, lev:1
    For
      Assign('i' := ...)
        Var('i')
        Literal(1) -> type:INTEGER, lev:1
      Limit(to)
        Var('rows') -> tab_index:67, type:INTEGER, lev:1
      Block -> block_index:5, lev:1
        For
          Assign('j' := ...)
            Var('j')
            Literal(1) -> type:INTEGER, lev:1
          Limit(to)
            Var('cols') -> tab_index:68, type:INTEGER, lev:1
          Block -> block_index:5, lev:1
            Assign('m' := ...)
              Var('m') -> tab_index:66, type:ARRAY, lev:1
              BinOp '+' -> tab_index:0, type:INTEGER, lev:1
              Var('i') -> tab_index:69, type:INTEGER, lev:1
              Var('j') -> tab_index:70, type:INTEGER, lev:1
VarDecl(name: 'i') -> tab_index:69, type:INTEGER, lev:1
VarDecl(name: 'j') -> tab_index:70, type:INTEGER, lev:1
Block -> block_index:0, lev:0
  Assign('count' := ...)
    Var('count') -> tab_index:53, type:INTEGER, lev:0
    Literal(5) -> type:INTEGER, lev:0
  Assign('points' := ...)
    Var('points') -> tab_index:49, type:ARRAY, lev:0
    Field(x) -> tab_index:44, type:INTEGER, lev:0
    Literal(1) -> type:INTEGER, lev:0
  Assign('points' := ...)
    Var('points') -> tab_index:49, type:ARRAY, lev:0
    Field(y) -> tab_index:45, type:INTEGER, lev:0
    Literal(2) -> type:INTEGER, lev:0
  Assign('points' := ...)

```

```

    Var('points') -> tab_index:49, type:ARRAY, lev:0
    Field(x) -> tab_index:44, type:INTEGER, lev:0
    Literal(3) -> type:INTEGER, lev:0
Assign('points' := ...)
    Var('points') -> tab_index:49, type:ARRAY, lev:0
    Field(y) -> tab_index:45, type:INTEGER, lev:0
    Literal(4) -> type:INTEGER, lev:0
    procedure/function-call-statement name: InitializeArray -> tab_index:56,
type:NOTYPE, lev:0
    Args
        Var('numbers') -> tab_index:50, type:ARRAY, lev:0
        Literal(10) -> type:INTEGER, lev:0
    procedure/function-call-statement name: writeln -> tab_index:39, type:NOTYPE, lev:0
    Args
        Literal('Sum = ') -> type:STRING, lev:0
        Var('CalculateSum') -> tab_index:60, type:INTEGER, lev:0
    procedure/function-call-statement name: FillMatrix -> tab_index:65, type:NOTYPE,
lev:0
    Args
        Var('matrix') -> tab_index:51, type:ARRAY, lev:0
        Literal(5) -> type:INTEGER, lev:0
        Literal(5) -> type:INTEGER, lev:0
Assign('name' := ...)
    Var('name') -> tab_index:52, type:STRING, lev:0
    Literal('TestProgram') -> type:STRING, lev:0
    procedure/function-call-statement name: writeln -> tab_index:39, type:NOTYPE, lev:0
    Args
        Literal('Program name: ') -> type:STRING, lev:0
        Var('name') -> tab_index:52, type:STRING, lev:0

```

3.4 Test Case 4

Pengujian pada kasus komentar, percabangan, dan perulangan

input-4.txt

```

program CommentTest;

var
  x: integer;
  y: integer;
  i: integer;

begin
  { ini komentar biasa }
  x := 5;
  y := 90;

  (* ini komentar
    multi-line *)

  x := x + {ini ada komen di tengah2 sini tapi tetep aman} 10;

```

```

{ komentar dengan simbol aneh + - * / := }

if x == y then
    x := x + y;
if x <> y then
    x := x - y;
if x > y then
    x := x * y;
if x >= y then
    x := x div y;
if x <= y then
    x := x MOD y;

for i := 1 to 10 do
    begin
        i := i + 1;
    end;

for i := 10 downto 1 do
    begin
        i := i - 1;
    end;

while i > 0 do
    begin
        i := i - 1;
    end;

repeat
    i := i + 1;
until i == 10;

case i of
    1: i := 100;
    2: i := 200;
end;

end.

```

output-4-symtab.txt

[tab]							
Index	Identifier	Object	Type	Ref	Nrm	Lev	Address
0	and	TYPE	NOTYPE	0	1	0	0
1	array	TYPE	NOTYPE	0	1	0	0
2	begin	TYPE	NOTYPE	0	1	0	0
3	case	TYPE	NOTYPE	0	1	0	0
4	const	TYPE	NOTYPE	0	1	0	0

5	div	TYPE	NOTYPE	0	1	0	0		
6	downto	TYPE	NOTYPE	0	1	0	0		
7	do	TYPE	NOTYPE	0	1	0	0		
8	else	TYPE	NOTYPE	0	1	0	0		
9	end	TYPE	NOTYPE	0	1	0	0		
10	for	TYPE	NOTYPE	0	1	0	0		
11	function	TYPE	NOTYPE	0	1	0	0		
12	if	TYPE	NOTYPE	0	1	0	0		
13	mod	TYPE	NOTYPE	0	1	0	0		
14	not	TYPE	NOTYPE	0	1	0	0		
15	of	TYPE	NOTYPE	0	1	0	0		
16	or	TYPE	NOTYPE	0	1	0	0		
17	procedure	TYPE	NOTYPE	0	1	0	0		
18	program	TYPE	NOTYPE	0	1	0	0		
19	record	TYPE	NOTYPE	0	1	0	0		
20	repeat	TYPE	NOTYPE	0	1	0	0		
21	integer	TYPE	NOTYPE	0	1	0	0		
22	real	TYPE	NOTYPE	0	1	0	0		
23	boolean	TYPE	NOTYPE	0	1	0	0		
24	char	TYPE	NOTYPE	0	1	0	0		
25	string	TYPE	NOTYPE	0	1	0	0		
26	then	TYPE	NOTYPE	0	1	0	0		
27	to	TYPE	NOTYPE	0	1	0	0		
28	type	TYPE	NOTYPE	0	1	0	0		
29	until	TYPE	NOTYPE	0	1	0	0		
30	var	TYPE	NOTYPE	0	1	0	0		
31	while	TYPE	NOTYPE	0	1	0	0		
32	Integer	TYPE	INTEGER	0	1	0	0		
33	Real	TYPE	REAL	0	1	0	0		
34	Char	TYPE	CHAR	0	1	0	0		
35	Boolean	TYPE	BOOLEAN	0	1	0	0		
36	String	TYPE	STRING	0	1	0	0		
37	True	CONSTANT	BOOLEAN	0	1	0	1		
38	False	CONSTANT	BOOLEAN	0	1	0	0		
39	writeln	PROCEDURE	NOTYPE	0	1	0	0		
40	readln	PROCEDURE	NOTYPE	0	1	0	0		
41	write	PROCEDURE	NOTYPE	0	1	0	0		
42	read	PROCEDURE	NOTYPE	0	1	0	0		
43	CommentTest	PROGRAM	NOTYPE	0	1	0	0		
44	x	VARIABLE	INTEGER	0	1	0	0		
45	y	VARIABLE	INTEGER	0	1	0	0		
46	i	VARIABLE	INTEGER	0	1	0	0		
[atab]									
Index	Index	Type	Element	Type	ERef	Low	High	ElSize	Size
0		NOTYPE		NOTYPE	0	0	0	1	0
[btabs]									
Index	Last	Last	Param	Param	Size	Var	Size		
0	46		0		0		3		

output-4-AST.txt

```

ProgramNode(name: 'CommentTest') -> tab_index:43, type:NOTYPE, lev:0
  Declarations
    VarDecl(name: 'x') -> tab_index:44, type:INTEGER, lev:0
    VarDecl(name: 'y') -> tab_index:45, type:INTEGER, lev:0
    VarDecl(name: 'i') -> tab_index:46, type:INTEGER, lev:0
  Block -> block_index:0, lev:0
    Assign('x' := ...)
      Var('x') -> tab_index:44, type:INTEGER, lev:0
      Literal(5) -> type:INTEGER, lev:0
    Assign('y' := ...)
      Var('y') -> tab_index:45, type:INTEGER, lev:0
      Literal(90) -> type:INTEGER, lev:0
    Assign('x' := ...)
      Var('x') -> tab_index:44, type:INTEGER, lev:0

```

```

    BinOp '+' -> tab_index:0, type:INTEGER, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Literal(10) -> type:INTEGER, lev:0
If
  Condition
    BinOp '=' -> type:BOOLEAN, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Var('y') -> tab_index:45, type:INTEGER, lev:0
  Then
    Assign('x' := ...)
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    BinOp '+' -> tab_index:0, type:INTEGER, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Var('y') -> tab_index:45, type:INTEGER, lev:0
If
  Condition
    BinOp '<>' -> type:BOOLEAN, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Var('y') -> tab_index:45, type:INTEGER, lev:0
  Then
    Assign('x' := ...)
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    BinOp '-' -> tab_index:0, type:INTEGER, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Var('y') -> tab_index:45, type:INTEGER, lev:0
If
  Condition
    BinOp '>' -> type:BOOLEAN, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Var('y') -> tab_index:45, type:INTEGER, lev:0
  Then
    Assign('x' := ...)
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    BinOp '*' -> tab_index:0, type:INTEGER, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Var('y') -> tab_index:45, type:INTEGER, lev:0
If
  Condition
    BinOp '>=' -> type:BOOLEAN, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Var('y') -> tab_index:45, type:INTEGER, lev:0
  Then
    Assign('x' := ...)
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    BinOp 'div' -> tab_index:0, type:INTEGER, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Var('y') -> tab_index:45, type:INTEGER, lev:0
If
  Condition
    BinOp '<=' -> type:BOOLEAN, lev:0
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    Var('y') -> tab_index:45, type:INTEGER, lev:0
  Then
    Assign('x' := ...)
    Var('x') -> tab_index:44, type:INTEGER, lev:0
    BinOp 'mod' -> tab_index:0, type:INTEGER, lev:0

```



```

        Var('x') -> tab_index:44, type:INTEGER, lev:0
        Var('y') -> tab_index:45, type:INTEGER, lev:0
For
    Assign('i' := ...)
        Var('i')
        Literal(1) -> type:INTEGER, lev:0
    Limit(to)
        Literal(10) -> type:INTEGER, lev:0
    Block -> block_index:0, lev:0
        Assign('i' := ...)
            Var('i') -> tab_index:46, type:INTEGER, lev:0
            BinOp '+' -> tab_index:0, type:INTEGER, lev:0
            Var('i') -> tab_index:46, type:INTEGER, lev:0
            Literal(1) -> type:INTEGER, lev:0
For
    Assign('i' := ...)
        Var('i')
        Literal(10) -> type:INTEGER, lev:0
    Limit(downto)
        Literal(1) -> type:INTEGER, lev:0
    Block -> block_index:0, lev:0
        Assign('i' := ...)
            Var('i') -> tab_index:46, type:INTEGER, lev:0
            BinOp '-' -> tab_index:0, type:INTEGER, lev:0
            Var('i') -> tab_index:46, type:INTEGER, lev:0
            Literal(1) -> type:INTEGER, lev:0
While
    Condition
        BinOp '>' -> type:BOOLEAN, lev:0
        Var('i') -> tab_index:46, type:INTEGER, lev:0
        Literal(0) -> type:INTEGER, lev:0
    Block -> block_index:0, lev:0
        Assign('i' := ...)
            Var('i') -> tab_index:46, type:INTEGER, lev:0
            BinOp '-' -> tab_index:0, type:INTEGER, lev:0
            Var('i') -> tab_index:46, type:INTEGER, lev:0
            Literal(1) -> type:INTEGER, lev:0
Repeat
    Block -> block_index:0, lev:0
        Assign('i' := ...)
            Var('i') -> tab_index:46, type:INTEGER, lev:0
            BinOp '+' -> tab_index:0, type:INTEGER, lev:0
            Var('i') -> tab_index:46, type:INTEGER, lev:0
            Literal(1) -> type:INTEGER, lev:0
    Condition
        BinOp '=' -> type:BOOLEAN, lev:0
        Var('i') -> tab_index:46, type:INTEGER, lev:0
        Literal(10) -> type:INTEGER, lev:0
Case
    Expression
        Var('i') -> tab_index:46, type:INTEGER, lev:0
    CaseBlock
        Assign('i' := ...)
            Var('i') -> tab_index:46, type:INTEGER, lev:0
            Literal(100) -> type:INTEGER, lev:0
    CaseBlock

```

```
Assign('i' := ...)
  Var('i') -> tab_index:46, type:INTEGER, lev:0
  Literal(200) -> type:INTEGER, lev:0
```

3.5 Test Case 5

Pengujian kasus tidak valid karena penggunaan tipe dan subprogram yang tidak dideklarasikan dan kesalahan *assignment*.

input-5.txt

```
program illegalUse;

var
  a, b: integer;
  c : TPoint;

begin
  a := 90 / 5;
  b := a * a;
  solve(a, b);
  c.x := 0;
  c.y := 0;
  writeln('hi dunya!');

end.
```

output terminal

```
Semantic error: Tipe tidak ditemukan: 'TPoint'
Semantic error: Tipe assignment tidak kompatibel: lhs adalah
integer, rhs adalah real
Semantic error: Prosedur/fungsi tidak ditemukan: 'solve'
Semantic error: Akses field '.' hanya bisa dilakukan pada tipe
Record, ditemukan: notype
Semantic error: Tipe assignment tidak kompatibel: lhs adalah
notype, rhs adalah integer
Semantic error: Akses field '.' hanya bisa dilakukan pada tipe
Record, ditemukan: notype
Semantic error: Tipe assignment tidak kompatibel: lhs adalah
notype, rhs adalah integer
```

Kesimpulan dan Saran

4.1 Kesimpulan

Pada pengerjaan Milestone 3 Tugas Besar TBFO ini, kami berhasil untuk mengimplementasikan semantic analyzer yang mampu melakukan traversal Abstract Syntax Tree (AST) menggunakan pendekatan visitor pattern dan algoritma Depth First Search (DFS). Adapun hasil dari proses ini adalah Decorated AST yang dilengkapi dengan informasi semantiknya yang meliputi tipe data, level lexical, dan referensi symbol table.

Implementasi dari syntax analysis yang dihasilkan mencakup beberapa fitur penting diantaranya type checking, scope checking, declaration checking, validasi assignment, validasi parameter prosedur dan fungsi, serta pengecekan akses array dan record. Selain itu, sistem tab, btab, dan atab yang dihasilkan digunakan untuk mengelola identifier, scope, serta tipe kompleks seperti array dan subrange. Oleh karena itu, semantic analysis dapat melakukan lookup dan validasi identifier secara terstruktur dan efisien.

Secara keseluruhan, milestone ini sudah berhasil diimplementasikan secara lengkap hingga dilengkapi dengan semantic error detection, seperti undeclared variable, incompatible assignment, redeclaration, dan invalid parameter tanpa langsung mengentikan proses. Melalui pendekatan ini program mampu menampilkan seluruh kesalahan dan memudahkan kami ketika melakukan proses debugging. Implementasi milestone 3 ini juga sudah diintegrasikan dengan lexer dan parser pada milestone sebelumnya sehingga membentuk sebuah program yang lengkap dalam melakukan compiling program hingga ke tahap semantic analysis.

4.2 Saran

Untuk pengembangan selanjutnya pada Milestone 4 dan 5 yang berkaitan dengan Intermediate Code Generator dan Interpreter, diharapkan implementasi dapat berjalan dengan lancar serta diselesaikan dengan tepat waktu. Struktur semantic analysis yang telah dibangun pada milestone ini, seperti Decorated AST, symbol table, dan type checking, diharapkan dapat menjadi fondasi yang baik untuk mendukung proses pengembangan tahap berikutnya agar integrasi antar milestone tetap terjaga dengan baik.

Selain itu, kami juga berharap pengerjaan milestone terakhir nanti dapat selesai sebelum pelaksanaan UAS nantinya sehingga proses pengembangan compiler dapat diselesaikan dengan baik dan kami bisa fokus pada persiapan UAS. Dengan demikian, seluruh milestone dapat diselesaikan secara optimal dan memberikan hasil implementasi maupun evaluasi tugas besar yang terbaik.

Lampiran

1. Link Repository Github

<https://github.com/keirzka/AMN-Tubes-IF2224-2026.git>

2. Pembagian Tugas

Nama / NIM	Deskripsi Tugas	Persentase Kontribusi
13524071 Kalyca Nathania Benedicta Manullang	Mengerjakan implementasi type checker dan visitor declaration	25%
	Mengerjakan laporan bagian hasil implementasi	
13524073 Keisha Rizka Syofyani	Mengerjakan implementasi visitor expression	25%
	Mengerjakan laporan bagian landasan teori, kesimpulan, saran, merapikan laporan, dan finishing	
13524087 Muhammad Fakhry Zaki	Mengerjakan visitor statement, control flow, dan output	25%
	Melakukan testing dan debugging program	
	Mengerjakan laporan bagian testing	
13524109 Helena Kristela Sarhawa	Mengerjakan implementasi symbol table dan node	25%
	Mengerjakan laporan bagian perancangan program	